

LSM学习圣经：一次穿透10WQPS/PB级海量存储HBase/RocksDB的底层LSM结构

LSM tree 是很多数据库内部的核心数据结构，包括BigTable、ClickHouse、Cassandra、Scylla、RocksDB、HBase。

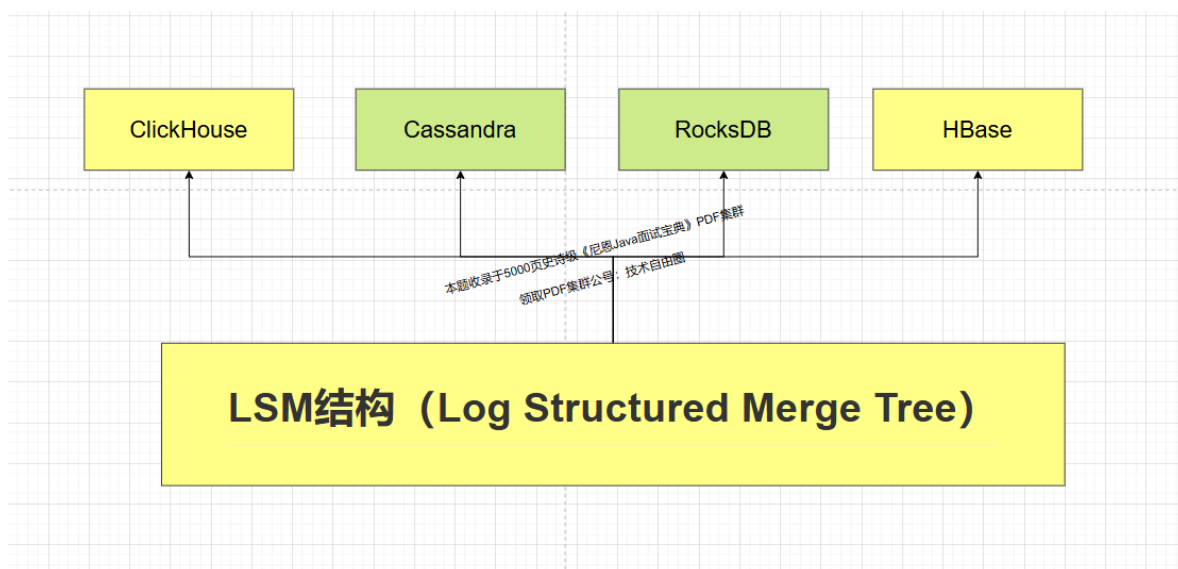
ClickHouse基于Log-Structured Merge-Tree 结构（思想），实现磁盘的顺序写入，和数据的预排序。

Cassandra 是一个使用 LSM 树作为内部数据结构，可以轻松存储万亿级日志，具体请参见：

[阿里2面：万亿级消息，如何做存储设计？](#)

RocksDB是 Facebook 开源的一个高性能持久化 KV 存储，越来越多的新生代数据库，都不约而同地选择 RocksDB 作为它们的存储引擎。

前几天时候尼恩辅导一个字节的小伙伴改造简历，得知他们内部使用的可以持久化的自研分布式 Redis，就是基于 RocksDB 做的二次架构。



回到工业级的场景：百亿级数据存储架构，只有分库分表吗？

很多的小伙伴来咨询尼恩，百亿级数据存储怎么架构，说他们的面试中，都遇到的。

他们回到回答了分库分表，比如，当一个表（比如t_order）达到500万条或2GB时，需要考虑水平分表。

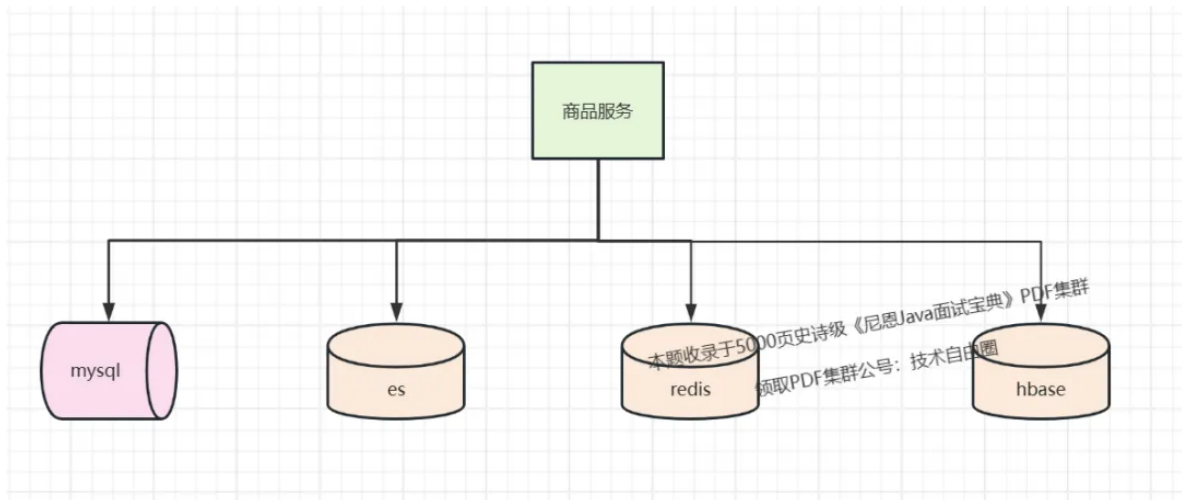
然后面试官，不满意。很多的小伙伴来咨询尼恩，为什么？

这里，尼恩用20年的技术功力，给大家做一个彻底性、系统化梳理，帮助大家吊打面试。

从0到1，百亿级数据存储架构，怎么设计？

咱们的生产需求上，百亿级数据存储架构，一般来说，需要具备以下四个能力：

- 高并发的在线ACID事务， 这里需要用到 分库分表
- 高并发的在线搜索， 这里需要用到 ElasticSearch
- 海量数据的离线处理， 这里需要用到 HBase
- 冗余表双写能力 （不同业务维度的副本）
- 把商品数据冗余存储在HBase 中， 实现海量数据的离线处理, 同时也具备高速访问的能力



从0到1, 百亿级数据存储架构, 怎么设计? 这里尼恩架构团队, 给大家录制了 **5章核心架构视频**

第37章: 百亿级数据存储架构: MYSQL双写 + HBASE +Flink +ES综合大实操

详情 参见 视频: https://mp.weixin.qq.com/s/M1CU_KTgEAtgguSveU6iOQ

第37章： 百亿级数据存储架构： MYSQL双写 + HABSE +Flink +ES综合大实操

- 1: 100亿级海量存储架构与实操视频的重要来源、重要价值、学习要点
- 2: 架构师视角解读:学习技术的两大绝招--按图索骥、顺藤摸瓜
- 3: 架构视角解读:100万级、1亿级、100亿级数据存储架构的演进
- 4: 核心面试解读:字节面试，百亿级存储，怎么设计?只是分库分表?
- 5: 架构核心要点:100亿数据，如何做mysql存储架构，需要多少的分表分库?
- 6: 核心面试解读:美团面试:百亿级分片，如何设计基因算法?
- 7: 架构视角解读:冗余表架构，解决百亿级数据的不同业务维度处理的全路由问题
- 8: 核心重点实操:用户订单、商户订单冗余表的同步双写、异步双写实操
- 9: 100亿级海星异构存储架构: es +hbase 完美组合
- 10: 顶奢解密:5大核心数据结构:红黑树、跳表、B+树、LSM-tree、前缀树
- 11: 顶奢解密:5大核心数据结构中的两大内存友好的数据结构--红黑树、跳表
- 12: 顶奢解密:5大核心数据结构中的两大磁盘友好的数据结构--B+树、LSM-tree
- 13: 100亿级存储技术选型:Apache Cassandra、MongoDB、HBase 和 ClickHouse 如何 技术选型
- 14: 100亿级存储技术选型:Apache Cassandra、MongoDB、HBase 和 ClickHouse 如何 技术选型2
- 15: 顶奢解密:4大nosql的底层结构:Cassandra、MongoDB、HBase 和 ClickHouse 存储结构和索引结构
- 16: 顶奢架构:10亿-100亿数据，如何设计ES集群架构，设计多少个es分片，多少个es节点合适
- 17: 顶奢架构:10亿-100亿数据，如何设计HABSE集群架构，设计多少个hbase分区，多少个hbase节点
- 18: 核心实操:-键 打造 HABSE +Flink +ES +xxljob 实操环境
- 19: 核心实操:如何设计 HABSE表，来存储 order 订单和订单item 数据?

20: 核心实操:画图索骥, KAFKA+ FLINK+ HBASE 数据持久化的核心链路设计

21: 核心实操:顺藤摸瓜, KAFKA+ FLINK+ HBASE 数据持久化的核心代码实操

22: 核心实操:画图索骥, KAFKA+ FLINK+ES 数据持久化的核心链路设计

23: 核心实操:顺藤摸瓜, KAFKA+ FLINK+ES 数据持久化的核心代码实操

24: 核心实操:顺藤摸瓜, 100亿数据HBASE+ES 完美组合高并发查询核心代码实操

25: 核心实操:使用xxl-job 定时任务, 保证多个副本的最终一致性

第38章: ElasticSearch 学习圣经: 从0到1, 精通 ElasticSearch 工业级实操

详情 参见 视频: https://mp.weixin.qq.com/s/M1CU_KTgEAtgguSveU6iOQ

第40章：HBASE学习圣经：从0到1, 精通HBASE的底层原理和实操

- 1: 入门学习:HBase简介和使用场景
- 2: 入门学习:HBase安装和配置
- 3: 入门学习:Hbase常用的shell 命令
- 4: 入门学习:springboot整合hbase 客户端访问
- 5: 进阶学习:HBase架构、数据模型
- 6: 进阶学习:hbase内部实现底层原理，读取数据流程
- 7: 进阶学习:hbase内部实现底层原理，写入数据流程
- 8: 高阶学习:Hbase优化设计，如何对rowKey进行优化设计？
- 9: 高阶学习:Hbase优化设计，内存设计和压缩算法选择
- 10: 高阶学习:Hbase优化设计，布隆过滤器、索引优化
- 11: 高阶学习:Hbase的应用案例

第39章：XXL-JOB学习圣经：从0到1, 精通 xxl-job工业级实操

详情 参见 视频：https://mp.weixin.qq.com/s/M1CU_KTgEAtgguSveU6iOQ

第39章：XXL-JOB学习圣经：从0到1，精通 xxl-job工业级实操

- 1: 入门学习:什么是xxl-job?
- 2: 入门学习:xxl-job的部署
- 3: 入门学习:开发第一个xxl-job调度任务
- 4: 进阶学习:xxl-job大规模任务分片调度
- 5: 源码学习:xxl-job系统总体架构
- 6: 源码学习:xxl-job服务器启动流程-总体流程
- 7: 源码学习:服务器启动流程-初始化注册线程池-剔除超时主机
- 8: 源码学习:服务器启动流程-失败重试和告警，初始化回调线程池
- 9: 源码学习:time-wheel 时间轮算法，以及时间轮调度
- 10: 源码学习:xxl-job客户端启动流程、注册流程
- 11: 源码学习:xxl-job服务端下发任务的流程
- 12: 源码学习:xxl-job客户端任务执行流程

第40章：HBASE学习圣经：从0到1，精通 HBASE的底层原理和实操

详情 参见 视频：https://mp.weixin.qq.com/s/M1CU_KTgEAtgguSveU6iOQ

第40章：HBASE学习圣经：从0到1, 精通HBASE的底层原理和实操

- 1: 入门学习:HBase简介和使用场景
- 2: 入门学习:HBase安装和配置
- 3: 入门学习:Hbase常用的shell 命令
- 4: 入门学习:springboot整合hbase 客户端访问
- 5: 进阶学习:HBase架构、数据模型
- 6: 进阶学习:hbase内部实现底层原理，读取数据流程
- 7: 进阶学习:hbase内部实现底层原理，写入数据流程
- 8: 高阶学习:Hbase优化设计，如何对rowKey进行优化设计？
- 9: 高阶学习:Hbase优化设计，内存设计和压缩算法选择
- 10: 高阶学习:Hbase优化设计，布隆过滤器、索引优化
- 11: 高阶学习:Hbase的应用案例

第41章：Flink学习圣经：从0到1, 精通 Flink 的底层原理和实操

详情 参见：https://mp.weixin.qq.com/s/M1CU_KTgEAtgguSveU6iQQ

第41章：Flink学习圣经：从0到1, 精通Flink 的底层原理和实操

1: 大数据及flink

2: flink安装及任务调度

3: flink批处理与流处理

4: flink架构和执行流程

5: flink高级开发(1)

6: flink高级开发(2)

7: flink窗口计算及table

后续版本: 更多的跟着源码学架构实操项目规划中

HBase 的底层结构，恰恰是LSM。所以，咱们必须首先搞定LSM。

LSM结构（Log Structured Merge Tree）的使用场景

log-structured merge-tree (LSM tree) 是一种被精心设计的数据结构，常用于处理大量写入的场景。

通过对写入操作进行顺序写入优化实现性能提升。

Hbase 适合存储 PB 级别的海量数据，在 PB 级别的数据以及采用廉价 PC 存储的情况下，能在几十到百毫秒内返回数据。这与 Hbase 的极易扩展性息息相关。正式因为 Hbase 良好的扩展性，才为海量数据的存储提供了便利。

1. 列式存储

这里的列式存储其实说的是列族存储，Hbase 是根据列族来存储数据的。列族下面可以有非常多的列，列族在创建表的时候就必须指定。

2. 极易扩展

Hbase 的扩展性主要体现在两个方面，一个是基于上层处理能力（RegionServer）的扩展，一个是基于存储的扩展（HDFS）。通过横向添加 RegionServer 的机器，进行水平扩展，提升 Hbase 上层的处理能力，提升 Hbase 服务更多 Region 的能力。备注：RegionServer 的作用是管理 region、承接业务的访问，这个后面会详细的介绍通过横向添加 Datanode 的机器，进行存储层扩容，提升 Hbase 的数据存储能力和提升后端存储的读写能力。

3. 高并发

由于目前大部分使用 Hbase 的架构，都是采用的廉价 PC，因此单个 IO 的延迟其实并不小，一般在几十到上百 ms 之间。这里说的高并发，主要是在并发的情况下，Hbase 的单个 IO 延迟下降并不多。能获得高并发、低延迟的服务。

4. 稀疏

稀疏主要是针对 Hbase 列的灵活性，在列族中，你可以指定任意多的列，在列数据为空的情况下，是不会占用存储空间的。

LSM结构 简介

全称 Log-Structured Merge-Tree 日志结构合并树，但不是树,它是利用了磁盘顺序读写能力，实现了一个多层的存储结构

1996年，一篇名为 The log-structured merge-tree (LSM-tree) 的论文创造性地提出了**日志结构合并树**（Log-Structured Merge-Tree）的概念，该方法既吸收了日志结构方法的优点，又通过将数据文件预排序克服了日志结构方法随机读性能较差的问题。

尽管当时 LSM-tree 新颖且优势鲜明，但它真正声名鹊起却是在 10 年之后的 **2006年**，2006 年，Google 发表了 BigTable 的论文。这篇论文提到 BigTable 单机上所使用的数据结构就是 LSM。

那年谷歌的一篇使用了 LSM-tree 技术的论文 Bigtable: A Distributed Storage System for Structured Data 横空出世，在分布式数据处理领域掀起了一阵旋风，

随后两个声名赫赫的大数据开源组件（2007 年的 HBase 与 2008 年的 Cassandra，目前两者同为 Apache 顶级项目）直接在其思想基础上破茧而出，彻底改变了大数据基础组件的格局，同时也极大地推广了 LSM-tree 技术。

目前，LSM 被很多存储产品作为存储结构，比如 Apache HBase, Apache Cassandra, MongoDB 的 Wired Tiger 存储引擎, LevelDB 存储引擎, RocksDB 存储引擎等。

简单地说，**LSM 的设计目标：是提供比传统的 B+ 树更好的写性能。**

LSM 通过将磁盘的**随机写入转化为顺序写入**来提高写性能，而付出的代价就是牺牲部分读性能、写放大（B+ 树同样有写放大的问题）。

LSM-tree 最大的特点是同时使用了两部分类树的数据结构来存储数据，并同时提供查询。

其中一部分数据结构（C0 树）存在于内存缓存（通常叫作 memtable）中，负责接受新的数据插入更新以及读请求，并直接在内存中对数据进行排序；

另一部分数据结构（C1 树）存在于硬盘上（这部分通常叫作 sstable），它们是由存在于内存缓存中的 C0 树冲写到磁盘而成的，主要负责提供读操作，特点是有序且不可被更改。

LSM 相比 B+ 树能提高写性能的本质原因是：

外存——无论磁盘还是 SSD，其随机读写都要慢于顺序读写。

Hash表、二叉树，到B树和B+树，LSM树

Hash表和B+树

在了解LSM树之前，我们需要对hash表和B+树有所了解。

hash表通过key值经过hash算法，直接定位到数据存储地址，然后取出value值。

时间复杂度 $O(1)$ ，找数据和存数据就需要那么一下子，就给找到了

hash存储方式**支持增、删、改以及随机读取操作**，但不支持**顺序扫描**，对应的存储系统为key-value存储系统。

对于key-value的插入以及查询，哈希表的复杂度都是 $O(1)$ ，明显比树的操作 $O(n)$ 快，

如果不需要有序的遍历数据，哈希表就是最佳选择

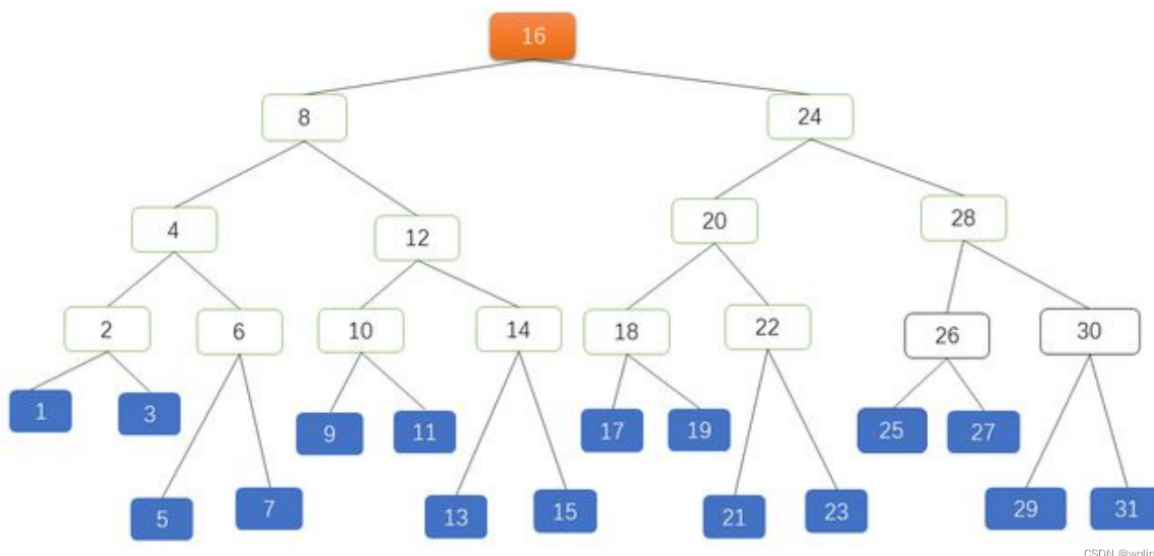
从二叉树，到B树和B+树的演进

首先从二叉树说起，一颗非常普通的树，非常容易退化为一张链表。



CSDN @架构师-尼恩

因为 二叉树 会产生退化现象，提出了平衡二叉树，

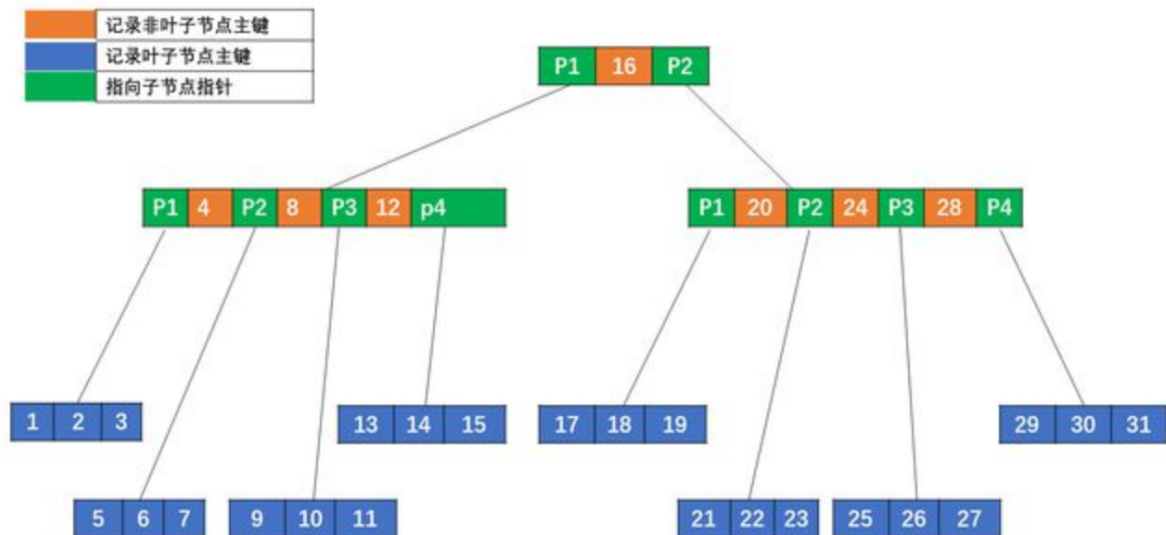


CSDN @wqjinf

在平衡二叉树基础上，需要减少遍历高度，怎样让每一层放的节点多一些数据，来引申出m叉树，

m叉搜索树同样会有退化现象，引出m叉平衡树，也就是B树，

B 树是一种多叉的 AVL 树。B-Tree 减少了 AVL 数的高度，增加了每个节点的 KEY 数量。



CSDN @架构师-尼恩

B树的问题：每个节点既放了key也放了value，怎样使每个节点放尽可能多的key值，以减少遍历高度呢（访问磁盘次数），

可以将每个节点只放key值，将value值放在叶子结点，在叶子结点的value值增加指向相邻节点指针，这就是优化后的B+树。

B+树所有叶子节点形成有序链表，便于**范围查询**，不用每次要检索树。

目前数据库多采用两级索引的B+树，树的层次最多三层，

因此可能需要5次磁盘访问才能更新一条记录（三次磁盘访问获得数据索引以及行id，然后再进行一次数据文件读操作及一次数据文件写操作）

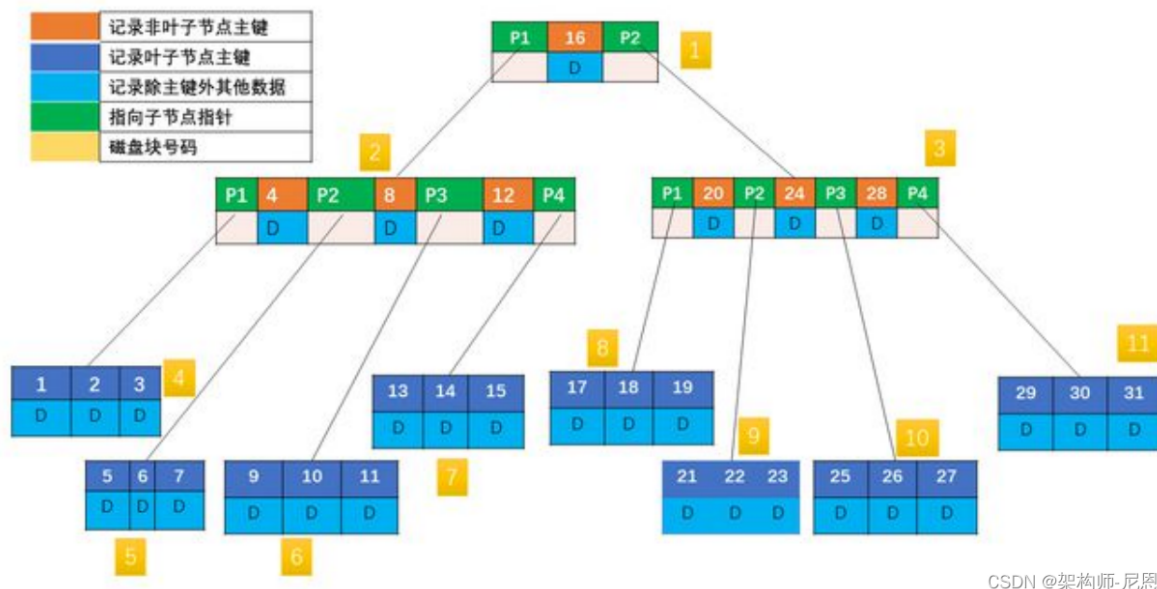
B~树(平衡多路二叉树)

B~树，又叫平衡多路查找树。一棵m阶的B~树 (m叉树)的特性如下：

- 1) 树中每个结点至多有m个孩子；
- 2) 除根结点和叶子结点外，其它每个结点至少有 $\lceil m/2 \rceil$ 个孩子；
- 3) 若根结点不是叶子结点，则至少有2个孩子；
- 4) 所有叶子结点都出现在同一层，叶子结点不包含任何关键字信息(可以看做是外部接点或查询失败的接点，实际上这些结点不存在，指向这些结点的指针都为null)；
- 5) 每个非终端结点中包含有n个关键字信息：(n, A0, K1, A1, K2, A2,, Kn, An)。

其中，

- a) K_i ($i=1\dots n$)为关键字，且关键字按顺序排序 $K_i < K_{(i-1)}$ 。
- b) A_i 为指向**子树根的接点**，且指针 $A_{(i-1)}$ 指向子树种所有结点的关键字均小于 K_i ，但都大于 $K_{(i-1)}$ 。
- c) 关键字的个数n必须满足： $\lceil m/2 \rceil - 1 \leq n \leq m - 1$



CSDN @架构师-尼恩

实际应用中，每个节点的最小单元不是 KEY，而一般是按照块（BLOCK）来算。

比如磁盘文件系统 EXT4 每块 4KB；数据库比如 PostgreSQL 是 8KB，MySQL InnoDB 是 16KB，MySQL NDB 是 32KB 等。

上图每个节点的基本单元是一个磁盘块（BLOCK，默认 4KB），根节点含有一个键值，其他节点含有 3 个键值，每个磁盘块包含对应的键值与数据。

比如现在要读取 KEY 为 31 的记录：先找到根节点磁盘块（1），读入内存。（第一次 IO）；

关键字 31 大于区间（16，），根据指针 P2 找到磁盘块 3，读入内存（第二次 IO）；

31 大于区间（20，24，28），根据指针 P4 读取磁盘块 11（第三次 IO），在磁盘块 11 中找到 KEY 为 31 的记录，返回结果。

三次 IO，前两次 IO 其实从磁盘读取了不必要的数据，因为只用比较 KEY，所以非叶子节点对应的 DATA 完全没有必要，如果 DATA 很大，那完全是浪费内存资源。

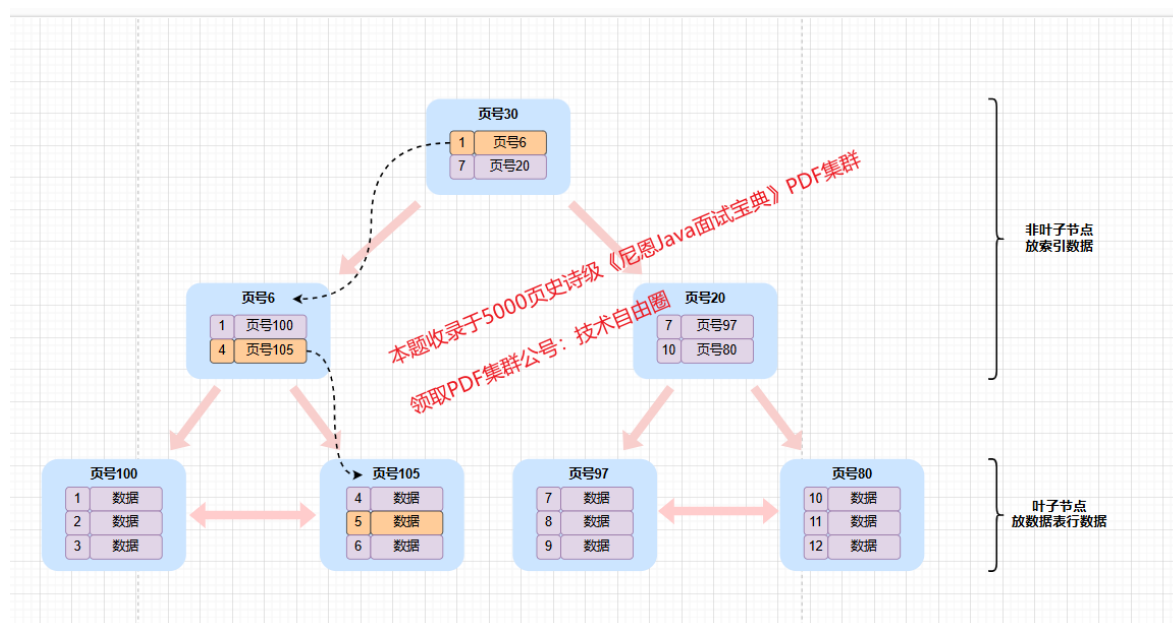
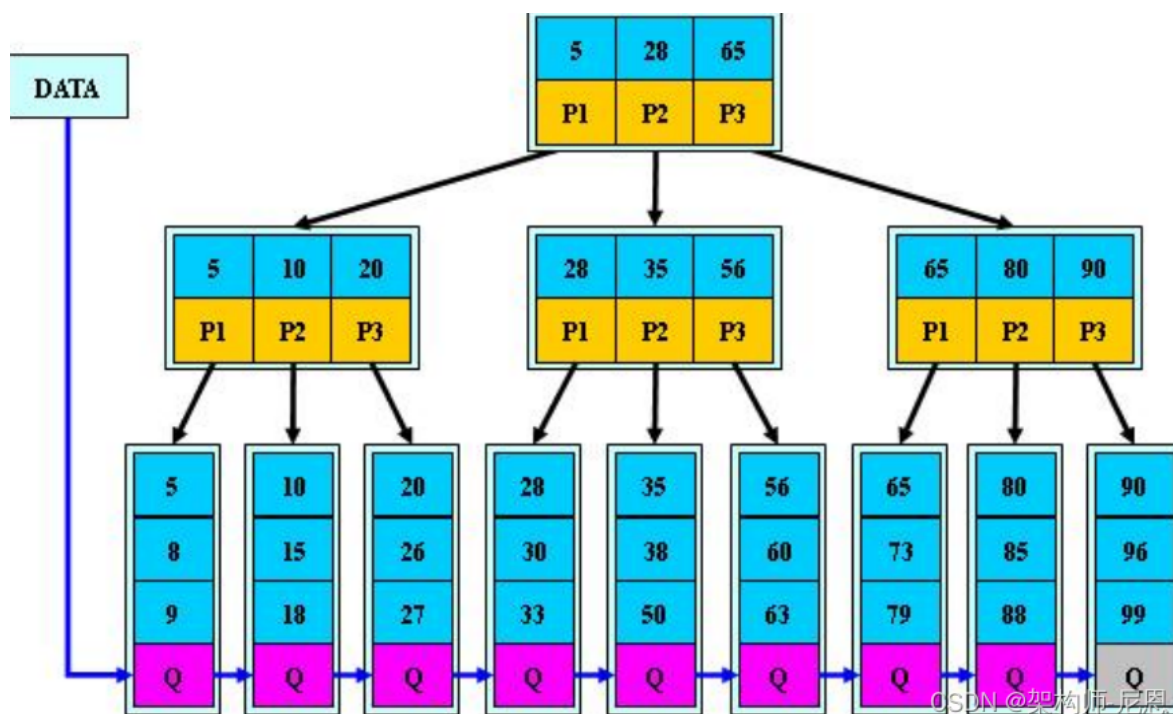
考虑下能否把非叶子节点的 DATA 拿掉？

B+树

B+树：是应文件系统所需而产生的一种B~树的变形树。

一棵m阶的B+树和m阶的B-树的差异在于：

- 1) 有n棵子树的结点中含有n个关键字；（B~树是n棵子树有n+1个关键字）
- 2) 所有的叶子结点中包含了全部关键字的信息，及指向含有这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大的顺序链接。（B~树的叶子节点并没有包括全部需要查找的信息）
- 3) 所有的非终端结点可以看成是索引部分，结点中仅含有其子树根结点中最大（或最小）关键字。（B~树的非终端结点也包含需要查找的有效信息）



非叶子节点不再包含除了主键外的数据，数据全部放在叶子节点，并且所有叶子节点存放在一个单向链表里，当然也可以双向链表。

可以看到，B+ 树同时具有平衡多叉树和链表的优点，即可兼顾 B 树对范围查找的高效，又可兼顾链表随机写入的高效，这也是大部分数据库都用 B+ 树来存储索引的原因。

(1) B+树的磁盘读写代价更低

我们都知道磁盘是可以块存储的，也就是同一个磁道上同一盘块中的所有数据都可以一次全部读取。

而B+树的内部结点并没有指向关键字具体信息的指针

比如文件内容的具体地址，比如说不包含B+树结点中的FileHardAddress[filenum]部分。

因此其内部结点相对B-树更小。如果把所有同一内部结点的关键字存放在同一盘块中，那么盘块所能容纳的关键字数量也越多。

这样，一次性读入内存中的需要查找的关键字也就越多。相对来说IO读写次数也就降低了。

举个例子，假设磁盘中的一个盘块容纳16bytes，而一个关键字2bytes，一个关键字具体信息指针2bytes。

一棵9阶B~树(一个结点最多8个关键字)的内部结点需要2个盘快。而B+树内部结点只需要1个盘快。

当需要把内部结点读入内存中的时候，B~树就比B+数多一次盘块查找时间(在磁盘中就是盘片旋转的时间)。

(2) B+树的查询效率更加稳定。

由于非终结点并不是最终指向文件内容的结点，而只是叶子结点中关键字的索引。

所以任何关键字的查找必须走一条从根结点到叶子结点的路。

所有关键字查询的路径长度相同，导致每一个数据的查询效率相当。

B-tree和B+树的区别：

比较项	B树	B+树
指针	所有内部和叶节点都有数据指针	只有叶节点有数据指针
搜索	由于并非所有键都在叶中可用，因此搜索通常需要更多时间。	所有的键都在叶节点，因此搜索更快更准确。
冗余键	树中没有保留键的副本。	保留密钥的副本，并且所有节点都存在于叶子中。
插入	插入需要更多时间，而且有时无法预测。	插入更容易，结果始终相同。
删除	内部节点的删除非常复杂，树必须经历很多变换。	删除任何节点都很容易，因为所有节点都在叶子上找到。
叶节点	叶节点不存储为结构链表。	叶节点存储为结构链表。
使用权	无法顺序访问节点	可以像链表一样顺序访问
高度	对于特定数量的节点高度较大	对于相同数量的节点，高度小于 B 树
应用	用于数据库、搜索引擎的 B 树	B+ 树用于多级索引、数据库索引
节点数	任何中间层 'l' 的节点数是 2 l。	每个中间节点可以有 n/2 到 n 个子节点。

B+树的不足，与LSM树诞生背景

传统关系型数据库使用btree或一些变体作为存储结构，能高效进行查找。

但保存在磁盘中时它也有一个明显的缺陷，那就是逻辑上相离很近但物理却可能相隔很远，这就可能造成大量的磁盘随机读写。

随机读写比顺序读写慢很多，为了提升IO性能，我们需要一种能将随机操作变为顺序操作的机制，于是便有了LSM树。

为啥 随机读写比顺序读写慢很多呢？

磁盘读写时涉及到磁盘上数据查找，地址一般由柱面号、盘面号和块号三者构成。

也就是说：

step1：移动臂先根据柱面号移动到指定柱面，

step2：然后根据 盘面号 确定盘面

step3：最后 块号 确定磁道，最后将指定的磁道段移动到磁头下，便可开始读写。

整个过程主要有三部分时间消耗，查找时间(seek time) +等待时间(latency time)+传输时间(transmission time)。分别表示**定位柱面的耗时**、将块号指定 **磁道段 移到磁头**的耗时、将**数据传到内存**的耗时。

整个磁盘IO最耗时的地方在查找时间，所以减少查找时间能大幅提升性能。

基于 B+ 树的 传统存储引擎，是为旋转磁盘设计的，写入速度很慢，并且只提供基于块的寻址。

B+ 树的 传统存储引擎，是为旋转磁盘设计的

磁盘读写时涉及到磁盘上数据查找，地址一般由柱面号、盘面号和块号三者构成。

也就是说移动臂先根据柱面号移动到指定柱面，然后根据盘面号确定盘面的磁道，最后根据块号将指定的磁道段移动到磁头下，便可开始读写。

整个过程主要有三部分时间消耗，查找时间(seek time) +等待时间(latency time)+传输时间(transmission time)。分别表示定位柱面的耗时、将块号指定磁道段移到磁头的耗时、将数据传到内存的耗时。

整个磁盘IO最耗时的地方在查找时间，所以减少查找时间能大幅提升性能。

关于磁盘IO这一部分，其实要区别看待。

如果采用的是ssd，那么对于任意地址而言，其实寻址时间可以认为是固定的，它与最传统的chs以及lba寻址方式不一样。如果是ssd的话，随机读写和顺序读写，开销差距大吗？

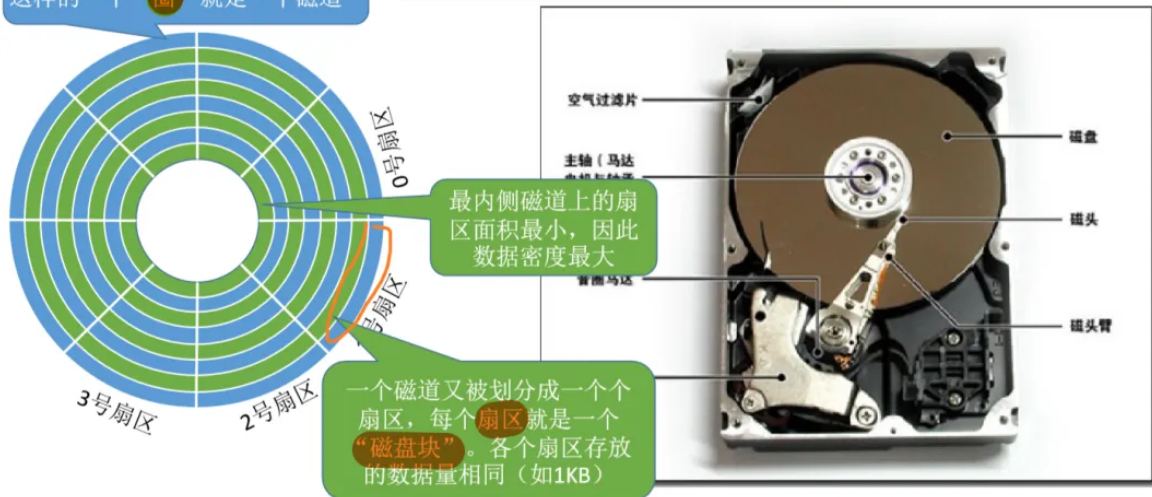
HDD机械硬盘的扇区 (sector) 结构

机械硬盘的性能为啥那么慢？看看结构就知道：

磁盘、磁道、扇区

磁盘的盘面被划分成一个个磁道。这样的一个个“圈”就是一个磁道

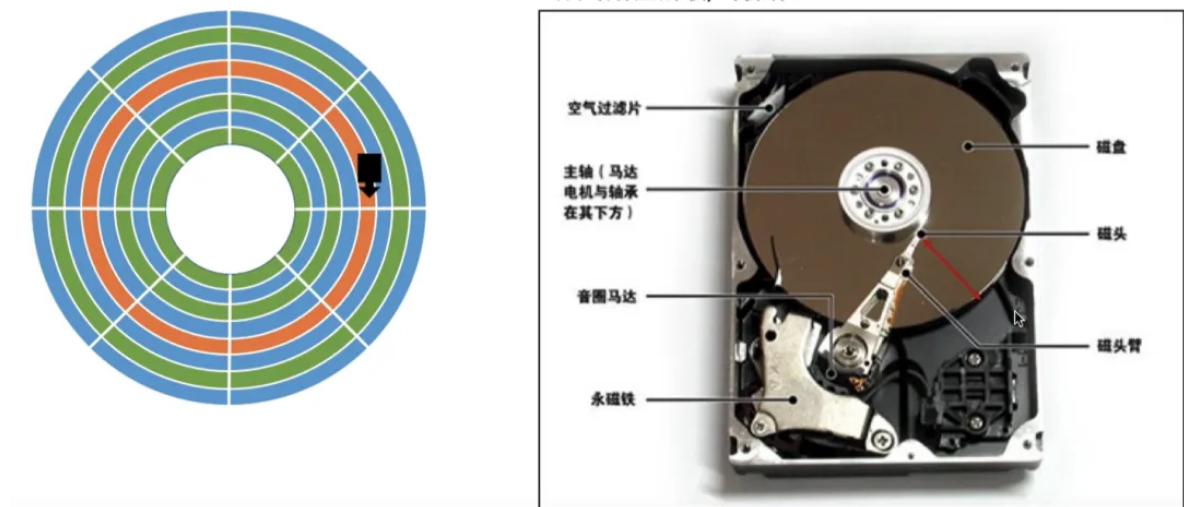
磁盘的表面由一些磁性物质组成，可以用这些磁性物质来记录二进制数据



机械磁盘上的每个磁道被等分为若干个弧段，这些弧段称之为扇区。

如何在磁盘中读/写数据？需要物理动作，去移动“磁头”到目标扇区

需要把“磁头”移动到想要读/写的扇区所在的磁道。磁盘会转起来，让目标扇区从磁头下面划过，才能完成对扇区的读/写操作。



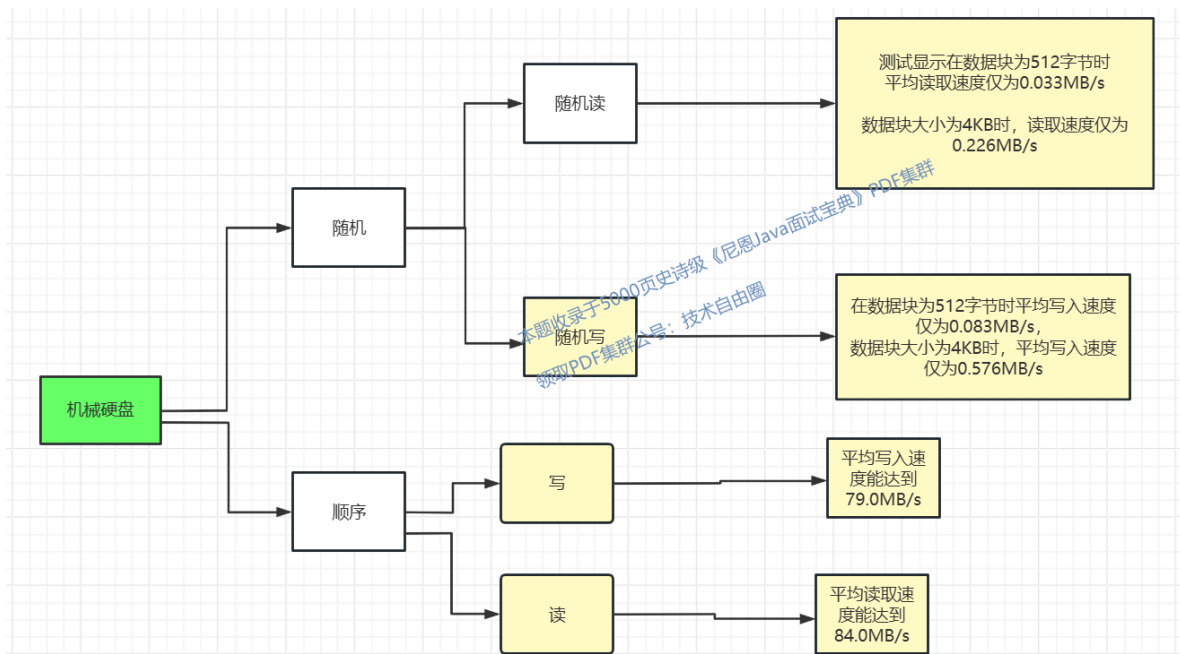
机械磁盘的读写以扇区为基本单位。

硬盘的物理读写以扇区为基本单位。通常情况下每个扇区的大小是 512 字节。

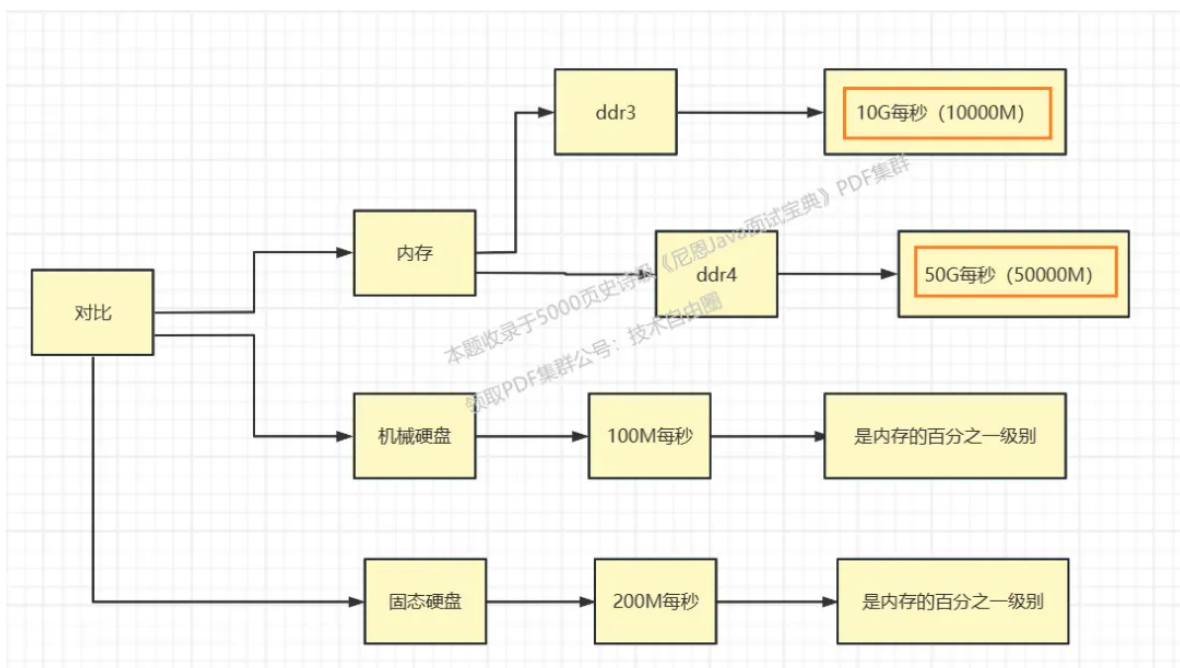
然而，今天的应用程序是写入密集型的，并且会生成大量数据。因此，需要重新考虑 DBMS 中现有存储引擎的设计。

内存和磁盘的访问速度对比

机械硬盘的读写速度，大致如下



固态硬盘的读写速度，大致如下



机械硬盘的顺序和随机读写的性能比较

机械硬盘在顺序读写场景下有相当出色的性能表现，但一遇到随机读写性能则直线下降。

顺序读是随机读性能的400倍以上。顺序读能达到84MB/S。

顺序写是随机读性能的100倍以上。顺序写性能能达到79M/S。

原因：是因为机械硬盘采用传统的磁头探针结构，随机读写时需要频繁寻道，也就需要磁头和探针频繁的转动，而机械结构的磁头和探针的位置调整是十分费时的，这就严重影响到硬盘的寻址速度，进而影响到随机写入速度。

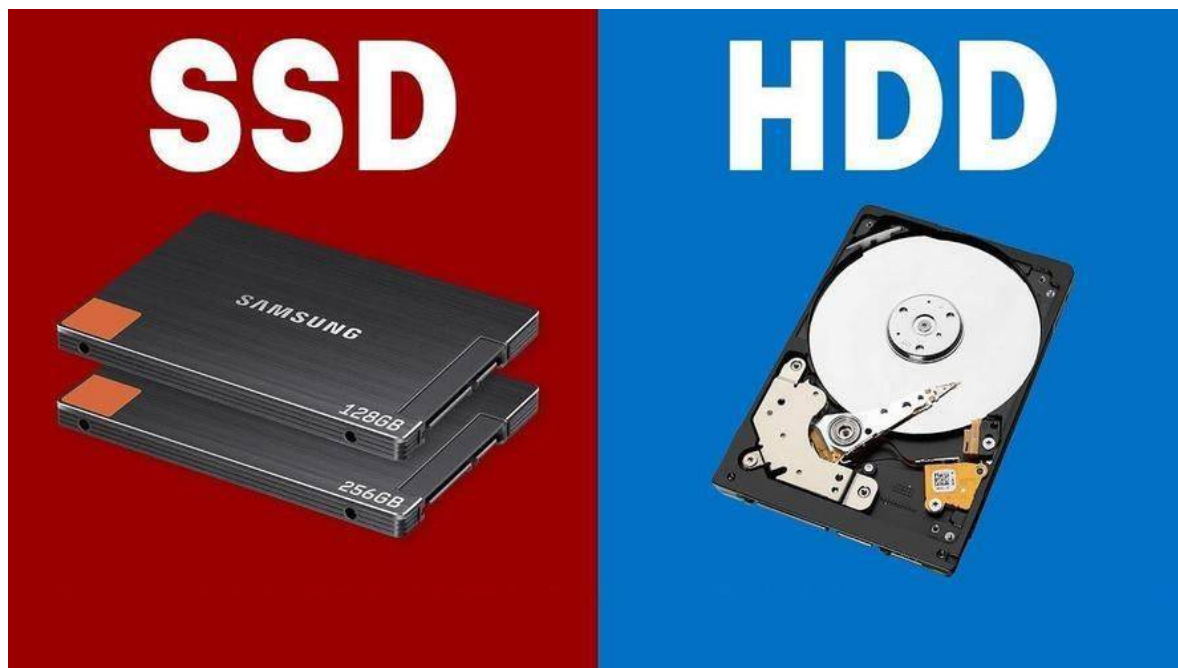
SSD固态硬盘 的顺序和随机读写的性能比较

顺序读：220.7MB/s。随机读：24.654MB/s。

顺序写：77.2MB/s。随机写：68.910MB/s。

总结：对于固态硬盘，顺序读的速度仍然能达到随机读的10倍左右。但是随机写还是顺序写，差别不大。

HDD机械硬盘和SSD固态硬盘性价比



1、固态硬盘（SSD），优点：读取速度快、抗震性强、功耗低、运行安静。缺点：价格较高、写入寿命有限、容量较小。

2、机械硬盘（HDD），优点：价格低廉、大容量、写入寿命长。缺点：读取速度慢、抗震性差、功耗高、运行噪音大。

选择适合的硬盘需考虑具体需求，如性能需求选SSD，大容量存储选HDD。

像HBASE这样的海量存储集群，一般都选择HDD。

在HDD的硬件基础上，B+树的写入性能低，那么LSM树就是为写而生。

LSM树就是为写而生。

LSM树就是为写而生。

LSM树就是为写而生。

当然，真实的海量存储中间件，也会对读进行性能优化，比如引入布隆过滤器。

布隆过滤器的事情后面再说，咱们言归正传。

什么是LSM树

LSM树，即日志结构合并树(Log-Structured Merge-Tree)。其实它并不属于一个具体的数据结构，它更多是一种数据结构的设计思想。

大多NoSQL数据库核心思想都是基于LSM来做的，或者说，几乎所有NoSQL数据库都使用LSM Tree的变体作为其底层存储引擎，因为它允许它们实现快速的写入吞吐量和对主键的快速查找。

LSM-Tree 全名叫Log-Structured Merge Tree，最早建立在日志结构文件之上，现在基于合并和压缩排序文件原理的存储引擎都统称为LSM存储引擎。

我们通常把LSM看成一种思想：**保存在后台合并的一系列SStable**

这种思想简单且有效：

- 即使数据集远大于内存，LSM-tree也能正常工作
- 由于键值有序，范围查询相比于hash表有很大优势
- 由于写入是顺序的（归并是后台线程在空闲时间做的），所以可以提供非常高的写入吞吐量

查找的性能优化：

在LSM系统中查找一个不存在的键时会导致查询时间长，因为要从最新的数据一直往前查找，所以LSM一般会使用布隆过滤器进行优化

接下来，在介绍LSM 之前，先说说 SStable

SSTables 四大操作流程

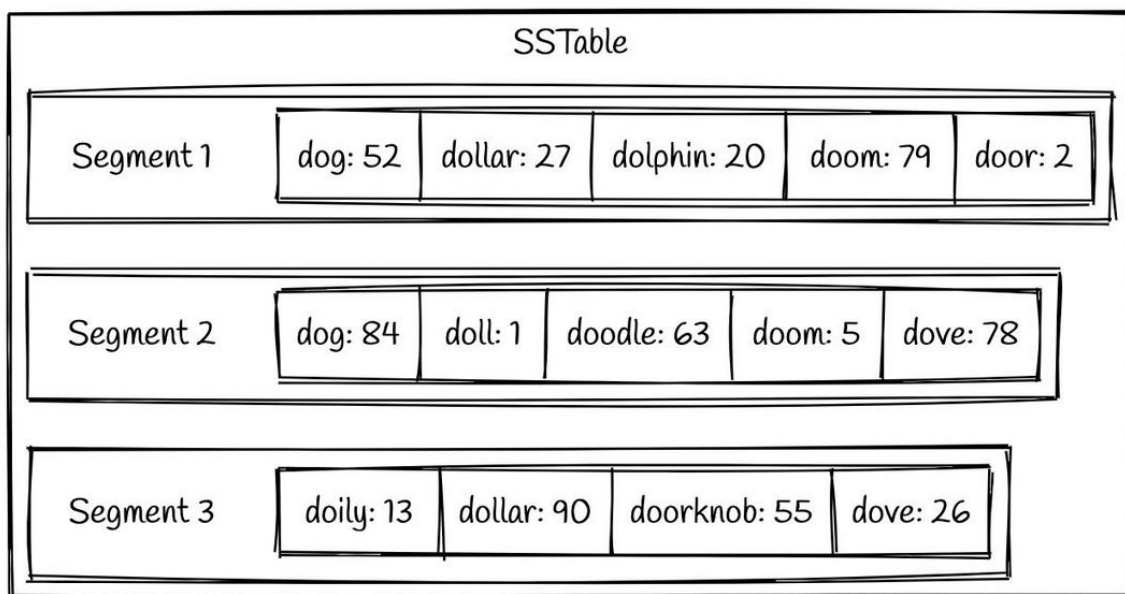
LSM tree 通过一种叫做 **SStable (Sorted Strings Table)** 的格式，持久化到硬盘上。

顾名思义，SStable 是一种用来存储有序的键值对的格式，其中**键 是有序存储的**。

一个SStable 会包括多个有序的子文件，被称为 *segment* 。

这些 *segments* 一旦被写入硬盘，就不可以再修改了。

一个简单的SStable 例子如下图所示：



我们可以看到，在每个 segment 中的键值对都是按照键的顺序有序组织的。

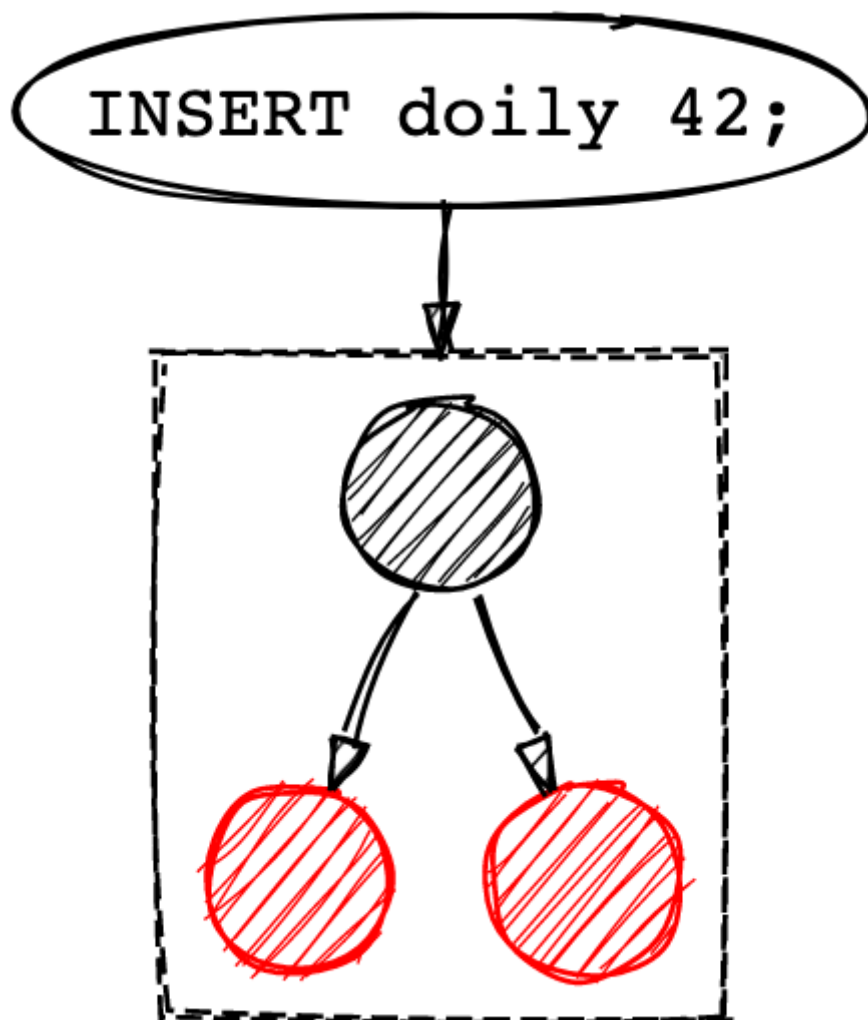
SSTables 写入数据的基本流程

由于 LSM tree 只会进行顺序写入，所以自然而然地就会引出这样一个问题：

写入的数据可能是任意顺序的，我们又如何保证数据能够保持 SStable 要求的有序组织呢？

这就需要引入新的常驻内存 (*in-memory*) 数据结构: memtable_table,

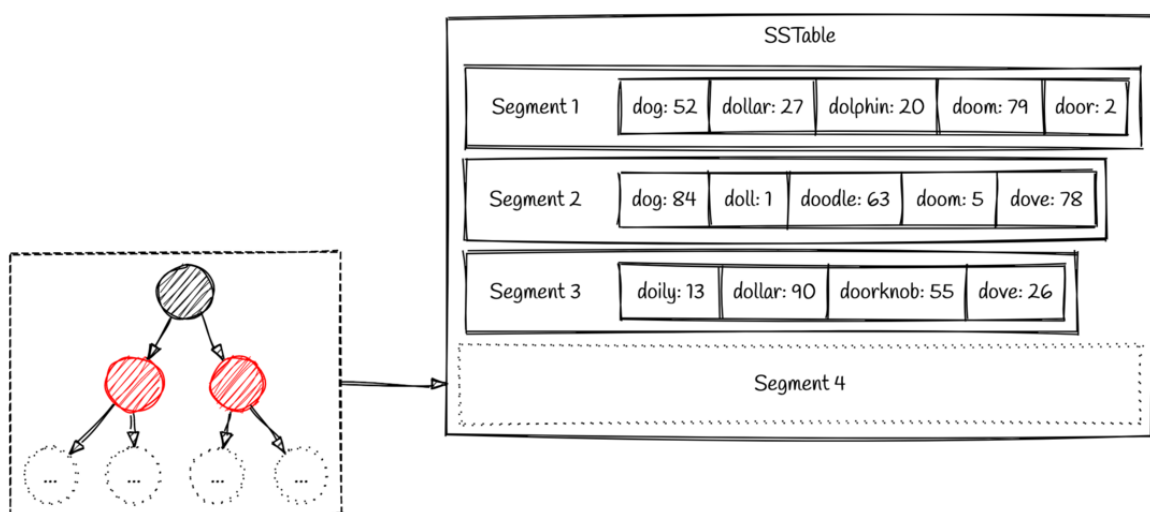
memtable_table 的底层数据结构则有点像红黑树,当由新的写入操作则将数据插入到红黑树中。



写入操作会先把数据存储到红黑树中，直至红黑树的大小达到了预先定义的大小。

一旦红黑树的大小达到阈值，就会把数据整个刷到磁盘中，这个过程就可以把数据保证有序写入了。

经过一层数据结构的承接，就可以保证单向顺序写入的同时，也能保证数据的有序了。



SSTables 读取数据的基本流程

那么我们是如何从SSTable中查找数据的呢？

一种naive的方法就是遍历所有的 segments，寻找我们需要的key。

从最新的 segment 到最老的 segment ——遍历，知道找到目标key为止。

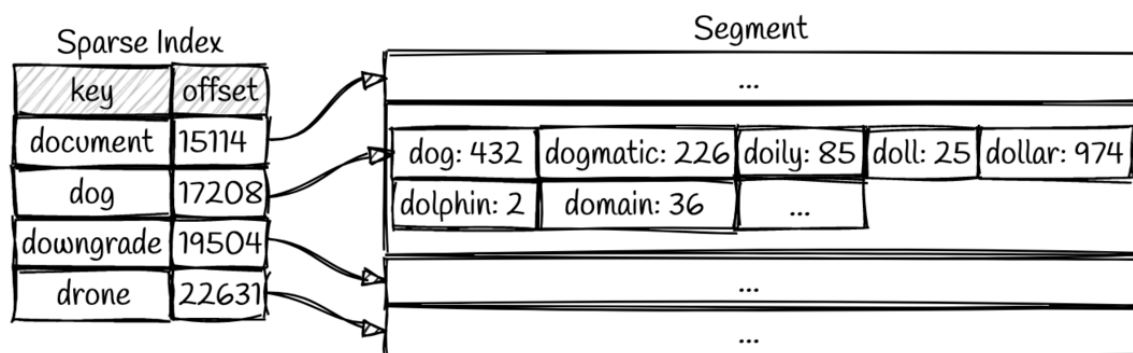
显然，这种方式在寻找刚刚写入的数据是比较快的，但是文件一多就不太行了。

针对这个问题，如何的优化？

需要引入索引：比如 跳表、比如 稀疏索引。不同的索引，根据不同设计具体选择。

先看看简单的 稀疏索引。

稀疏索引就是一种在内存中对数据检索进行优化的技术。



我们可以通过这个索引快速找到所需键的前面和后面的偏移量（就是最近的相邻值），这样我们就只需要扫描很小一部分的 segments 文件就可以了。

以如图所示的场景举例，当我们需要搜索 `dollar` 这个值，我们可以通过二分查找搜索稀疏索引，可以知道 `dollar` 处于 `dog` 和 `downgrade` 之间。

因此我们只需要搜索 17208 和 19504 之间的 segment 来得到我们所需的值，如果搜索不到则可返回未命中。

上面优化在查找存在的数据其实已经不错了，但是在搜索不存在的key值的时候还是要遍历所有的 segment 才可以确定。

为了解决这个问题，就需要引入 布隆过滤器。

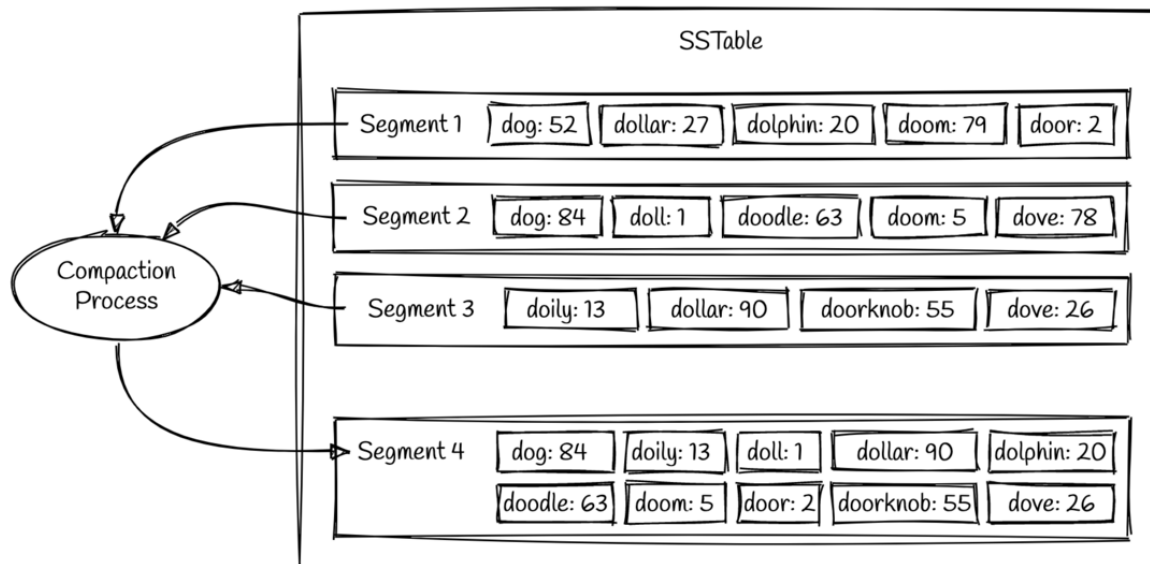
布隆过滤器是一种以空间换时间的数据结构，能够帮助我们快速确定某个值是否存在（如果布隆过滤器认为该值存在，也可能是实际不存在的）。我们可以在写入数据的时候同时更新布隆过滤器，来加速不存在数据的检索。

SSTables 数据合并的基本流程

随着时间的推移，整个存储系统将会存储非常多的segment文件，所以这些文件需要进行一定的整理和合并，避免文件太多无法访问。

这个文件整理的过程被称为“数据合并”(compaction)。

数据合并是一个后台线程，将会持续地将老的segment 合并到一起变成新的 segment。



如图所示，我们可以看到 segment 1 和 segment 2 都有 key 为 `dog` 的两个值。

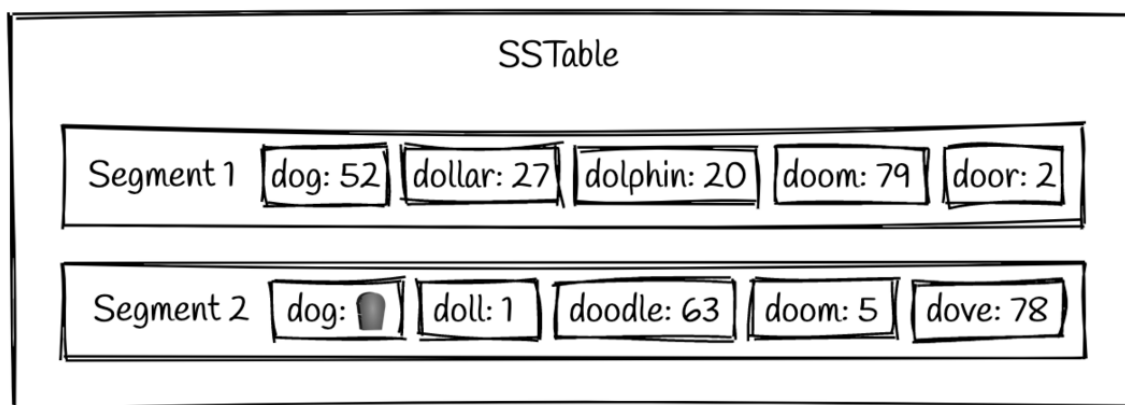
合并后的新 segment 将会保留更新的值，因此会保留原有 segment 2 里面的值 `84`，即 segment 4 中的值是 `dog => 84`。一旦合并过程已经完成新的 segment 写入，那么原有的老 segment 文件将会被删除。

SSTables 删除数据的基本流程

我们已经解释了读取数据和写入数据的过程，那么删除数据又是如何处理的呢？

我们已经知道 SSTable 是不可变的，所以里面的数据当然也不能够删除。

其实删除操作其实和写入数据的操作是一样的，当需要删除数据的时候，我们把一个特定的标记（我们称之为 **墓碑**(tombstone)）写入到这个key对应的位置，以标记为删除。



上图演示了原来 key 为 `dog` 的值为 `52`，而删除之后就会变成一个墓碑的标记。

当我们搜索键 `dog` 的时候，将会返回数据无法查询，这就意味着删除操作其实也是占用磁盘空间的，最后墓碑的值将会被压缩，最后将会从磁盘删除。

SSTables 四大操作流程总结

我们已经基本描述了 LSM tree 引擎是如何工作的：

1. 写入操作是先写入内存的（被成为 memtable）。所有的用于加速查询的数据结构（布隆过滤器和稀疏索引）都会被同时更新；
2. 当内存中的 memtable 太大了，将会被刷到磁盘中，注意是有序的；
3. 当查询时我们先回查询布隆过滤器，如果布隆过滤器返回说键不存在，则实际不存在，如果布隆过滤器说存在，进一步遍历 segment 文件；

4. 对于遍历 segment 文件的过程，我们将会先通过稀疏索引找到最小的文件范围，并开始由新到老开始遍历，找到一个key则直接返回。

上面至少一个简单的介绍，方便大家快速了解。

接下来，才正式开始介绍LSM。

LSM树原理

LSM树能让我们进行顺序写磁盘，从而大幅提升写操作的性能，作为代价的是牺牲了一些读性能。

LSM树的特点：**用牺牲读性能来换取写性能**，将对数据的修改增量保持在内存中，达到指定的大小限制后将这些修改操作批量写入磁盘

LSM树的核心思想：放弃部分读性能，提高写性能

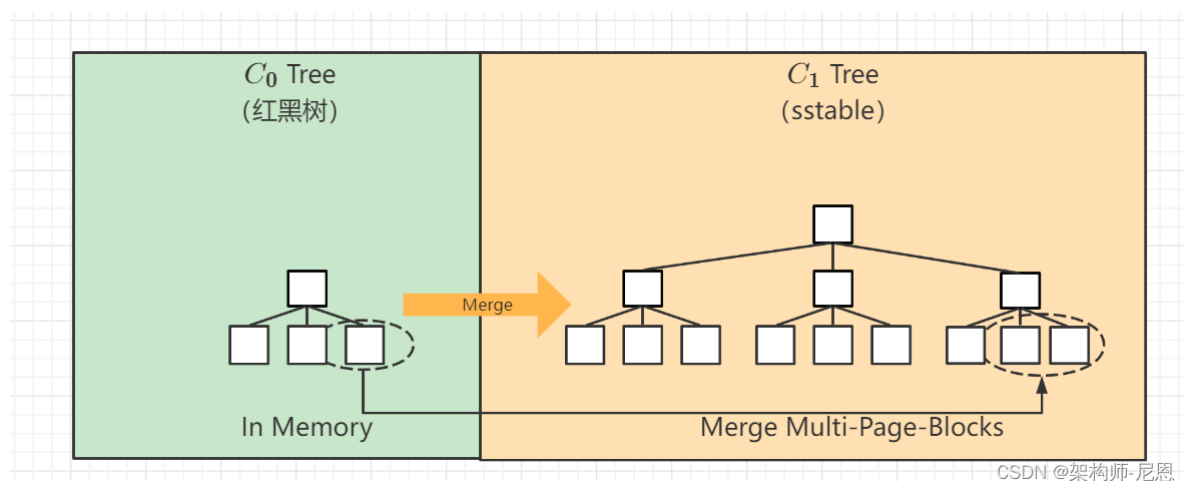
代表数据库：nessDB、LevelDB、HBase等非关系型数据库

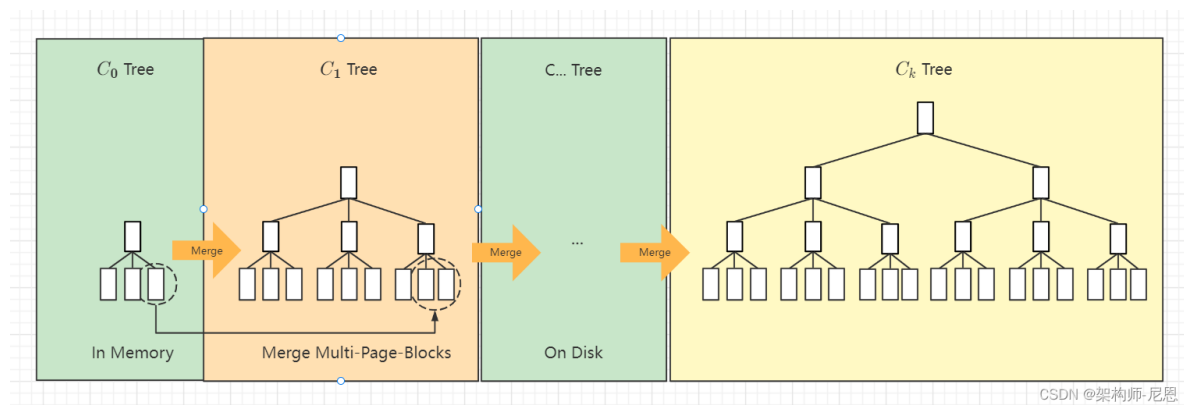
LSM树由两个或以上的存储结构组成，

最简单的两个存储结构：

- 一个存储结构常驻内存中，称为C0 tree，具体可以是任何方便键值查找的数据结构，比如红黑树、map之类，甚至是跳表。
- 另外一个存储结构常驻在硬盘中，称为C1 tree，一般为sstable。

C1所有节点都是100%满的，节点的大小为磁盘块大小。





40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

LSMTree和SSTable

LSM树的核心特点是利用“顺序写”来提高写性能。

LSM数据存储分为内存和文件两部分。

这样的设计，是通过牺牲小部分读性能换来高性能写。

LSM树的核心就是放弃部分读能力，换取写入的最大化能力，放弃磁盘读性能来换取写的顺序性。极端的说，基于LSM树实现的HBase的写性能比Mysql高了一个数量级，读性能低了一个数量级。

LSM树由Patrick O'Neil等人在论文《The Log-Structured Merge Tree》：

<https://www.cs.umb.edu/~poneil/lsmtree.pdf>，提出的

LSM树它实际上不是一棵树，而是2个或者多个树或类似树的结构（注意这点）的集合。

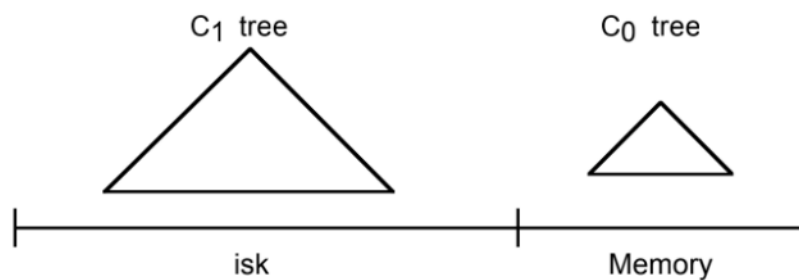


Figure 2.1. Schematic picture of an LSM-tree of two components

在LSM树中，最低一级也是最小的C0树位于内存里，而更高级的C1、C2...树都位于磁盘里。数据会先写入内存中的C0树，当它的大小达到一定阈值之后，C0树中的全部或部分数据就会刷入磁盘中的C1树，如下图所示。

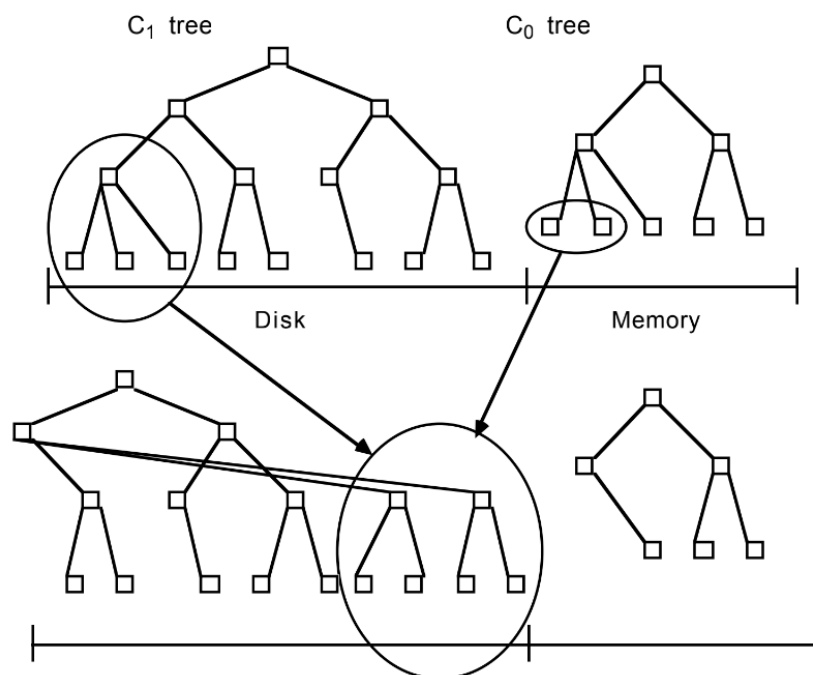
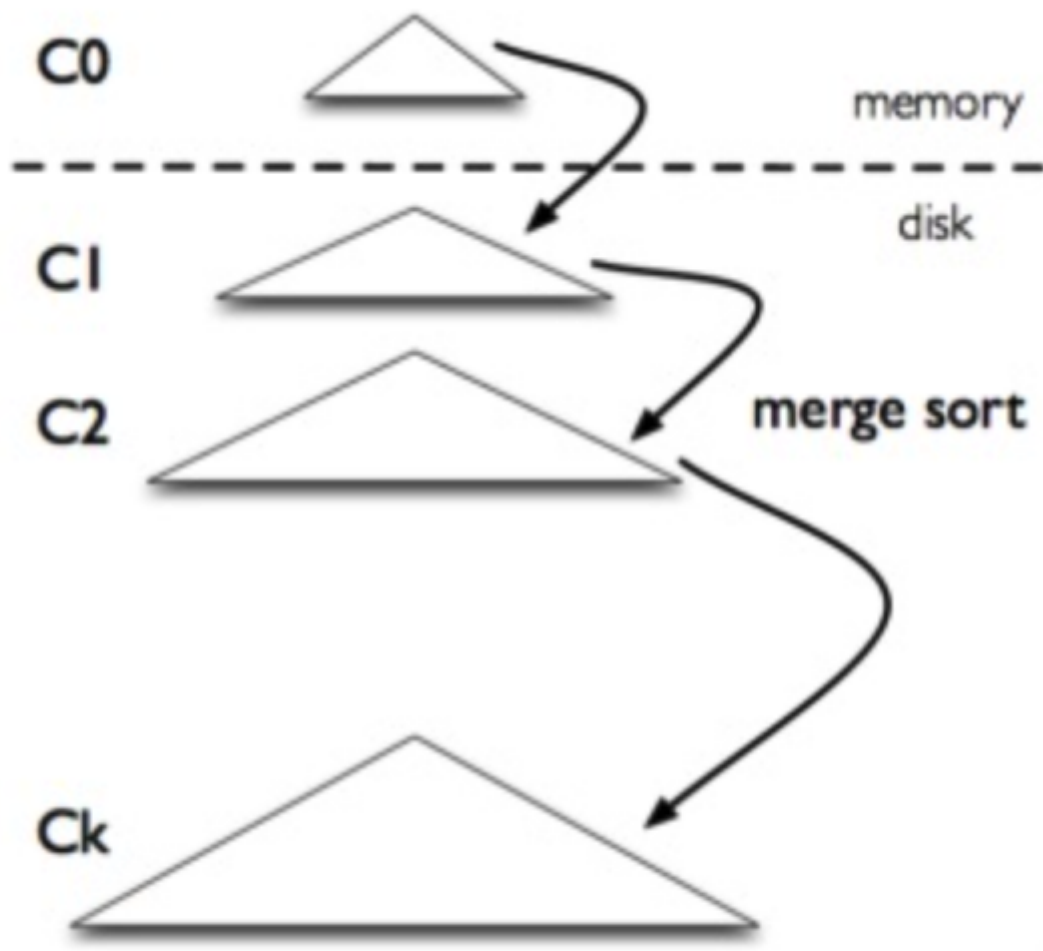


Figure 2.2. Conceptual picture of rolling merge steps, with result written back to disk

LSM树的合并过程:



(a) LSM-tree

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

LSM Tree的读写流程

LSM树它实际上不是一棵树，而是2个或者多个树或类似树的结构（注意这点）的集合。

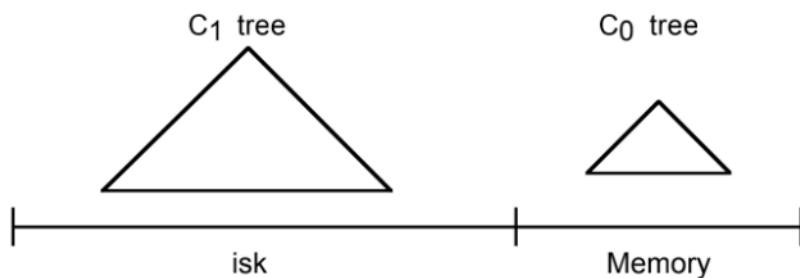


Figure 2.1. Schematic picture of an LSM-tree of two components

在LSM树中，最低一级也是最小的C0树位于内存里，而更高级的C1、C2...树都位于磁盘里。数据会先写入内存中的C0树，当它的大小达到一定阈值之后，C0树中的全部或部分数据就会刷入磁盘中的C1树，如下图所示。

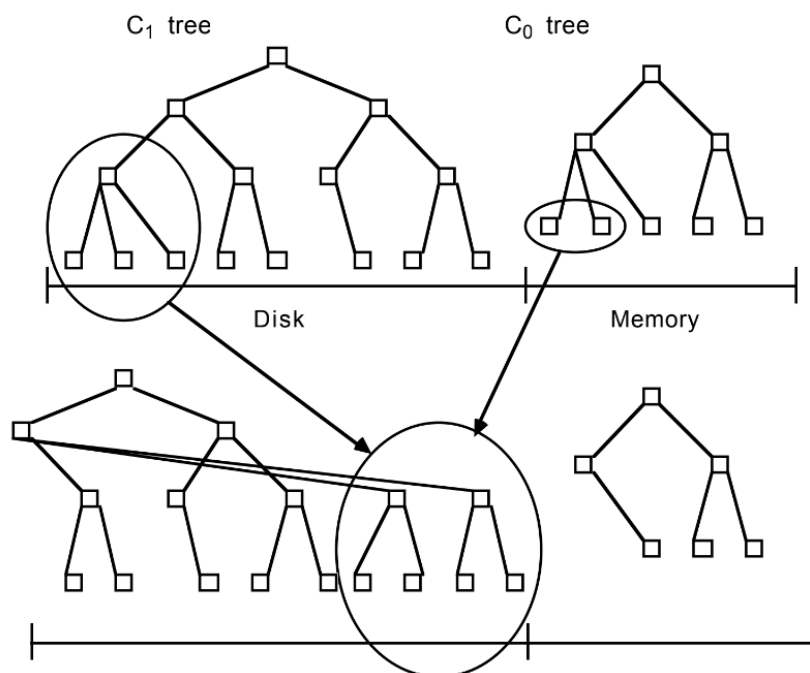


Figure 2.2. Conceptual picture of rolling merge steps, with result written back to disk

LSM Tree的顺序写的流程：

LSM 是由一棵棵小树组成，最小的树在内存，当我们新数据写入时，会在内存中暂存，这样能够获得非常大的写并发处理能力。

而当内存中数据积累到一定程度后，会将内存中数据和索引做 **顺序写**，落地形成一个数据块。

这个数据块内保存着一棵小树和具体的数据，新生成的数据块会保存在Level 0 层（最大有几层可配置），Level 0 层会有多个类似的数据块文件。

每一层的数据块和数据量超过一定程度时，合并不同Level的数据，将多个数据块内的数据和索引合并在一起，并推送到Level的下一层。

通过这种方式，每一层的数据块个数和数据量就能保持一定的数量，合并后的数据会更紧密、更容易被找到。

这样的设计，可以让一个Key存在于多个Level或者数据块中，但是最新的常用的数据肯定是在Level最顶部或内存（0~4层，0为顶部）中最新的数据块内。

LSM Tree的 查询 的流程:

而当我们查询一个key的时候，会先查内存Level 0 层的树。

如果没找到，会从Level 0层到下层，每层按生成最新到最老的顺序去查询每层的数据块。

同时为了减少IO次数，每个数据块都会有一个BloomFilter辅助索引，来辅助确认这个数据块中是否 **可能** 对应的Key；如果当前数据块没有，那么可以快速去找下一个数据块，直到找到为止。

上面LSM的概念很复杂，接下来，咱们来一步一步历险记

一步一步，开始LSM-Tree 历险记

LSM-Tree 全名叫Log-Structured Merge Tree，最早建立在日志结构文件之上，现在基于合并和压缩排序文件原理的存储引擎都统称为LSM存储引擎。

我们通常把LSM看成一种思想：保存在后台合并的一系列SStable

这种思想简单且有效：

- 即使数据集远大于内存，LSM-tree也能正常工作
- 由于键值有序，范围查询相比于hash表有很大优势
- 由于写入是顺序的（归并是后台线程在空闲时间做的）LSM-tree可以提供非常高的写入吞吐量

查找的性能优化：

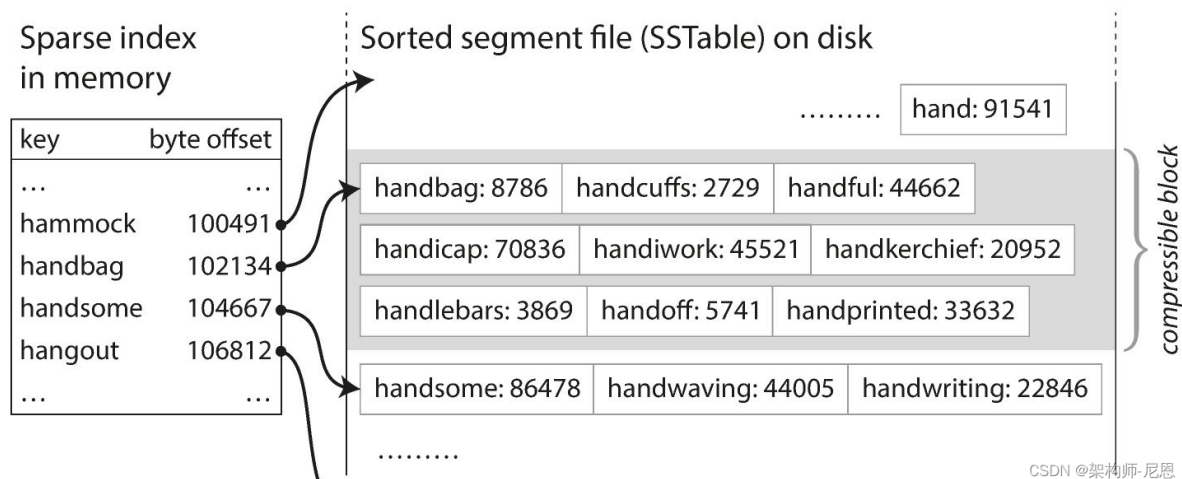
在LSM系统中查找一个不存在的键时会导致查询时间长，因为要从最新的数据一直往前查找，所以LSM一般会使用布隆过滤器进行优化

接下来，详细说说 SStable

什么是SStable呢？

SStable (Sorted String Table) 是排序字符串表的简称，

SStable 文件结构如下（不同的存储引擎具体实现会有差异）：



SSTable 本身是个简单而有用的数据结构，而往往由于工业界对于它的过载，导致大家误解。

不同厂商对 SSTable 的实现差异还是非常大的。

例如 Cassandra 中 SSTable 并不是单一的文件，而是由多个文件如 Data.db、Index.db、Summary.db、Filter.db 等多个文件组成；

而LevelDB 的 SSTable 文件就是单一的文件，文件分成数据块、元信息块、索引块等信息。

实际上，Google 的 LevelDB 中的 SSTable 的实现是更接近于 Bigtable 论文中的描述。

正如名字本身所包含的意思一样，SSTable是一个简单的抽象，用来高效地存储大量的键-值对数据，同时做了优化来实现顺序读/写操作的高吞吐量。

- SSTable在功能上，只是对hash加入了一个按**键值排序**。
- 由于是排序数据，我们不必像hash索引一样记录所有数据的位置，索引可以是稀疏的，例如对于段文件中的上千条数据，只保留一个索引

例如：

对于数据键1~100000，我们每隔1000个数据保留一个索引，当查找5500时，对应索引应该是5000，之后向后查找500条（或者使用二分）。

使用方法似乎很好理解，麻烦的是构建和维护SSTable。

SSTable基本工作流程是：

- 写入时，将内容插入内存中的排序数据结构（例如红黑树）
- 当内存大于一定阈值时，将其写入磁盘，作为磁盘的一部分

SSTable的读取：

- 如果接收到新的**读取请求**，要先尝试在内存中读取，其次是磁盘的最新排序部分，然后是次新的，依次类推
- 后台进程周期性将各个排序部分归并，并且出现相同数据时只会保留最新值。

如果使用sstable，先进行内存写，所以也要考虑日志系统保证持久性，类似innodb里redolog方式来保证持久性。

为什么需要 SSTable ？

我们知道，对于 key-value 类型的数据存储，通常有哈希存储引擎可供使用，它一般使用哈希表索引实现。

哈希表索引对单个键的查询非常高效，可以 $O(1)$ 的时间复杂度内完成。

但是哈希表索引也有非常明显的缺点：

- 哈希表索引必须全部载入内存，由于它是一个稠密索引（对写入的每一条数据都要进行索引），如果存在大量的键，会占用大量内存。原则上，可以在磁盘上维护哈希表索引，但不幸的是，很难在磁盘上实现性能良好的哈希表索引。磁盘上的哈希表索引会产生大量的随机访问 I/O，当哈希表变满时，继续增长代价昂贵，并且哈希冲突时需要复杂的处理逻辑。
- 区间查询效率不高。

SSTable 可以摆脱这些限制，它非常适合于写量级远大于读量级的情况。

目前，SSTable 已经被广泛应用于一些耳熟能详的存储引擎中，如 Bigtable、Cassandra、Hbase、RocksDB、LevelDB、ScyllaDB等 key-value 型存储引擎。

SSTable的定义

SSTable (Sorted String Table) 是**排序字符串表**的简称，来源于大名鼎鼎的 Google Bigtable 论文。

它用于 Bigtable 内部数据文件的存储，它是一个种高效的 key-value 型文件存储格式。

要解释这个术语的真正含义，最好的方法就是从它的出处找答案，所以重新翻开BigTable的论文。

在这篇论文中，最初对SSTable是这么描述的（第三页末和第四页初）：

SSTable

The Google **SSTable** file format is used internally to store Bigtable data.

An **SSTable** provides a **persistent, ordered immutable map from keys to values, where both keys and values are arbitrary byte strings**. Operations are provided to look up the value associated with a specified key, and to iterate over all key/value pairs in a specified key range. Internally, each SSTable contains a sequence of blocks (typically each block is 64KB in size, but this is configurable). A block index (stored at the end of the SSTable) is used to locate blocks; the index is loaded into memory when the SSTable is opened. A lookup can be performed with a single disk seek: we first find the appropriate block by performing a binary search in the in-memory index, and then reading the appropriate block from disk. Optionally, an SSTable can be completely mapped into memory, which allows us to perform lookups and scans without touching disk.

简单的非直译：

SSTable是Bigtable内部用于数据的文件格式，它的格式为文件本身就是一个排序的、不可变的、持久的Key/Value对Map，其中Key和value都可以是任意的byte字符串。

使用Key来查找Value，或通过给定Key范围遍历所有的Key/Value对。

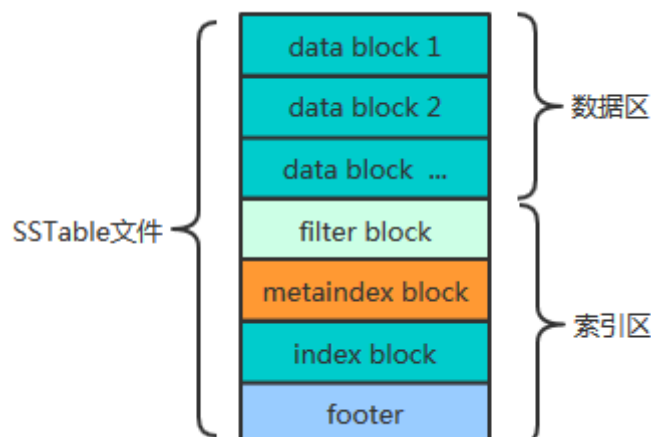
每个SSTable 包含一系列的Block（一般Block大小为64KB，但是它是可配置的），

在SSTable的末尾是Block索引，用于定位Block，**这些索引在SSTable打开时被加载到内存中**，在查找时首先从内存中的索引二分查找找到Block，然后一次磁盘寻道即可读取到相应的Block。

还有一种方案是将这个SSTable加载到内存中，从而在查找和扫描中不需要读取磁盘。

LevelDB 中的SSTable

先看看SSTable文件的结构



整体上看 SSTable文件分为数据区与索引区，

尾部的footer指出了meta index block与data index block的偏移与大小，

index block指出了各data block的偏移与大小， metaindex block指出了filter block的偏移与大小。

- 1) data block: 存储key-value记录，分为Data、type、CRC三部分
- 2) filter block: 默认没有使用，用于快速从data block 判断key-value是否存在
- 3) metaindex block : 记录filter block的相关信息
- 4) Index block: 描述一个data block，存储着对应data block的最大Key值，以及data block在文件中的偏移量和大小
- 5) footer: 索引的索引，记录metaindex block和Index block在SSTable中的偏移量和大小

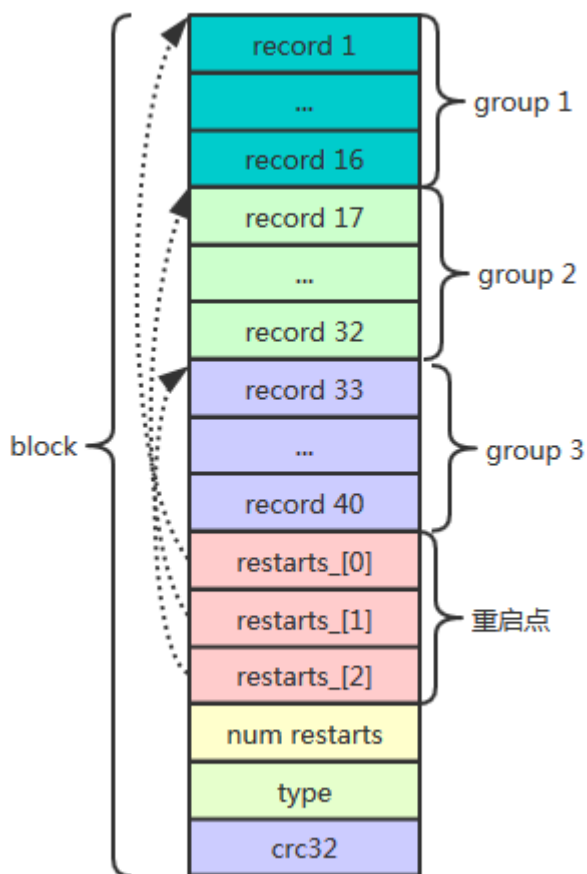
下面再具体看看各个部分物理结构

block

sstable中data block 、 metaindex block、 index block都用这种block这种结构。

对于data block,当block大小（record、restarts数组、以及num_restarts）超过4k时，就切换一个新的block继续往SSTable写数据，

而metaindex block、index block就只有一个block，所以上图看起来data block有多个。



block主要由数据区record和restarts组成。为什么是这种结构？

data block主要是存储数据，block内给一定数量（默认16）key-value分组，

每组又用restarts数组记录起始位置，因此可以根据restarts读取每组起始位置key-value，

由于**block内的数据是从小到大有序存储的**，所以可以通过restarts数组，获取每组起始key-value,比较起始key key(n)与查找的key大小，如果key(n)>key，那么key一定在序号>=n组之后，否则在 < n组之前。

因此可以通过二分查找思想通过restarts获取起始key，来定位key的位置，避免线性查找低效。

因此，restarts的思想就是：**提高block内key-value查找效率**，直接定位key所在group。

record结构

下面再来看看record结构。

record相对有意思，不是简单的用key-length | key-data | value-length|value-data存储。



data block中的key是有序存储的，相邻的key之间可能有重复，因此存储时采用前缀压缩，后一个key只存储与前一个key不同的部分。

重启点指出的位置即每组起始位置的key不按前缀压缩，而是完整存储该key。

type是表示数据是否压缩，以怎样的方式压缩，crc32是该block校验码。

index block

index block 的结构也是block 结构，是data block的索引，记录每个data block 最大key 和 起始位置以及大小。

具体的存储方式是以每个data block最大key 为key,以data block 起始位置和大小为value。因此可以根据每个

block的最大key与查询key比较，直接定位查询key所在的位置。

这是理论上key的存储方式，但是在sstable二次压缩的过程对key做了一个优化，它并不保存最大key，而是保存一个能分隔两个data block的最短Key，如：假定data block1的最大一个key为“abcdefg”，data block2最大key为“abzxcv”，则index可以记录data block1的索引key为“abd”；这样的分割串可以有很多，只要保证data block1中的所有Key都小于等于此索引，data block2中的所有Key都大于此索引即可。

这种优化缩减了索引长度，查询时可以有效减小比较次数。

因此，index block的思想是提高SSTable内key-value查找效率，直接定位key所在block。

metaindex_block

也是block结构。就只有一条记录，其key是filter. + filter_policy的name，value是filter大小和起始位置。

filter block

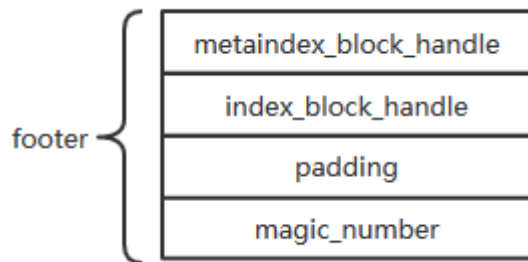
filter block就是一个bloom filter,关于bloom filter原理概念可以百度。

每个bloom filter是对data block 的key 经过hash num 次形成的字节数组,多少个data block对应多少个bloom filter。

bloom filter实质就是一个bit 数组, 对block 内key hash,将相应的位置设为1, 这种设计关键在于能提高不存在的key判断效率, 通过filter 计算, 如果不存在, 就不用通过data block内的restarts方式读取文件查找key是否存在, 但是如果filter判断存在, 还需通过restarts方式确定。

footer

footer位于SSTable文件尾部, 占用空间固定为48个字节。其末尾8个字节是一个magic_number。metaindex_block_handle与index_block_handle物理上占用了40个字节, metaindex_block_handle和index_block_handle是BlockHandle数据类型, 这种结构用于记录metaindex block 和index block 的起始位置和大小。



对于BlockHandle, 其实可以看作文件内容指针实现方式,BlockHandle记录数据位置及大小, 与c/c++指针 思想类似, 通过地址和大小可以读取数据。

BlockHandle格式

```
varint64 offset | varint64 size_
```

采用变长存储, 所以实际上存储可能连32字节都不到, 剩余填充0。

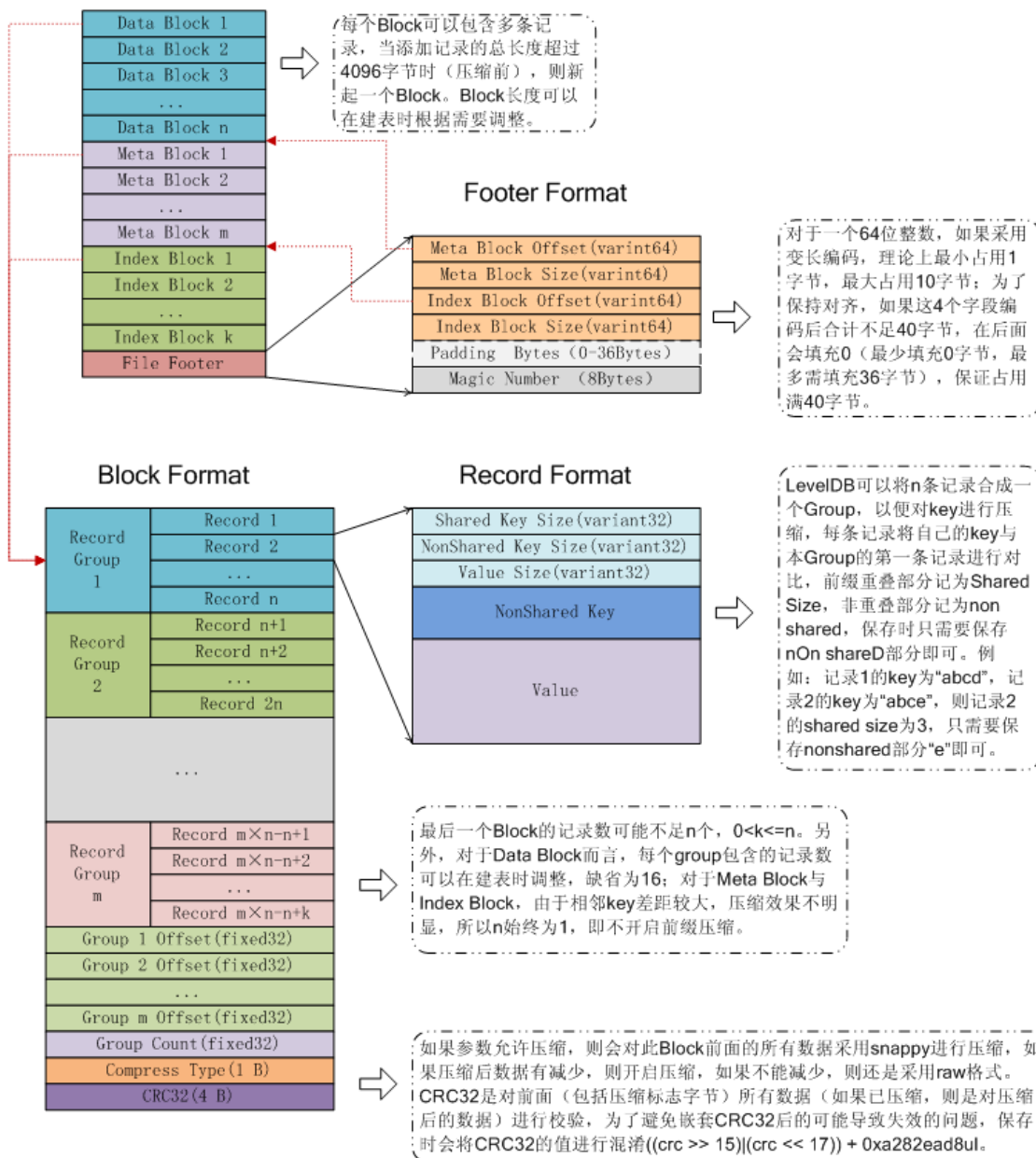
总结:

SSTable其实就是通过二次索引, 先读取footer, 根据footer中index_block_handler记录的index_block起始位置和大小, 读取index block, 通过index block 查询key所在data block, 再在data block内部通过restarts 进一步确定key所在group。

levelDB SSTable结构图

下面是完整的SSTable结构图

Level DB SSTable Format



Level DB 持久化数据格式

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

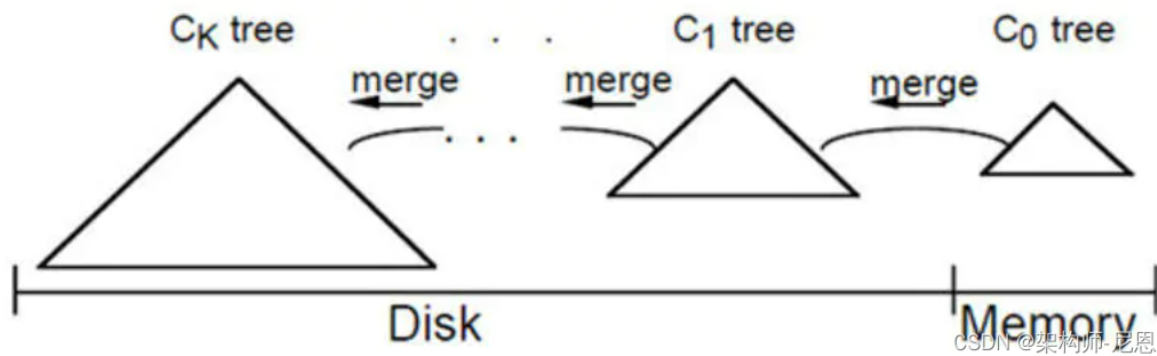
LSM插入步骤

插入一条新纪录时，首先在日志文件中插入操作日志，以便后面恢复使用，日志是以append形式插入，所以速度非常快；

将新纪录的索引插入到C0中，这里在内存中完成，不涉及磁盘IO操作；

当C0大小达到某一阈值时或者每隔一段时间，将C0中记录滚动合并到磁盘C1中；

对于多个存储结构的情况，当C1体量越来越大就向C2合并，以此类推，一直往上合并Ck。



LSM合并步骤

合并过程中会使用两个块：emptying block 和 filling block。

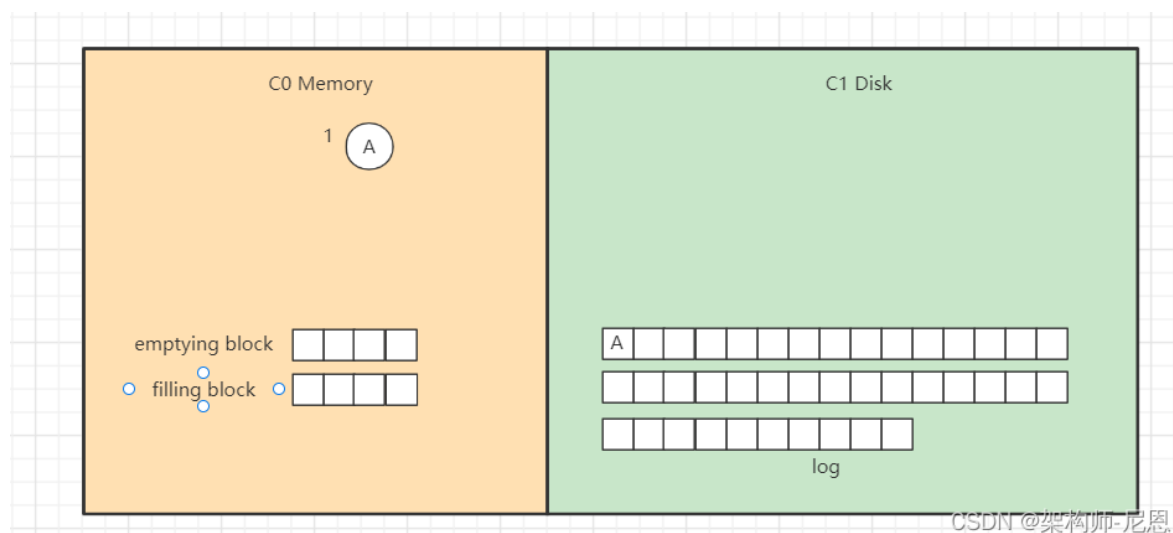
1. 从C1中读取未合并叶子节点，放置内存中的emptying block中。
2. 从小到大找C0中的节点，与emptying block进行**合并排序**，合并结果保存到filling block中，并将C0对应的节点删除。
3. 不断执行第2步操作，合并排序结果不断填入filling block中，当其满了则将其追加到磁盘的新位置上，注意是追加而不是改变原来的节点。合并期间如故宫emptying block使用完了则再从C1中读取未合并的叶子节点。
4. C0和C1所有叶子节点都按以上合并完成后，即完成一次合并。

LSM插入案例

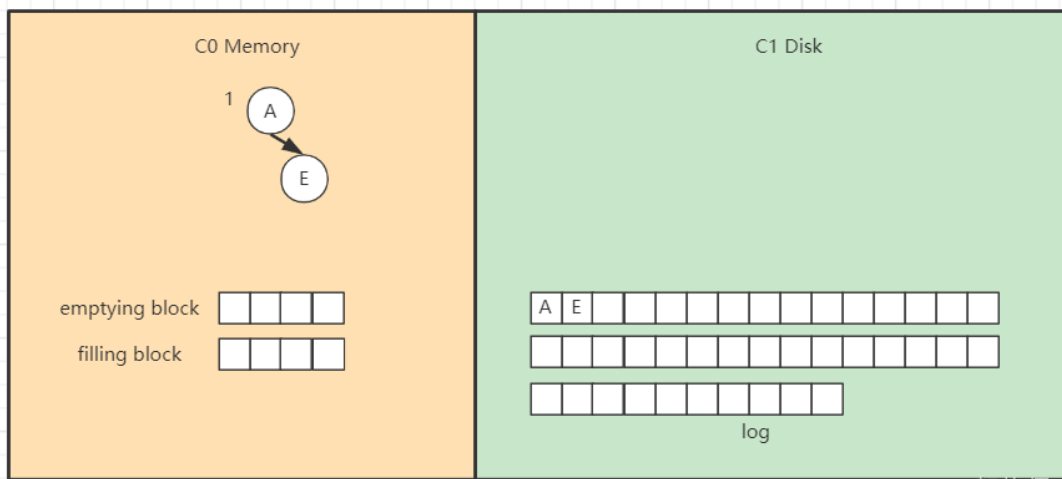
向LSM树中插入 `A E L R U`，首先会插入到内存中的C0树上，

这里使用AVL树，插入“A”，

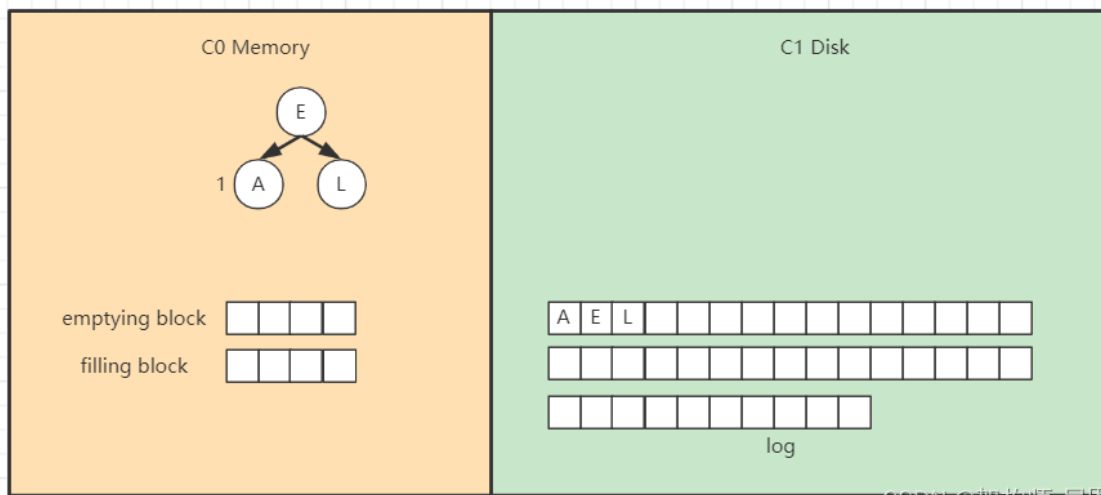
当然，得先WAL，预先写入日志，向磁盘日志文件追加记录，然后再插入C0，



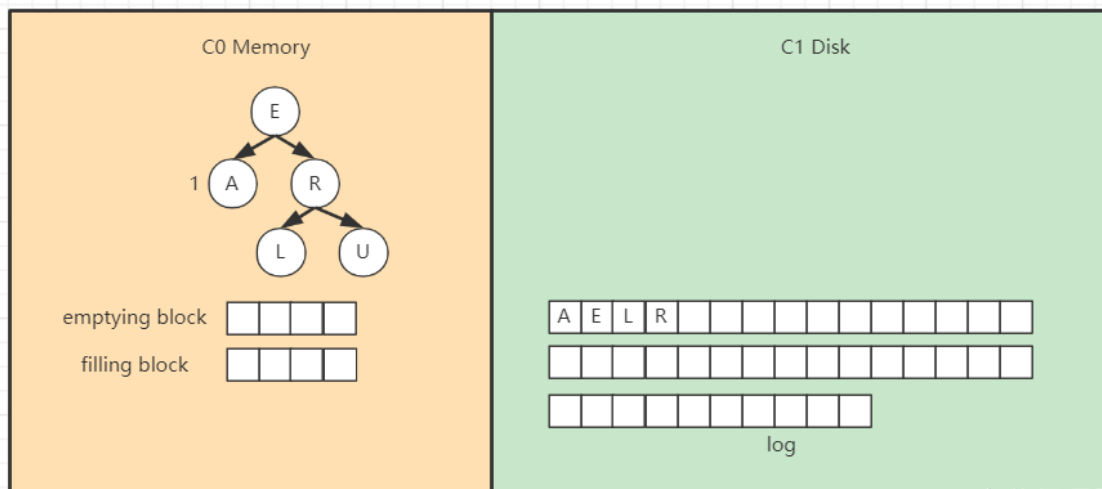
插入“E”，同样先追加日志再写内存，



继续插入“L”，旋转后如下，



插入“R”“U”，旋转后最终如下。

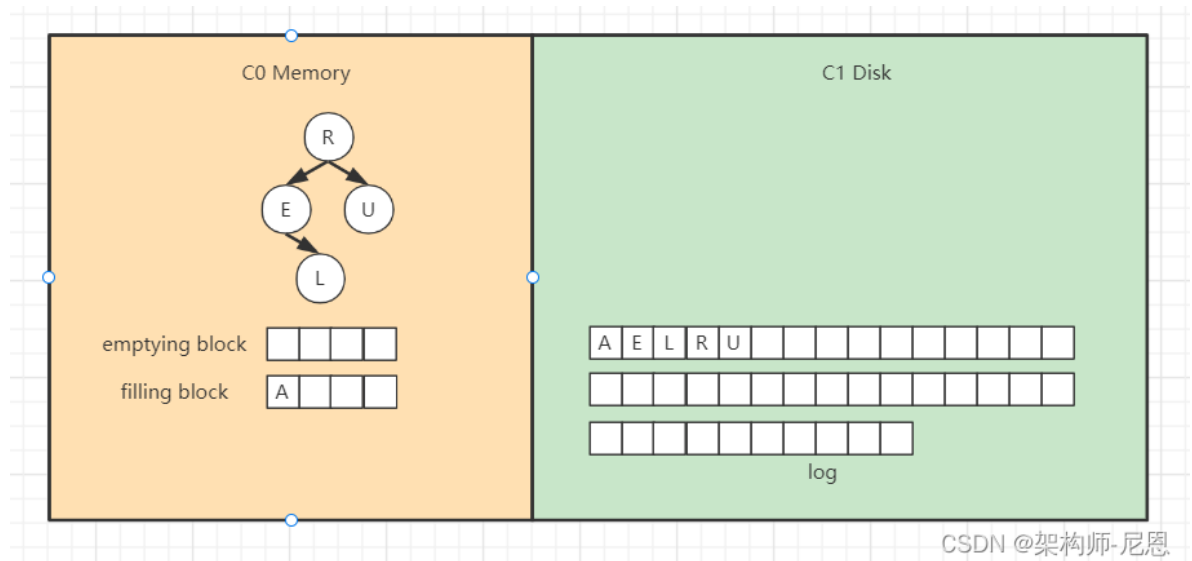


假设此时触发合并，

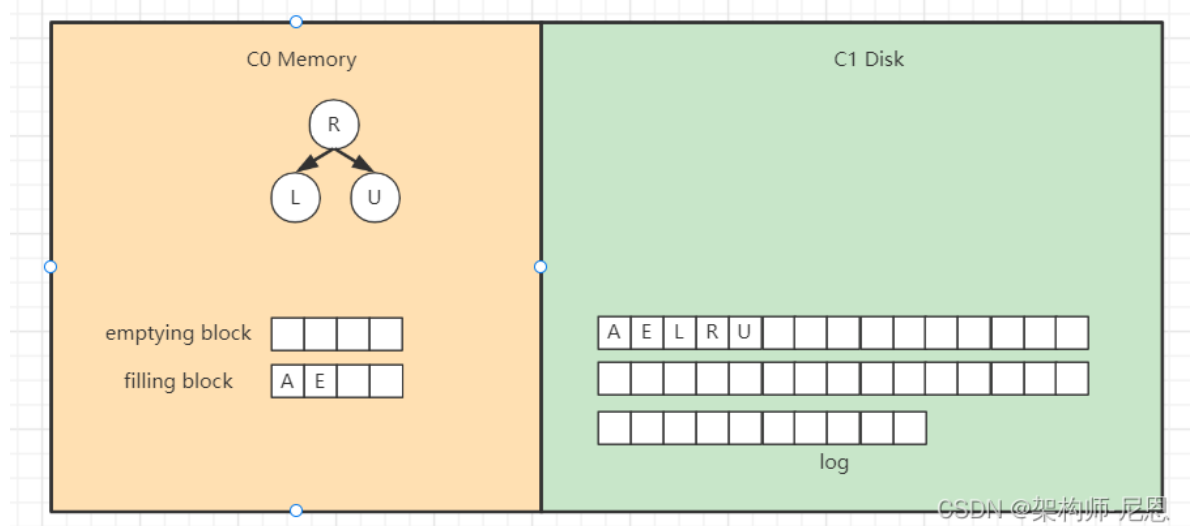
则因为C1还没有树，所以emptying block为空，直接从C0树中依次找最小的节点。

filling block长度为4，这里假设磁盘块大小为4。

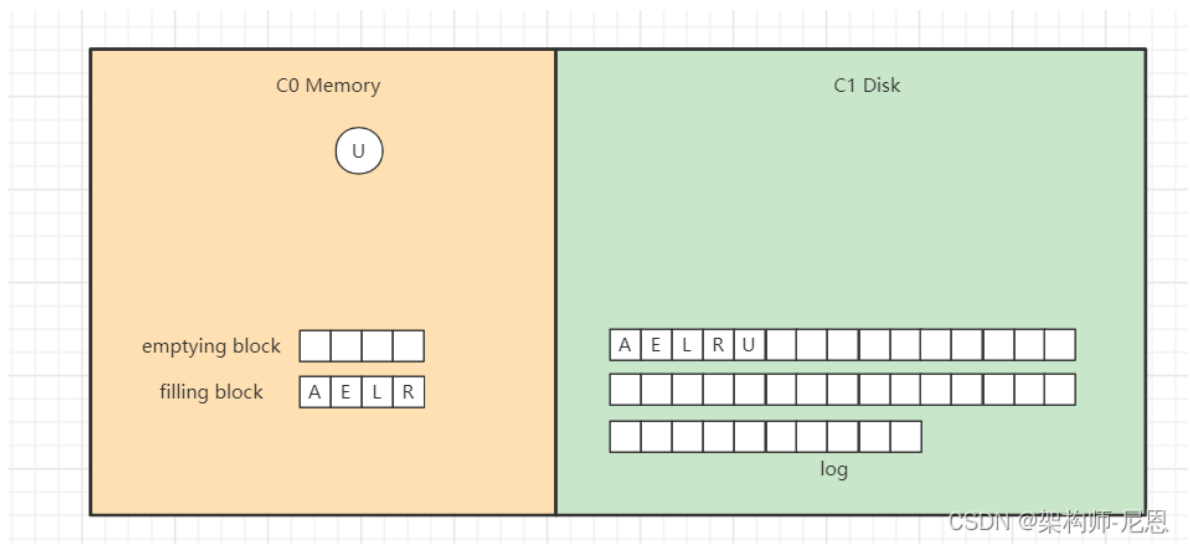
开始找最小的节点，并放到filling block中，



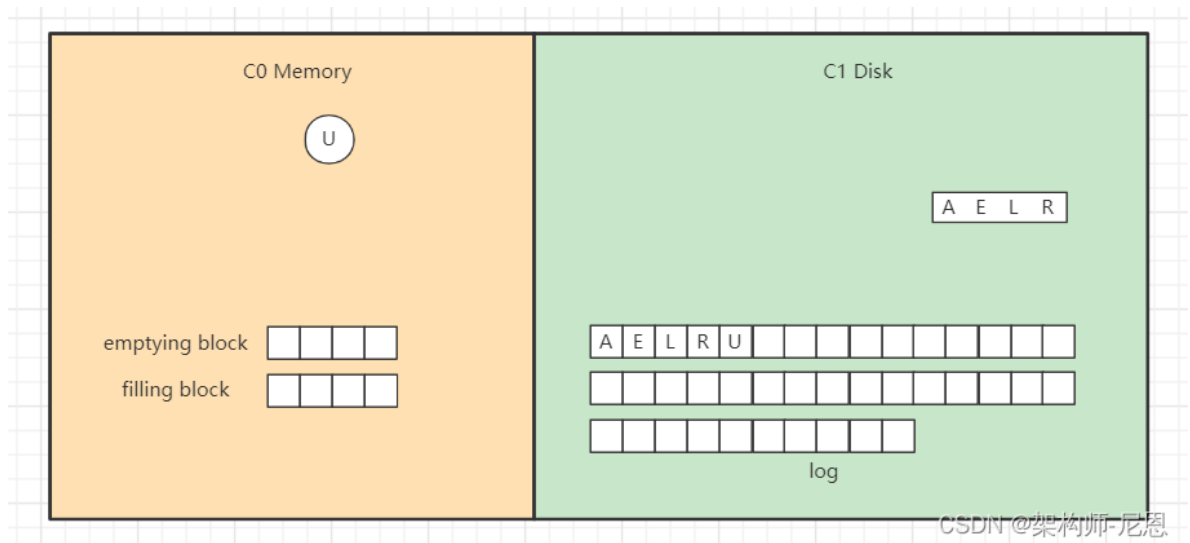
继续找第二个节点，



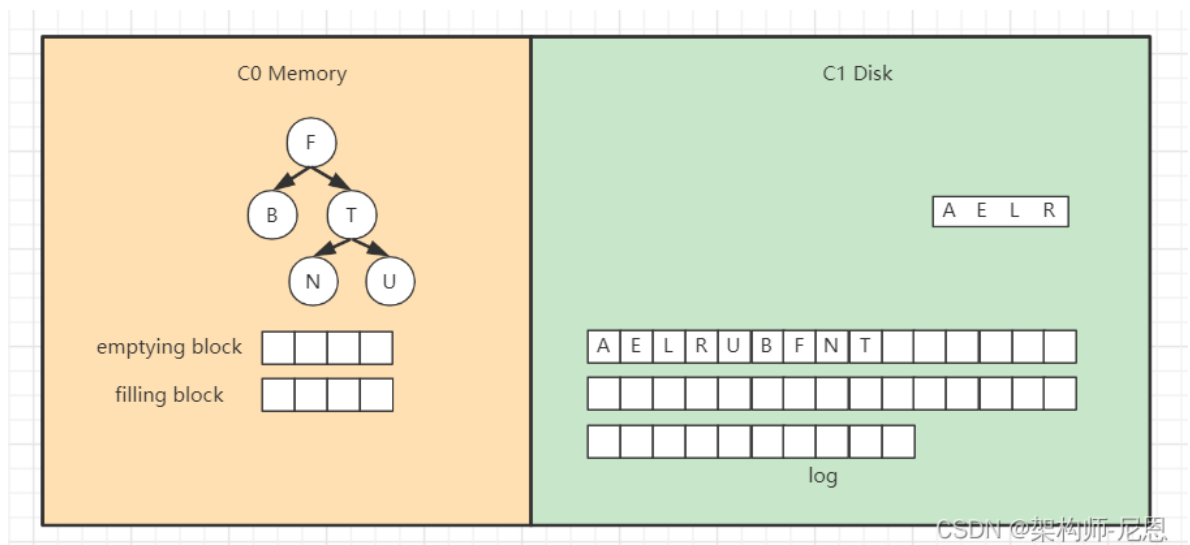
以此类推，填满filling block，



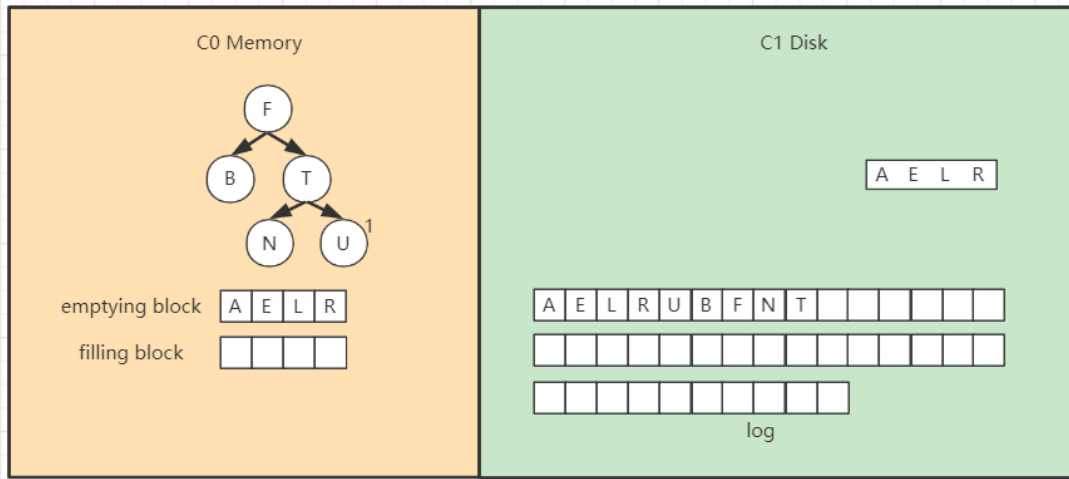
开始写入磁盘，C1树，



继续插入 **B F N T**，先分别写日志，然后插入到内存的C0树中，

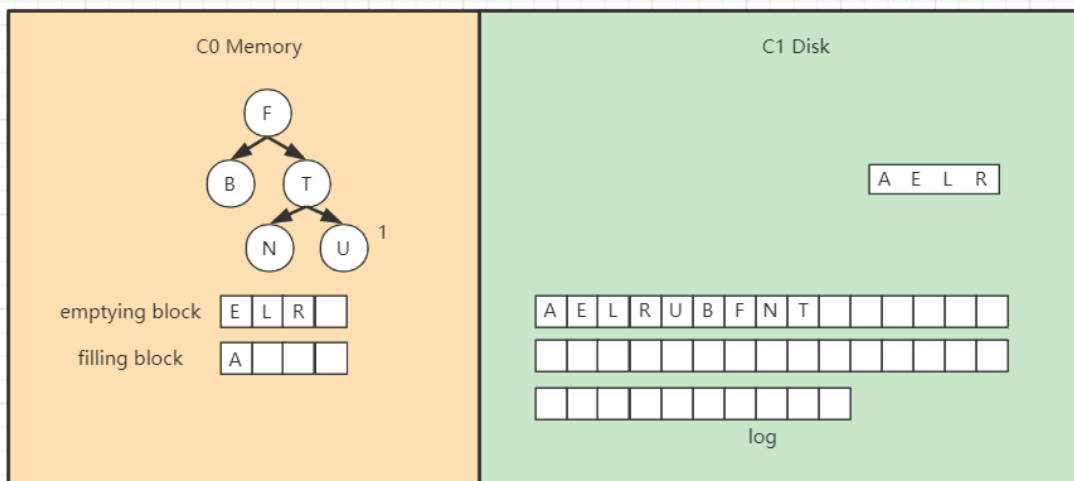


假如此时进行合并，先加载C1的最左边叶子节点到emptying block，



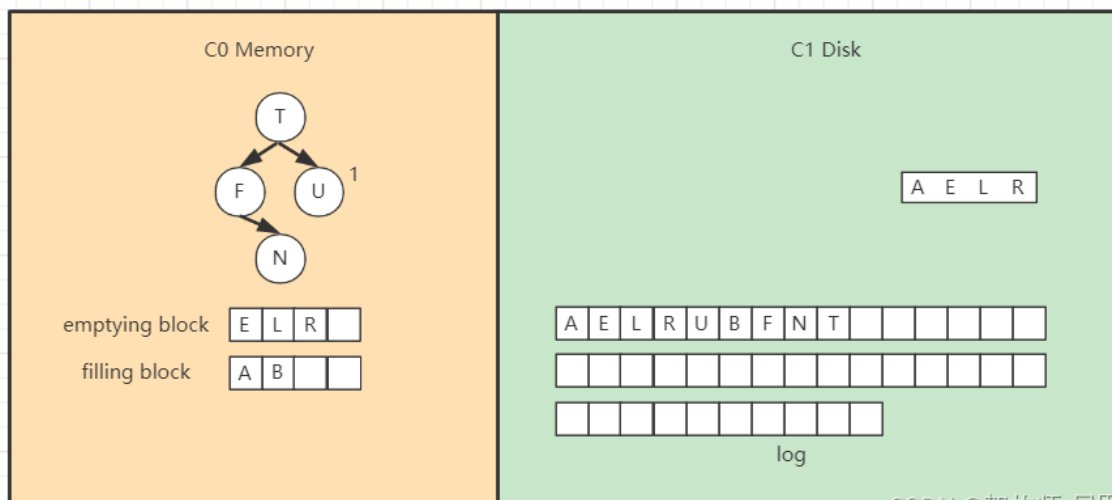
CSDN @架构师-尼恩

接着对C0树的节点和emptying block进行**合并排序**，首先是“A”进入filling block，



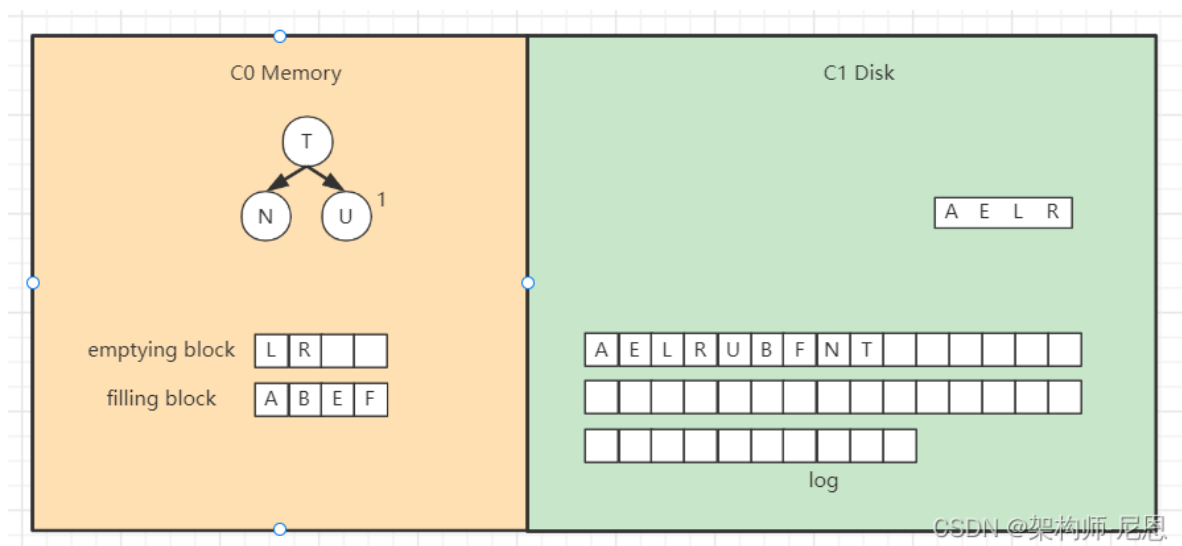
CSDN @架构师-尼恩

然后是“B”，

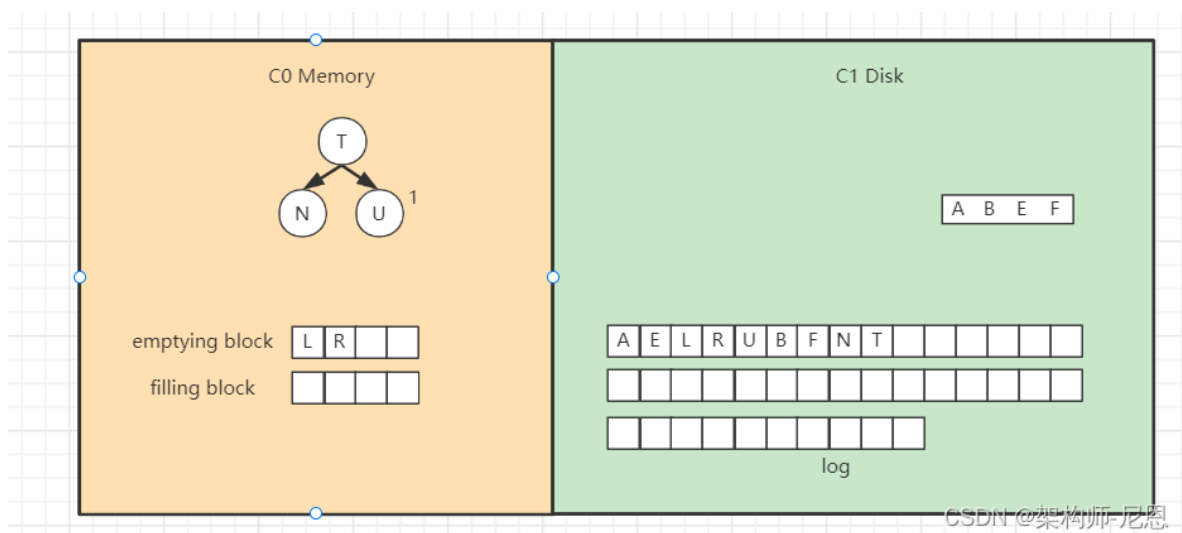


CSDN @架构师-尼恩

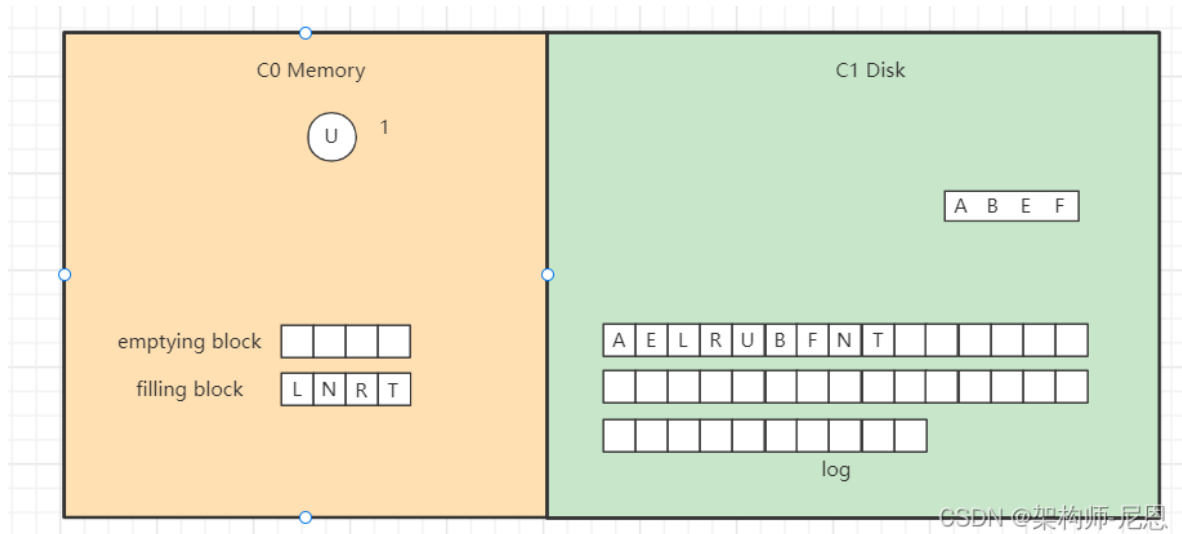
合并排序最终结果为，



将filling block追加到磁盘的新位置，将原来的节点删除掉，



继续合并排序，再次填满filling block，



将filling block追加到磁盘的新位置，上一层的节点也要以磁盘块（或多个磁盘块）大小写入，尽量**避免随机写**。

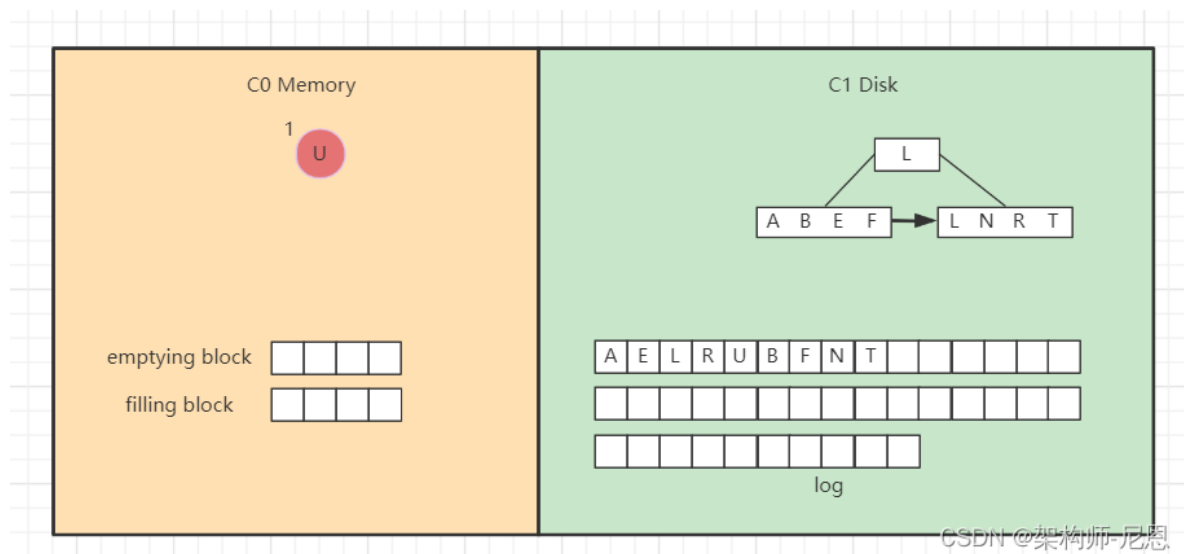
另外由于合并过程可能会导致上层节点的更新，可以暂时保存在内存，后面在适当时机写入。

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

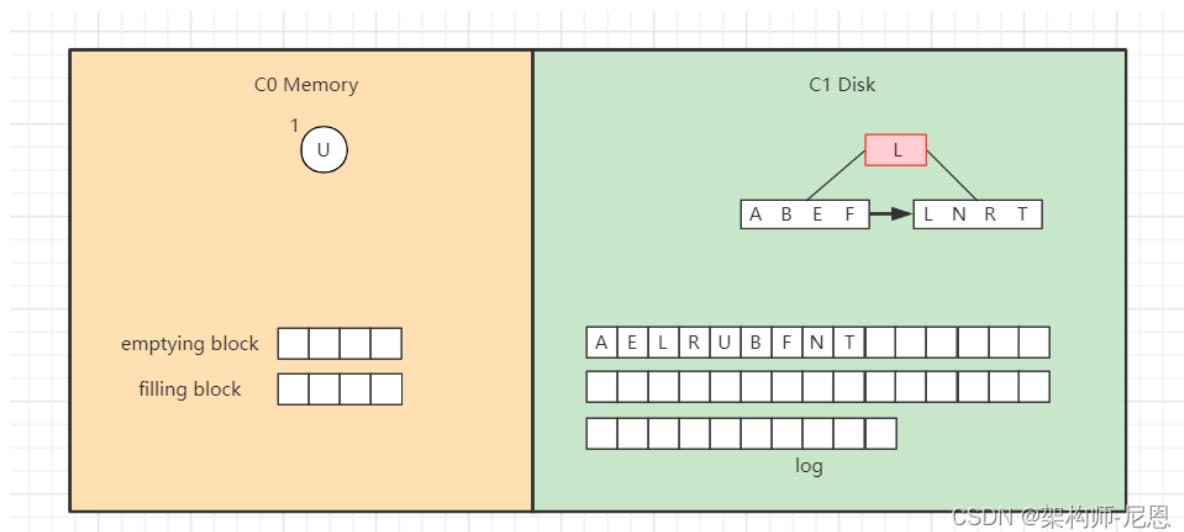
LSM查找操作

查找总体思想是先找内存的C0树，找不到则找磁盘的C1树，然后是C2树，以此类推。

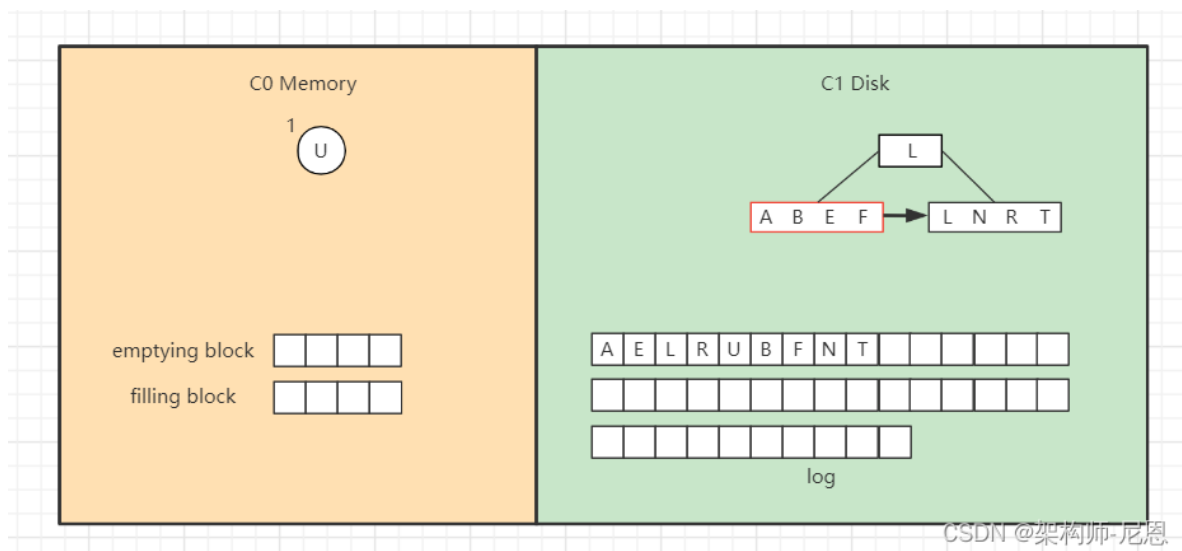
假如要找“B”，先找C0树，没找到。



接着找C1树，从根节点开始，



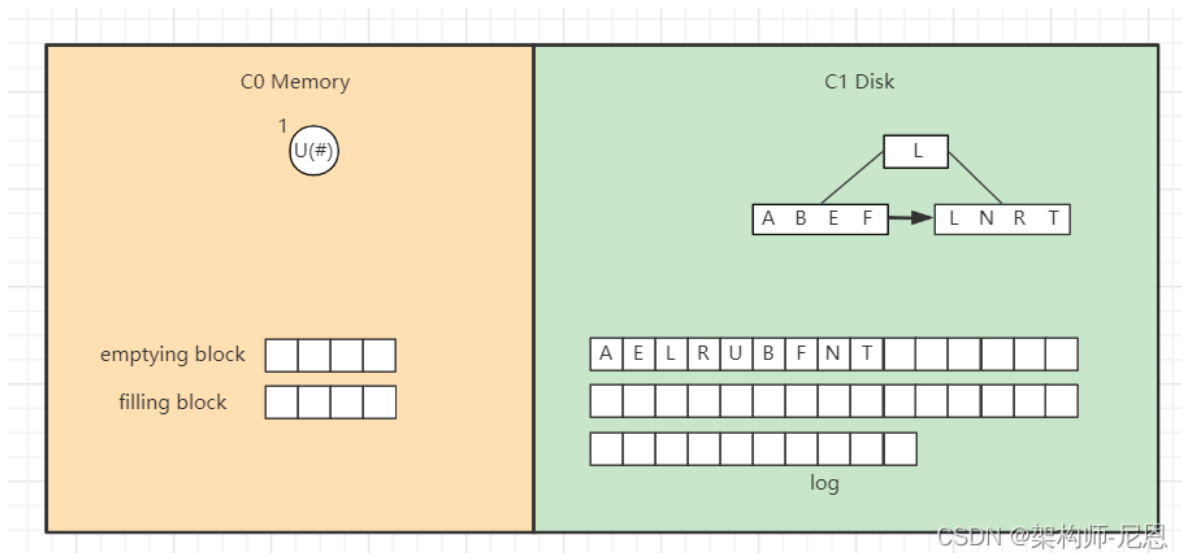
找到“B”。



LSM删除操作

删除操作为了能快速执行，主要是通过标记来实现，在内存中将要删除的记录标记一下，后面异步执行合并时将相应记录删除。

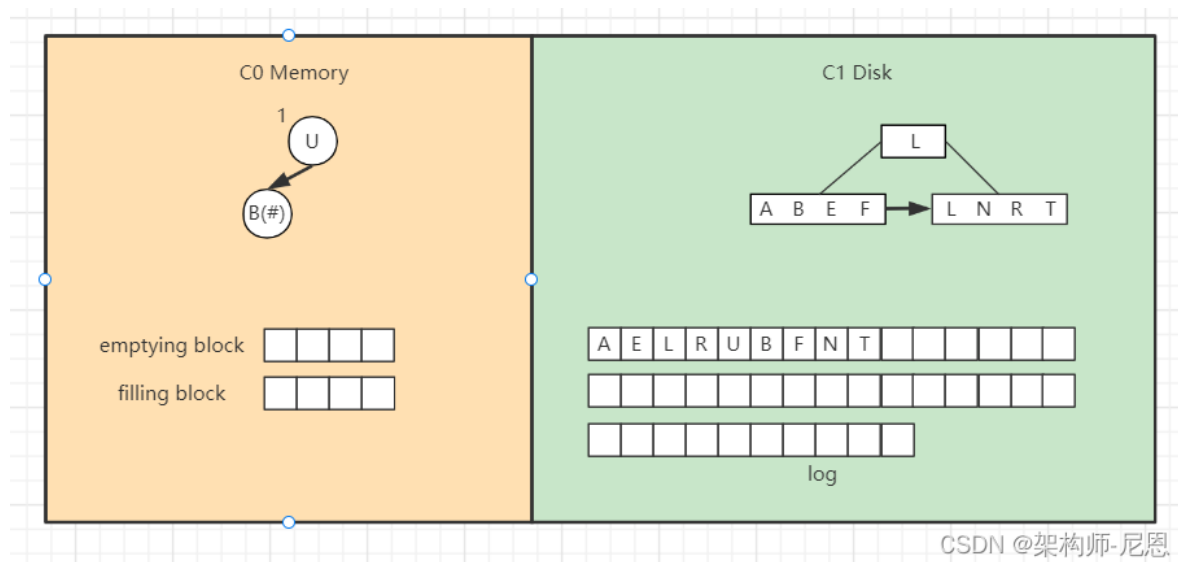
比如要删除“U”，假设标为#的表示删除，则C0树的“U”节点变为，



而如果C0树不存在的记录，

则在C0树中生成一个节点，并标为#，查找时就能再内存中得知该记录已被删除，无需去磁盘找了。

比如要删除“B”，那么没有必要去磁盘执行删除操作，直接在C0树中插入一个“B”节点，并标为#。



40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

LSM树的特点：用牺牲读性能，来换取写性能

优化写性能

如果我们对写性能特别敏感，我们最好怎么做？

—— Append Only：所有写操作都是将数据添加到文件末尾。这样做的写性能是最好的，大约等于磁盘的理论速度（200 ~ 300 MB/s）。

但是 Append Only 的方式带来的问题是：

- 读操作不方便。
- 很难支持范围操作。
- 需要垃圾回收（合并过期数据）。

所以，纯粹的 Append Only 方式只能适用于一些简单的场景：

- 数据库的 WAL（预写日志）。
- 能知道明确的 offset，比如 Bitcask。

如果要优化读性能

如果我们对读性能特别敏感，一般我们有两种方式：

- 有序存储，比如 B+ 树，SkipList 等。
- Hash 存储 —— 不支持范围操作，适用范围有限。

读写性能的权衡

如何获得（接近） Append Only 的写性能，而又能拥有不错的读性能呢？

以 LevelDB 为代表的 LSM 存储引擎给出了一个参考答案。

注意，LevelDB 实现的是优化后的 LSM，原始的 LSM 可以参考论文。

以 LevelDB 例子，LevelDB 的写操作主要由两步组成：

- 写日志并持久化（Append Only）。
- Apply 到内存中的 memtable（SkipList）。

所以，LevelDB 的写速度非常快。

memtable 写“满”后，会转换为 immutable memtable，

然后被后台线程 compaction 成按 Key 有序存储的 sst 文件（顺序写）。

由于 sst 文件会有多个，所以 LevelDB 的读操作可能会有多次磁盘 IO（LevelDB 通过 table cache、block cache 和 bloom filter 等优化措施来减少读操作的磁盘 IO 次数）。

基于 LSM 数据结构的 NO SQL 的适用场景：

- 写请求多。
- 写性能要求高：（高吞吐+低延迟）。

LSM-tree 的另一大特点是除了使用两部分类树的数据结构外，还会使用日志文件（通常叫作 commit log）来为数据恢复做保障。

这三类数据结构的协作顺序一般是：

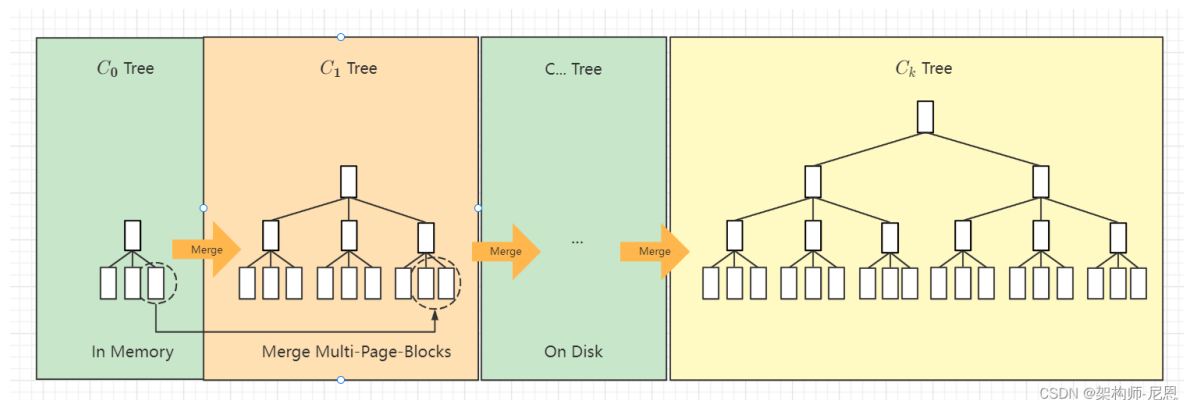
所有的新插入与更新操作都首先被记录到 commit log 中——该操作叫作 WAL（Write Ahead Log 预写日志），然后再写到 memtable，最后当达到一定条件时数据会从 memtable 冲写到 sstable，并抛弃相关的 log 数据；

memtable 与 sstable 可同时供查询；当 memtable 出问题时，可从 commit log 与 sstable 中将 memtable 的数据恢复。

理论上，可以是内存中树的一部分和磁盘中第一层树做合并，对于磁盘中的树直接做 update 操作有可能会破坏物理 block 的连续性，

但是实际应用中，一般 LSM 树有多层，

当磁盘中的小树合并成一个大树的时候，可以重新排好顺序，使得 block 连续，优化读性能。



LSM 树的特点：**用读性能来换取写性能**，将对数据的修改增量保持在内存中，达到指定的大小限制后将这些修改操作批量写入磁盘

LSM 树的核心思想：放弃部分读性能，提高写性能

代表数据库：nessDB、LevelDB、HBase 等非关系型数据库

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HBase 产生背景

上面提到，随着数据规模越来越大，大量业务场景开始考虑数据存储的水平扩展，海量数据量存储成为提升应用性能的瓶颈，单台机器无法负载海量的数据处理，随之而来的出现了很多的分布式存储解决方案，HBase就是其中之一。

HBase - - DataBase on Hadoop，基于分布式文件系统上面建立的数据库，HBase是面向列的开源数据库。

开源团队根据2008年Google发布了一篇关于Google搜索引擎BigTable的核心思想的论文，实现了基于分布式文件系统的列数据库。

随后加入Apache基金会，成为Hadoop生态圈中的顶级项目被大家熟知。

MySQL瓶颈

MySQL是一个关系型数据库，有很高的数据一致性和持久性保证，当访问量特别高时，特别是写入操作，会有很大的O性能瓶颈。

虽然可以通过主从读写分离、分库分表的方式解决，但随着数据量不断增大、并发不断增高，MySQL应用开发越来越复杂，也越来越具有技术挑战性。

另外，分表分库的规则的设置都是需要经验的，虽然有Cobar、MyCat、Sharding-JDBC、TDDL、DBProxy中间件层来屏蔽开发者的复杂性，但是避免不了整个架构的复杂性。

分库分表的子库到一定阶段又面临扩展问题，需求的变更可能又需要一种新的sharding。

MySQL的扩展性差、大数据下IO压力大、表结构更改困难，正是MySQL开发人员面临的问题，也是MySQL的瓶颈。

虽然有这些瓶颈，但对数据一致性要求特别高的业务，还是要使用它，但是有的应用场景不需要太高的一致性，在大数据量、高并发的业务中，可以选择其他存储方案。

NOSQL的出现

NoSQL数据库种类繁多，但是一个共同的特点都是去掉关系数据库的关系型特性，数据之间无关系，这样就非常容易扩展。

NOSQL有如下特点：

- 模式自由：不像传统的关系型数据库需要定义数据库、数据表等结构才可以存取数据，数据表中的每一条记录都可能有不同的属性和格式；
- 逆范式：去除约束，降低事务要求，更利于数据的分布式存储，与MySQL范式相反；
- 多分区存储：存储在多个节点上，很好地进行水平扩展，提高数据的读、写性能；
- 多副本异步复制：为了保证数据的安全性，会保存数据的多个副本；
- 弹性可扩展：可以在系统运行过程中动态的增删节点，数据自动平衡移动，不需要人工的干预操作；
- 软事务：事务是关系型数据库的一个特点，NoSQL数据库不能完全满足事务的ACID特性，但是能保证事务的最终一致性；

NOSQL有一些理论支持：

- CAP理论：就是平衡一致性、可用性、分区容错性，因为最多只能同时实现2个，需要做平衡取舍；
- BASE模型：Basically Availble 基本可用，Soft state 状态可以有一段时间不同步，Eventual Consistency 最终一致性，不保证在任意时刻任意节点上的同一份数据都是相同的，但是随着时间的迁移，不同节点上的同一份数据总是在向趋同的方向变化；

HBase的特性

高性能

- 对每个 sstable使用布隆过滤器（Bloom Filter），以加速对数据在该 sstable的存在性进行判定，从而减少数据的总查询时间。

LSM树和B+树的差异主要在于读性能和写性能进行权衡，在牺牲的读性能的同时，寻找其余补救方案。

B+树存储引擎，不仅支持单条记录的增、删、读、改操作，还支持顺序扫描（B+树的叶子节点之间的指针），对应的存储系统就是关系数据库。但随着写入操作增多，为了维护B+树结构，节点分裂，读磁盘的随机读写概率会变大，读性能会逐渐减弱。

LSM树（Log-Structured MergeTree）存储引擎和B+树存储引擎一样，同样支持增、删、读、改、顺序扫描操作。而且通过批量存储技术规避磁盘随机写入问题。

当然凡事有利有弊，LSM树和B+树相比，LSM树牺牲了部分读性能，用来大幅提高写性能。

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HBase系统基本架构以及每个组成部分的作用

HBase系统基本架构包括客户端、Zookeeper服务器、Master主服务器、Region服务器

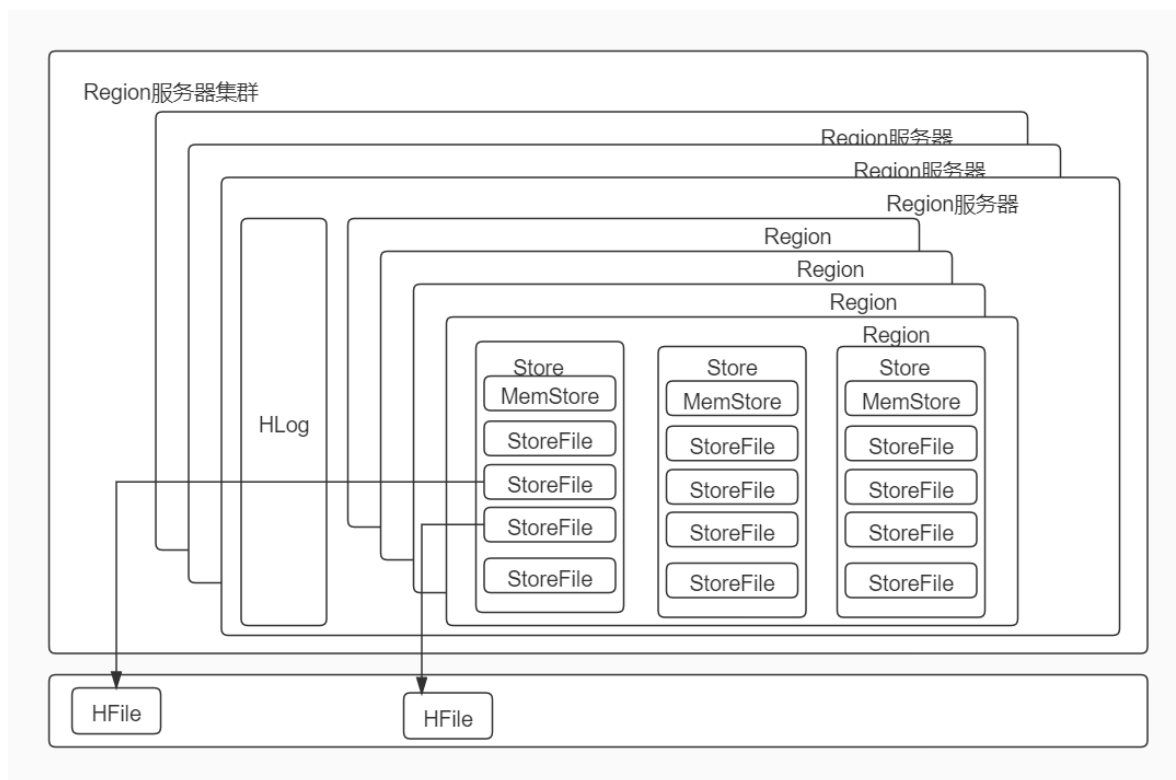
- 客户端：包含访问 HBase 的接口，缓存了已经访问过的 Region 位置信息，使用HBase的RPC机制与Master和Region服务器进行通信
- Zookeeper服务器：提供了稳定可靠的协同服务
 - 实时监控每个Region服务器的状态并通知给Master
 - 帮助选举出一个Master作为集群的总管，避免了Master的“单点失效”问题
 - 保存了-ROOT-表的地址和Master的地址
 - 存储了HBase的模式，包括有哪些表，每个表有哪些列族
- Master服务器：主要负责表和Region的管理工作
 - 管理用户对表的增加、删除、修改、查询等操作。
 - 实现不同Region服务器之间的负载均衡。
 - 在Region分裂或合并后，负责重新调整Region的分布。
 - 对发生故障失效的Region服务器上的Region进行迁移
- Region服务器：是HBase中最核心的模块，
 - 维护分配给自己的 Region，并响应用户的读写请求
 - HBase 自身并不具备数据复制和维护数据副本的功能，而是将HDFS 作为底层存储

HBase各功能组件及其作用

HBase包含一个Master主服务器和许多个Region服务器

- Region服务器：负责存储和维护分配给自己的Region，处理来自客户端的读写请求
- Master服务器：负责管理和维护 HBase表的分区信息。Master知道一个表被分成了哪些Region，每个Region被存放在哪台Region服务器上

Region服务器向HDFS文件系统中读写数据的基本原理



初始时，每个表只包含一个Region，随着数据的不断插入，Region会持续增大，当一个Region中包含的行数量达到一个阈值时，就会被自动等分成两个新的Region。随着表中行的数量继续增加，就会分裂出越来越多的Region

通过META表，META表中存了Region标识符和位置信息，Region标识符=表名 + 开始主键 + RegionID

在HBase三层结构下，客户端是如何访问到数据的？

客户端首先访问 Zookeeper，获取-ROOT-表的位置信息，然后访问-ROOT-表，获得.META.表的信息，接着访问.META.表，找到所需的Region具体位于哪个Region服务器，最后才会到该Region服务器读取数据

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

布隆过滤器

布隆过滤器对任意给定元素w，给出的存在性结果为两种：

- w**可能存在**于集合A中。
- w**肯定不在**集合A中。

布隆过滤器由一个长度为N的01数组array组成。首先将数组array每个元素初始设为0。

对集合A中的每个元素w，做K次哈希，第i次哈希值对N取模得到一个 index(i)，即 $\text{index}(i) = \text{HASH}_i(w) \% N$ ，将array数组中的array[index(i)]置为1。最终array变成一个某些元素为1的01数组。

HBase与布隆过滤器

由于布隆过滤器只需占用极小的空间，便可给出“可能存在”和“肯定不存在”的存在性判断，可以提前过滤掉很多不必要的数据块，从而节省了大量的磁盘IO。

HBase的Get操作就是通过运用低成本高效率的布隆过滤器来过滤大量无效数据块的，从而节省大量磁盘IO。

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HBase为什么要有列族(ColumnFamily)?

在HBase中，数据是按Column Family来分割的，同一个Column Family下的所有列的数据放在一个文件（为简化下面的描述在此使用文件这个词，在HBase内部使用的是Store）中。

为什么要这样子做呢？

HBase本身的设计目标是支持稀疏表，而稀疏表通常会有很多列，但是每一行有值的列又比较少。

如果不使用Column Family的概念，那么有两种设计方案：

1.把所有列的数据放在一个文件中（也就是**传统的按行存储**）。那么当我们想要访问少数几个列的数据时，需要遍历每一行，读取整个表的数据，这样子是很低效的。

2.把每个列的数据单独分开存在一个文件中（**按列存储**）。那么当我们想要访问少数几个列的数据时，只需要读取对应的文件，不用读取整个表的数据，读取效率很高。然而，由于稀疏表通常会有很多列，这会导致文件数量特别多，这本身会影响文件系统的效率。

而 Column Family的提出就是为了在上面两种方案中做一个折中。

HBase中将一个Column Family中的列存在一起，而不同Column Family的数据则分开。

由于在HBase中Column Family的数量通常很小，同时HBase建议把经常一起访问的比较类似的列放在同一个Column Family中，这样就可以在访问少数几个列时，只读取尽量少的数据。

HBase是一个分布式、可扩展、高性能的列式存储系统，基于Google的Bigtable设计。

HBase提供了一种高效的数据存储和查询方法，可以处理大量数据并提供快速的读写操作。

HBase的核心数据结构是列族(column family)，这一概念在HBase中非常重要，对于HBase的性能和可扩展性都有很大影响。

列族的定义

列族(column family)是HBase中最基本的数据结构，它是一组列(column)的集合。

假设我们有一张表（其中只有一条数据）：

RowKey	ColumnFamily : CF1		ColumnFamily : CF2		TimeStamp
	Column: C11	Column: C12	Column: C21	Column: C22	
"com.google"	"C11 good"	"C12 good"	"C12 bad"	"C12 bad"	T1

- 1) RowKey: 行键, 可理解成MySQL中的主键列。
- 2) Column: 列, 可理解成MySQL列。
- 3) ColumnFamily: 列族, HBase引入的概念

列族中的列具有相同的前缀，列族可以理解成一种逻辑上的分组。列族在HBase中有以下几个重要的特点：

1. 列族是HBase中数据存储的基本单位，一个表可以有多个列族。
2. 列族内的列名是有序的，列名的前缀相同，即属于同一个列族。
3. 列族在HBase中的存储结构是有序的，同一列族的数据会被存储在同一块磁盘空间上，这有助于提高读写性能。
4. 列族在HBase中的存储格式是列式存储，即同一列族中的列可以不同时存在，这有助于节省存储空间。

列族与列的关系

列族和列之间的关系是一种包含关系。列族包含了多个列，列具有唯一的列名和列值。

在HBase中，列名是由列族名和具体的列名组成的。

例如，如果有一个列族名为“user”，那么在这个列族下可以有多个列名，如“name”、“age”、“gender”等。

列族与行(row)的关系

列族和行之间的关系是一种多对一的关系。一个行可以包含多个列族，而一个列族可以包含多个行。

在HBase中，行是数据的唯一标识，每个行都有一个唯一的行键(rowkey)。同一个行可以包含多个列族，而同一个列族可以包含多个行。

HBase文件存储模型

在HBase中，每张表对应一个目录，

在表目录下，每个Region对应一个目录，

在Region目录下，每个Store对应一个目录（一个Store对应一个ColumFamily）。

结构如下:

HBase

1

---Table

1

---XXX(Region的目录)

1

----ColumnFamily (Store对应一个目录)

1

1

```
|          ---文件  
  
|  
  
---YYYYYY(另一个Region的目录)
```

不同的ColumnFamily对应不同的Store, 并且被写入了不同的目录, 这意味着:

1. 通过将一张表分解成了不同的ColumnFamily, HBase可以从磁盘一次读取更少的内容 (IO操作往往是计算机系统中最慢的一环)。
2. 我们不应该将需要一次查询出的列, 分解在不同的ColumnFamily中, 否则以为着HBase不得不读取两个文件来满足查询要求。

另外, 一个ColumnFamily中的每一列是连续存储的。即如果一个ColumnFamily中存在C1,C2两列, 一段具有100行记录的存储格式是:

```
c1(1),c2(1),c1(2),c2(2),c1(3),c2(3).....c1(100),c2(100)
```

与其说HBase是基于列的数据库, 更不如说HBase是基于“列族”的数据库。

HBase的核心数据结构

HBase的一个列簇 (Column Family) 本质上是一棵LSM树 (Log-Structured Merge-Tree)。

LSM树分为内存部分和磁盘部分。

内存部分是一个维护有序数据集的数据结构。

一般来讲, 内存数据结构可以选择平衡二叉树、红黑树、跳跃表 (SkipList) 等维护有序集的数据结构,

由于考虑并发性能, HBase选择了表现更优秀的跳表。

磁盘部分是由一个个独立的文件组成, 每一个文件又是由一个个数据块组成。

对于数据存储在磁盘上的数据库系统来说, 磁盘寻道以及数据读取都是非常耗时的操作 (简称IO耗时)。

为了避免不必要的IO耗时, 可以在磁盘中存储一些额外的二进制数据, 这些数据用来判断对于给定的key是否有可能存储在这个数据块中, 这个数据结构称为布隆过滤器 (BloomFilter)。

跳表(SkipList)

跳跃表 (SkipList) 是一种能高效实现插入、删除、查找的内存数据结构, 复杂度都是 $O(\log N)$ 。

与红黑树以及其他的二分查找树相比, 跳跃表的优势在于实现简单, 而且在并发场景下加锁粒度更小, 从而可以实现更高的并发性。

LSM树

LSM树是一种磁盘数据的索引结构。

LSM树的索引对写入请求更友好。因为无论是何种写入请求，LSM树都会将写入操作处理为一次顺序写，而HDFS擅长的正是顺序写（且HDFS不支持随机写）。

一个LSM树的索引主要由两部分构成：内存部分和磁盘部分。

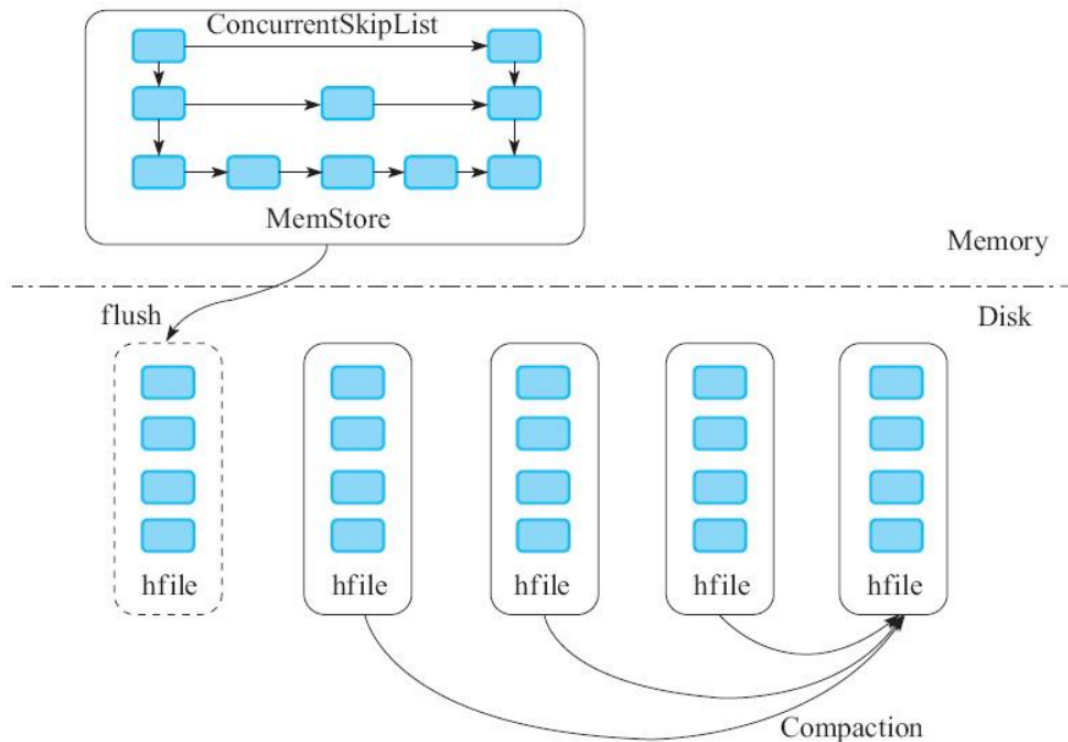
内存部分是一个ConcurrentSkipListMap，Key是rowkey、column family、qualifier、type以及timestamp，Value是字节数组。

随着数据不断写入MemStore，一旦内存超过阈值会将数据flush到磁盘，生产HFile；多个小HFile文件会compact成一个大HFile。

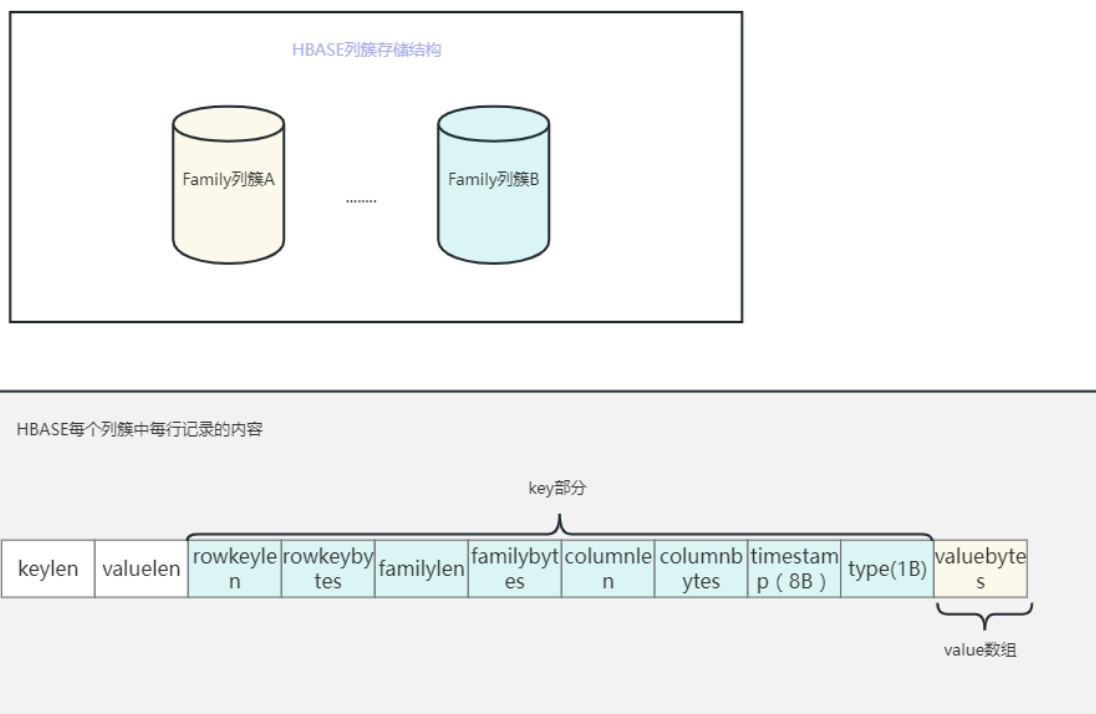
compact操作分成两种类型。

- major compact，是将所有的hfile一次合并成一个文件。好处是，合并之后只有一个文件，读取的性能好；但它的问题是，合并所有的文件可能需要很长的时间并消耗大量的IO带宽，所以major compact不宜使用太频繁，适合周期性地跑。
- minor compact，即选中少数几个hfile 合并成一个文件。优点是，可以进行局部的compact，通过少量的IO减少文件数量，提升读取操作的性能，适合较高频率地跑；但它的缺点是，只合并了局部的数据，对于那些全局删除操作，无法在合并过程中完全删除。

minor compact虽然能减少文件，但却无法彻底清除那些delete操作。而major compact能完全清理那些delete操作，保证数据的最小化。



Hbase对于每个列簇使用单独的文件存储，每个列簇的记录单独存放在一起，不同列簇的数据记录存放在不同的文件中，具体存放的记录格式如下所示：



其实由hbase的记录存储格式我们可以知道，hbase是一个KV存储的数据库，它擅长于通过key查找对应的value，当然它也可以适合应用于小范围的key扫描获取value的列表。

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HBase 的数据模型和索引模型

HBase它以行键（row key），列簇（columnFamily），列名（Column Qualifier）和时间戳（timestamp）为索引。

HBase 和 mysql 的索引的表面区别：

- 1、MySQL使用 行+ 列 定位数据
- 2、HBase类似于坐标系 (x,y,z)，**多维定位数据**
- 3、HBase 列簇、列名、时间戳，**面向列进行存储**

HBase 的NameSpace 命名空间 | 数据库

• 注意

- 1、**命名空间**是类似于关系数据库系统中的数据库的概念，他其实是**表的逻辑分组**。这种抽象为多租户相关功能奠定了基础。
- 2、命名空间是可以管理维护的，可以创建，删除或更改命名空间
- 3、HBase有**两个特殊预定义的命名空间**：
 - 1、**default** - 没有明确指定名称空间的表将自动落入此名称空间

- 2、**hbase** - 系统命名空间，用于包含HBase内部表
- **描述**
 - 1、为了保护我们的表，所采用的更高层次的单位
 - 2、表的逻辑分组

HBase 的Table 表

- **描述**
 - 1、存储相同数据的一个逻辑单元
 - 2、Hbase的table由**多个行**组成，行有很多**列**组成
 - 3、HBase是一个**半结构的数据库**，所以每一行的列都有可能是不同
- **图文**

Row Key	TimeStamp	Column Family1	Column Family2	Column Family3
111	t6		CF2:q1=val1	CF3:q3=val3
112	t3	CF1:q2=val3		
	t2		CF1:q8=val2	

HBase 的RowKey 主键

- **注意**
 - 1、**RowKey 是用来检索记录的主键，是一行数据的唯一标识**
 - 2、**RowKey行键** (RowKey)可以是**任意字符串**(最大长度是64KB，实际应用中长度一般为 10-100bytes)，RowKey以**字节数组**保存。
 - 3、存储时，数据按照RowKey的**字典序(byte order)排序存储**。设计RowKey时，要充分排序存储这个特性，将经常一起读取的行存储放到一起。
- **描述**
 - 1、可以理解为一行数据的主键
 - 2、HBase按照**列**进行存储，所以我们查询数据的时候会看到很多RowKey相同的列
 - 3、RowKey可以看做成主键：最多可以有64K字节组成，一般为 10~40 个字节即可
 - 将来设计 RowKey 是 HBase 使用的重中之重
 - 4、RowKey 存储的时候**默认以字典序排序**

Column Family 列簇

- **注意**
 - 1、列簇在物理上包含了许多的列与列的值，每个列簇都有一些存储的属性可配置
 - 例如是否使用缓存，压缩类型，存储版本数等。在表中，每一行都有相同的列簇，尽管有些列簇什么东西也没有存。
 - 2、将功能属性相近的列放在同一个列簇，而且同一个列簇中的列会存放在同一个Store中
 - 3、列簇一般需要在创建表的时候就进行声明，而且
 - 一般一个表中的列簇数不要超过3个
 - 这个和后期的优化相关
 - 4、列隶属于列簇，列簇隶属于表
- **描述**

- 1、可以理解为**多个列的集合**
- 2、方便队列进行**查找和管理**
- 3、一个表的列簇需要在使用前声明（创建表，后期维护表）
- 4、**列隶属于列簇，列簇隶属于表**
- 5、**一个表中列簇是固定的，但是列簇中列是不固定的**

Column Qualifier 列

- **注意**
 - 1、列簇的限定词，理解为列的唯一标识
 - 但是列标识是可以改变的，因此每一行可能有不同的列标识
 - 2、使用的时候必须 **列簇：列**
 - 3、列可以根据需求动态添加或者删除，**同一个表中不同行的数据列都可以不同**
- **描述**
 - 1、一个列簇中的列是**不固定的**
 - 2、又肯有的数据行一个列簇中有3个列，有的行相同列簇有8个列

Timestamp数据版本

- 1、通过rowkey和column family,column qualifier确定的一个存贮单元通过时间戳来索引。
 - **时间戳**的类型是 64 位整型
 - 默认时间戳是**精确到毫秒**的当前系统时间。
 - 时间戳也可以由客户显式赋值，如果应用程序要避免数据版本冲突，就必须自己生成**具有唯一性的时间戳**。
- 2、每个cell中，不同版本的数据按照时间倒序排序，即最新的数据排在最前面
 - 查询数据的时候，如果不指定版本数，**默认显示版本号最新（高）的数据**
- 3、为了避免数据存在过多版本中造成管理 (包括存贮和索引)负担，HBASE 提供了两种数据版本回收方式
 - 一是保存数据的**最后n个版本**
 - 二是保存**最近**一段时间内的版本（比如最近七天）。
- **描述**
 - 默认就是**时间戳**，解决HDFS不能随时修改数据的弊端
 - 默认数据版本就是时间戳
 - 查询数据的时候默认显示最新的数据

Cell 数据

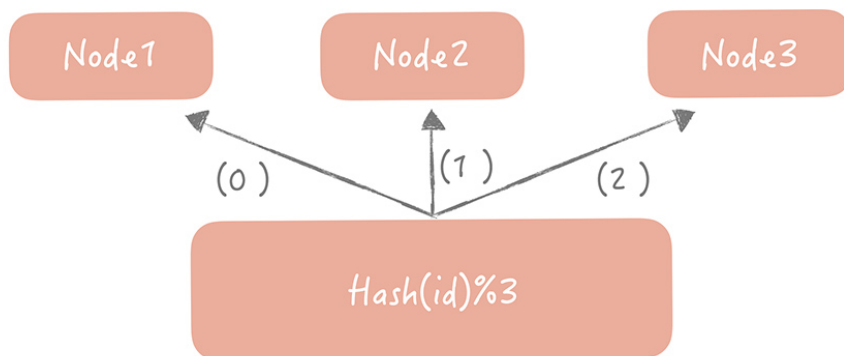
- **注意**
 - 1、**Cell是由row, column family,column qualifier,version 组成的**
 - 2、cell中的数据是没有类型的，全部是字节码形式存贮。
 - 因为HDFS上的数据都是**字节数组**
- **描述**
 - row, column family, column qualifier, version 所有的数据都是字符串

HBase的范围分片与rowkey路由

常见的数据分片方式一：Hash分片

Hash 分片的算法就是对缓存的 Key 做哈希计算，然后对总的缓存节点个数取余。

比如说，我们部署了三个缓存节点组成一个缓存的集群，当有新的数据要写入时，我们先对这个缓存的 Key 做比如 crc32 等 Hash 算法生成 Hash 值，然后对 Hash 值模 3，得出的结果就是要存入缓存节点的序号。



Hash分片算法示意图

这个算法最大的优点就是简单易理解，缺点是当增加或者减少缓存节点时，缓存总的节点个数变化造成计算出来的节点发生变化，从而造成缓存失效不可用。所以我建议你，如果采用这种方法，最好建立在你对于这组缓存命中率下降不敏感，比如下面还有另外一层缓存来兜底的情况下。那有没有更好的方案能解决这个问题呢？那就是一致性 Hash 分片算法。

一致性 Hash 分片算法, 请参考 尼恩的热门文章：

[得物面试：为啥Redis用哈希槽，不用一致性哈希？](#)

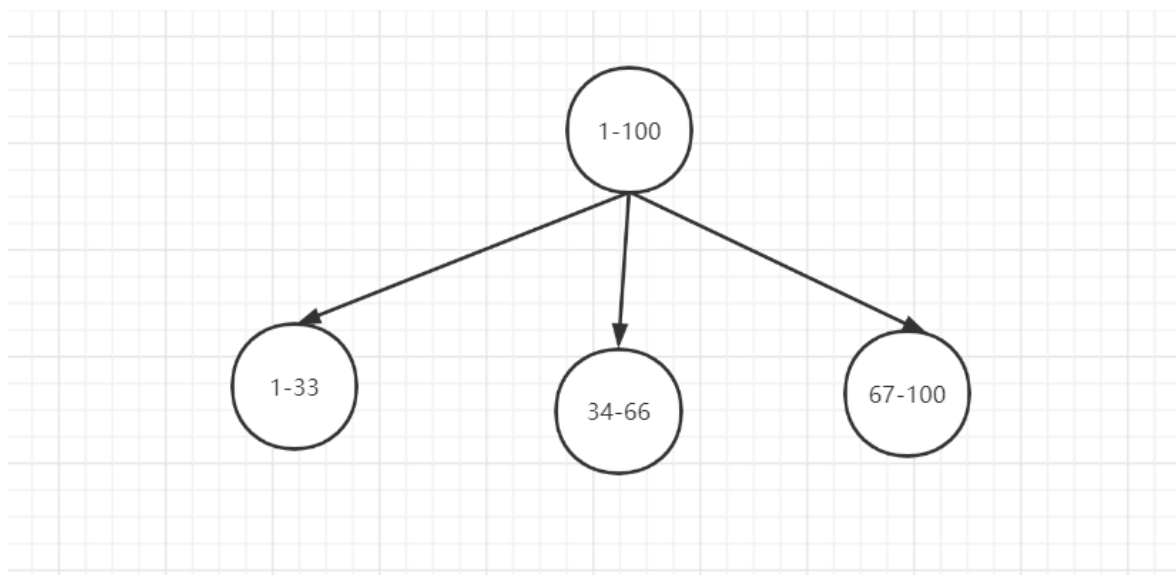
常见的数据分片方式二：范围分片

范围分片是按照 range 来分，就是每个片，一段连续的数据，这个一般是按比如时间范围/数据范围来。

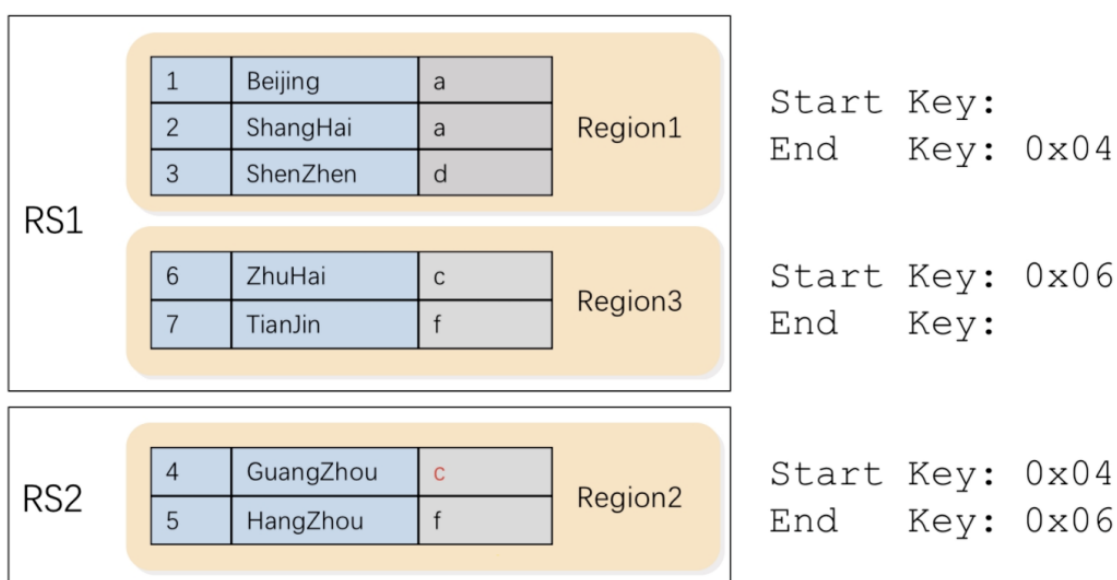
比如，安装数据范围分片，把1到100个数字，要保存在3个节点上

按照顺序分片，把数据平均分配三个节点上

- 1号到33号数据保存到节点1上
- 34号到66号数据保存到节点2上
- 67号到100号数据保存到节点3上



1. HBase分片模型:范围分片



在HBase中，分片是基于rowkey排序后，按照不同范围进行拆分的，即[startKey,endKey)这样一个左闭右开区间，每个分片称为一个Region。

1. 一个HBase集群中有多张表，每张表包含1个或者多个Region，每个Region有且只有一台机器进行映射，换言之，每台机器会承载0个或者多个Region，这里的机器在HBase中叫做RegionServer。
2. 由于Region中rowkey是已经排序好的，因此后一个Region的startKey实际上是前一个Region的endKey。并且在第一个Region中是没有startKey的，同理最后一个Region也是没有endkey的。所以当所有region组合在一起，就可以覆盖这个表中任意的rowkey数值。

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

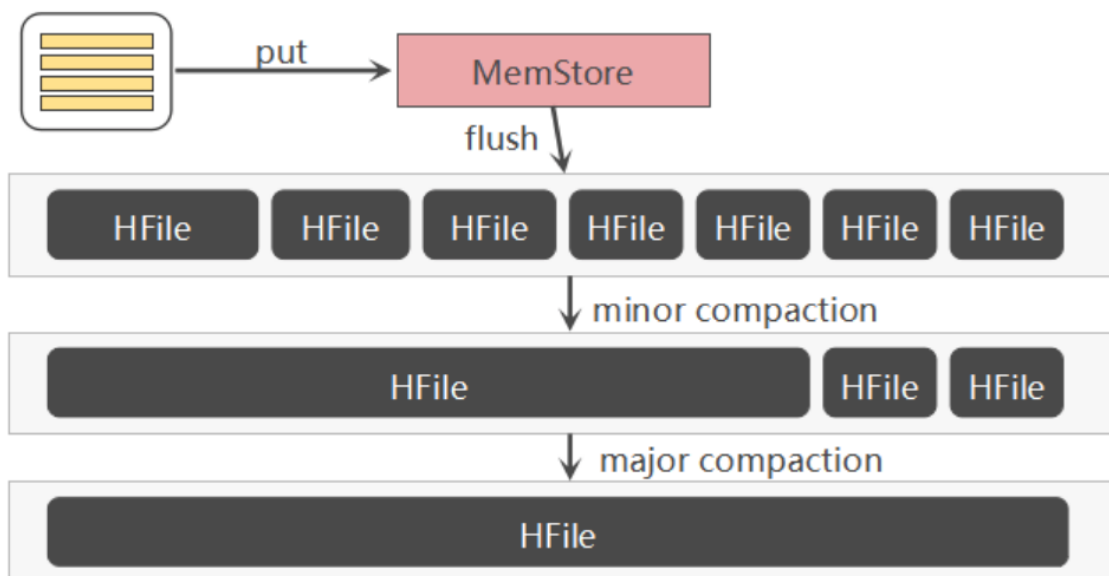
2. 元数据路由策略

- 在HBase中数据的查询，涉及两个层级的路由：一是rowkey到region的路由，二是region到RS的路由。两级路由信息均存放在.meta表中,meta表实际上也是稀疏索引，只记录了startKey和endKey的值，通过稀疏索引可以定位key对应Region的位置。

Row	Column Family	Column Qualifier	Value
table Name (表名) talbe produce epoch (建表的时间戳)	info	regioninfo	NAME =>(Region名字) STARTKEY => (起始key) ENDKEY => (结束key) ENCODED => (编码)
		server	RegionServer地址:Port
		serverstartcode	RegionServer启动的时间戳

3. LSM存储结构与优化

- HBase使用LSM(Log-Structured Merge Tree)的存储结构，将磁盘的随机IO转化为顺序IO来提高批量读写的性能，代价就是在点查询上性能有所牺牲。
- 在HBase中当写入一条数据后，率先会写入WAL，然后写入MemStore中。当Mem-Store满足一定条件后，开始flush数据到磁盘中，随着写入的不断增加，磁盘文件HFile也会越来越多，由于数据位置不确定，所以要遍历所有的HFile，因此在点查询时LSM树读性能时没有B+树好(这也是为什么在点查询上HBase不如Mysql的主要原因)。但是HBase也做了一定的优化，会定期合并若干个HFile，即多个文件合并成1个文件，以此来提高读性能。



4. 更加灵活的调度

- 在HBase中每个Region内部都是有序的，当Region过大或者有Hot Key出现时，会按照相应规则切分Region，此时就不必受hash函数的制约，Region可以自由的拆分和迁移。
- HBase存储层和计算层实际上是分离的，这也是现在主流架构，因此当Region迁移时不需要迁移物理数据，因此迁移成本很低。

那么现有系统中有哪个系统使用范围分片的方法进行路由分片呢？

实际上有很多，如下图所示。

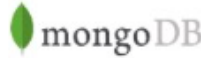
❑ NoSQL

- ❑ Bigtable
- ❑ HBase
- ❑ Cassandra
- ❑ Scylla
- ❑ MongoDB



❑ Storage Engine

- ❑ LevelDB
- ❑ RocksDB
- ❑ MyRocks



❑ NewSQL

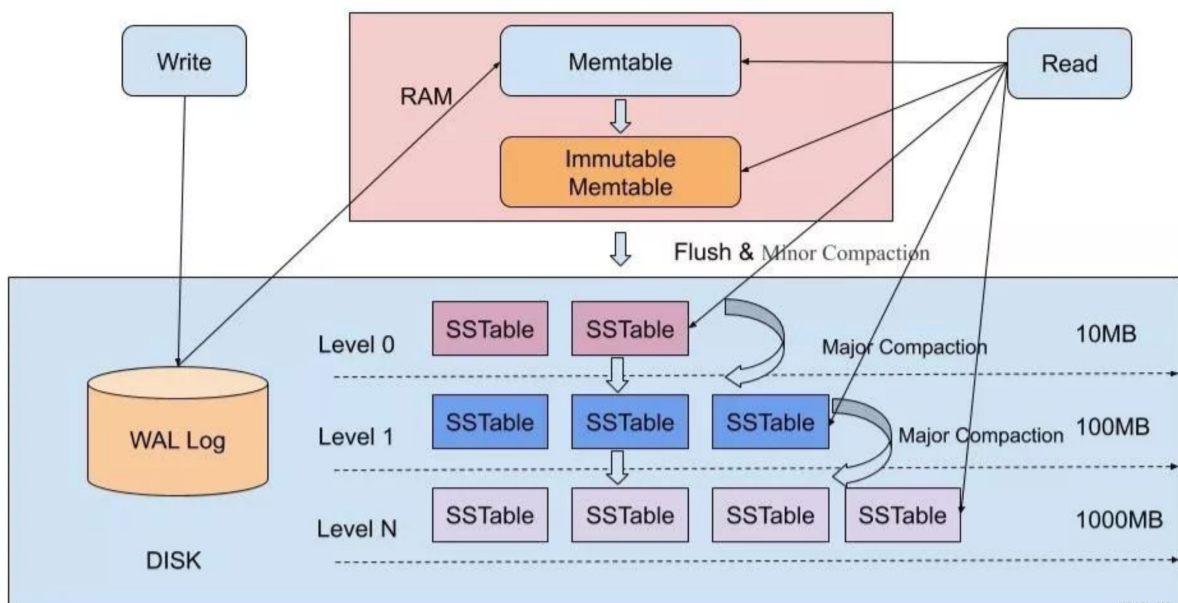
- ❑ TiDB(TiKV)
- ❑ CockroachDB



HBASE 的读写流程

先看 LSM 树读写架构图

Log Structured Merge Trees



LSM树的结构是横跨内存和磁盘的，包含 memtable、immutable memtable、SSTable 等多个部分。

- **memtable**

顾名思义，memtable是在内存中的数据结构，用以保存最近的一些更新操作，当写数据到memtable中时，会先通过WAL的方式备份到磁盘中，以防数据因为内存掉电而丢失。memtable可以使用跳跃表或者搜索树等数据结构来组织数据以保持数据的有序性。当memtable达到一定的数据量后，memtable会转化成为immutable memtable，同时会创建一个新的memtable来处理新的数据。

- **immutable memtable**

顾名思义，immutable memtable在内存中是不可修改的数据结构，它是将memtable转变为SSTable的一种中间状态。目的是为了在转存过程中不阻塞写操作。写操作可以由新的memtable处理，而不用因为锁住memtable而等待。

- **SSTable**

SSTable(Sorted String Table)即为有序键值对集合，是LSM树组在磁盘中的数据的结构。如果

SSTable比较大的时候，还可以根据键的值建立一个索引来加速SSTable的查询。下图是一个简单的SSTable结构示意图：

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HBase上Regionserver 的缓存

性能不够，缓存来凑。

HBase上Regionserver的内存分为两个部分：

- 一部分作为Memstore，主要用来写；
- 另外一部分作为BlockCache，主要用于读。

写请求会先写入Memstore，Regionserver会给每个region提供一个Memstore，当Memstore满64MB以后，会启动 flush刷新到磁盘。当Memstore的总大小超过限制时 ($\text{heapsize} * \text{hbase.regionserver.global.memstore.upperLimit} * 0.9$)，会强行启动flush进程，从最大的Memstore开始flush直到低于限制。

读请求先到Memstore中查数据，查不到就到BlockCache中查，再查不到就会到磁盘上读，并把读的结果放入BlockCache。由于BlockCache采用的是LRU策略，因此BlockCache达到上限($\text{heapsize} * \text{hfile.block.cache.size} * 0.85$)后，会启动淘汰机制，淘汰掉最老的一批数据。

HBASE 读操作的流程

HBASE 读操作 类似于分布式的 LSM 读操作。

由于hbase读取数据的时候需要从多个有序的文件(内存memstore+磁盘hfile)从查找数据，所以hbase的读数据性能比较有限，不过hbase内部做了一些优化，如使用 布隆过滤器 判断一个元素是否存在，如使用blockcache缓存hfile的数据，以及合并多个小的hfile小文件成一个大的hfile文件等，

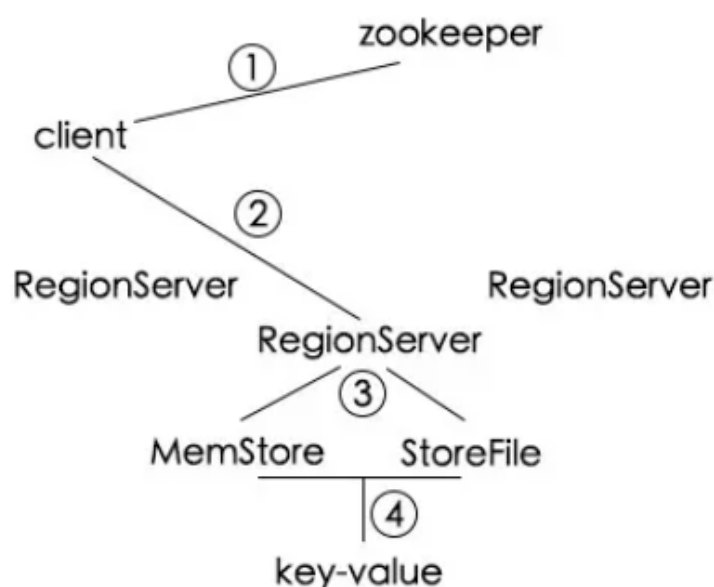
HBASE 读流程如下：

client --> zk --> root表所在的rs服务器->通过root表查找到meta元数据表后，根据要读取的数据所在的位置从meta元数据表中查找到对应数据所在的rs服务器->从rs服务器的region中（memstore+多个hfile文件）结合中读取数据

①HBase Client 要读数据了，先从 Zookeeper 中拿到 Meta 表信息，根据要查的 Rowkey 找到对应的数据在哪些 RegionServer 上。

②分别在这些 RegionServer 上根据列簇进行 StoreFile 和 MemStore 的查找，得到很多 key-value 结构的数据。

③根据数据的版本找到 数据进行返回。



- 1、从zookeeper拿到meta信息
- 2、找到数据分布的RegionServer
- 3、分别在MemStore和StoreFile查找
- 4、根据数据版本组织key-value进行返回

LSM 读操作的流程

一个storefile 类似一个 LSM 。

LSM 读操作，相较于B+树就会很慢，读操作需要依次读取memtable、immutable memtable、SSTable0、SSTable1....。

需要反序地遍历所有的集合，又因为写入顺序和合并顺序的缘故，序号小的集合中的数据一定会比序号大的集合中的数据新。

所以在这个反序遍历的过程中一旦匹配到了要读取的数据，那么一定是最新的数据，只要返回该数据即可。

但是如果一个数据的确不在所有的数据集合中，则会白白得遍历一遍。

读操作看上去比较笨拙，所幸可以通过布隆过滤器来加速读操作。

当布隆过滤器显示相应的SSTable中没有要读取的数据时，就跳过该SSTable。

布隆过滤器实际上是一个很长的二进制向量和一系列随机映射函数。

布隆过滤器可以用于检索一个元素是否在一个集合中。它的优点是空间效率和查询时间都远远超过一般的算法，缺点是有一定的误识别率和删除困难。

HBASE 写操作的流程

HBASE 读操作 类似于分布式的 LSM 读操作。

HBASE 整个写入流程如下：

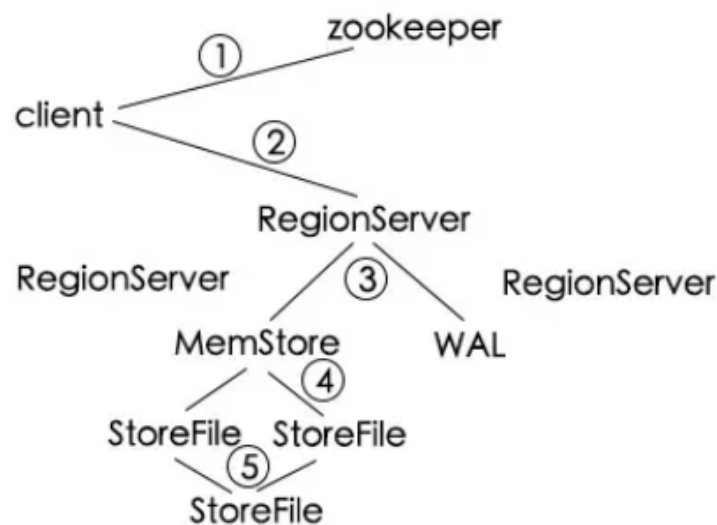
client --> zk --> root表所在的rs服务器-->通过root表查找到meta 元数据表后，根据要写入的数据所在的位置从meta元数据表中查找到写入数据对应的rs服务器-->数据写入rs服务器的memstore内存

①HBase Client 要写了，先从 Zookeeper 中拿到 Meta 表信息，根据数据的 Rowkey 找到应该往哪个 RegionServer 写。

②然后 HBase 会将数据写入对应 RegionServer 的内存 MemStore 中，同时记录操作日志 WAL。

③当 MemStore 超过一定阈值，就会将内存 MemStore 中的数据刷写到硬盘上，形成 StoreFile。

④在触发了一定条件的时候，小的 StoreFile 会进行合并，变成大的 StoreFile，有利于 HDFS 存储。



- 1、从zookeeper拿到meta表信息
- 2、根据rowkey确定往哪个RegionServer写
- 3、写MemStore内存同时记录日志
- 4、内存满后刷写StoreFile
- 5、StoreFile一定条件下触发合并

LSM 写操作的流程

一个storefile 类似一个 LSM。

写操作首先需要通过WAL将数据写入到磁盘Log中，防止数据丢失，然后数据会被写入到内存的memtable中，这样一次写操作即已经完成了，只需要1次磁盘IO，再加1次内存操作。

相较于B+树的多次磁盘随机IO，大大提高了效率。随后这些在memtable中的数据会被批量的合并到磁盘中的SSTable当中，将随机写变为了顺序写。

hbase 写数据速度极高，吞吐量极大，

主要归因于写入数据是先写入memstore内存的操作(当然这里还有wal顺序写日志防止机器崩溃)，

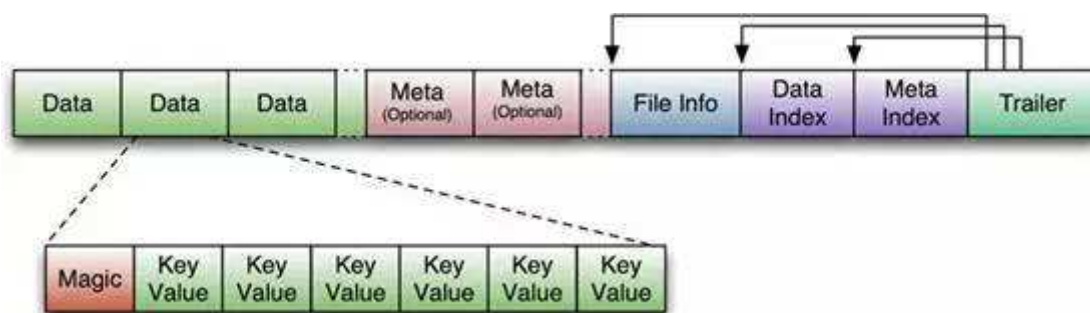
只有等到内存满了，才会批量顺序flush数据到磁盘，这样就避免了磁盘的随机io，

40岁老架构师尼恩提示：以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

HFile的三层索引

最初的HFile格式(HFile V1)，参考了Bigtable的SSTable以及Hadoop的TFile(HADOOP-3315)。

HFile V1 如下图所示：

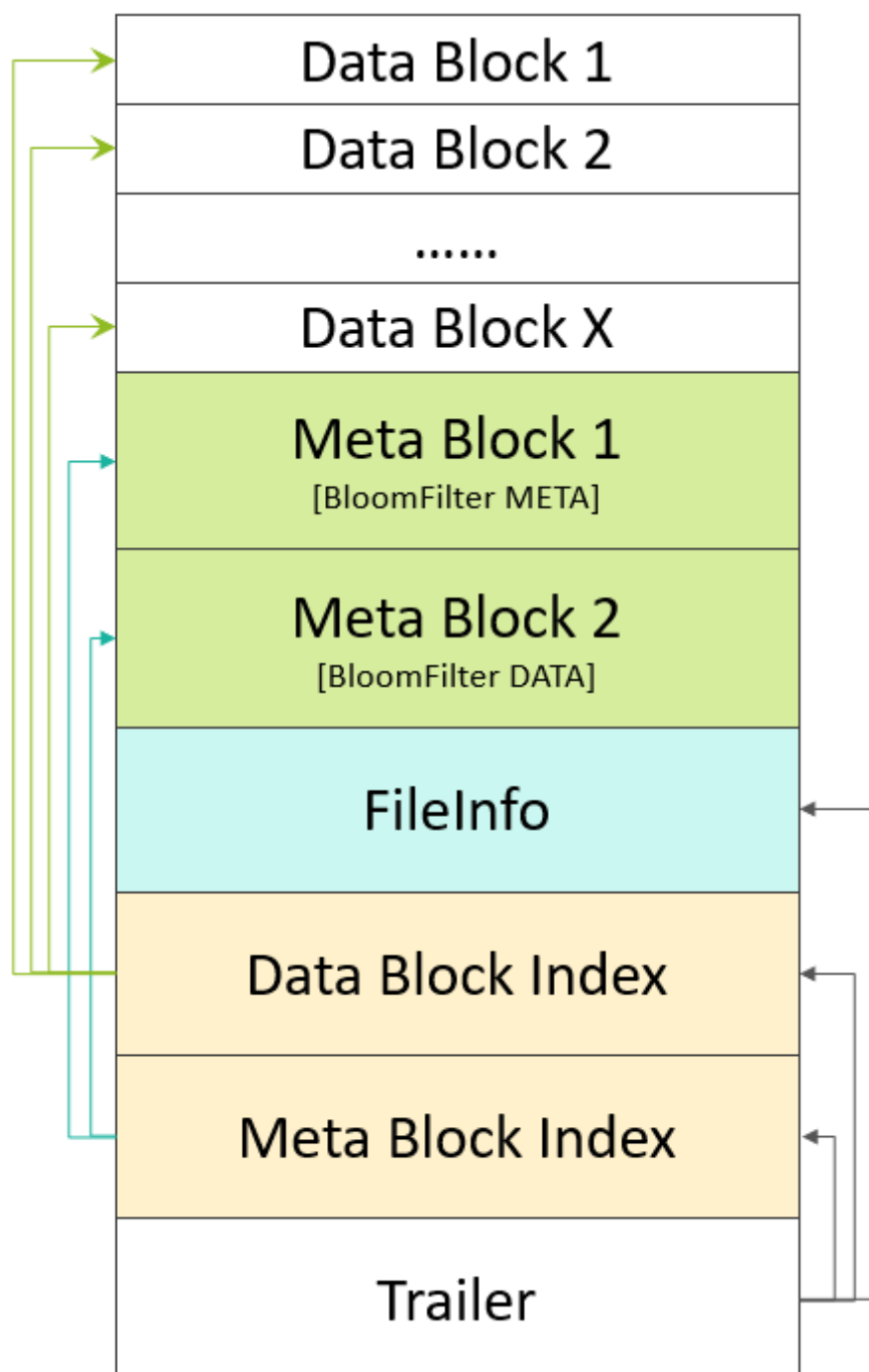


HFile在生成之前，数据在内存中已经是按序组织的。

存放用户数据的KeyValue，被存储在一个个默认为64kb大小的Data Block中，在Data Index部分存储了每一个Data Block的索引信息{Offset, Size, FirstKey}，而Data Index的索引信息{Data Index Offset, Data Block Count}被存储在HFile的Trailer部分。

除此以外，在Meta Block部分还存储了Bloom Filter的数据。

下图更直观的表达出了HFile V1中的数据组织结构：



这种设计简单、直观。

但用，对于这个HFile版本所存在的问题应该深有痛楚：Region Open的时候，需要加载所有的Data Block Index数据，另外，第一次读取时需要加载所有的Bloom Filter数据到内存中。

一个HFile中的Bloom Filter的数据大小可达百MB级别，一个RegionServer启动时可能需要加载数GB的Data Block Index数据。

这在一个大数据量的集群中，几乎无法忍受。

对于这个HFile版本所存在的问题应该深有痛楚：Region Open的时候，需要加载所有的Data Block Index数据，另外，第一次读取时需要加载所有的Bloom Filter数据到内存中。一个HFile中的Bloom Filter的数据大小可达百MB级别，一个RegionServer启动时可能需要加载数GB的Data Block Index数据。这在一个大数据量的集群中，几乎无法忍受。

Data Block Index究竟有多大？

一个Data Block在Data Block Index中的索引信息包含{Offset, Size, FirstKey}, BlockOffset使用Long型数字表示, Size使用Int表示即可。假设用户数据RowKey的长度为50bytes, 那么, 一个64KB的Data Block在Data Block Index中的一条索引数据大小约为62字节。

假设一个RegionServer中有500个Region, 每一个Region的数量为10GB (假设这是Data Blocks的总大小), 在这个RegionServer上, 约有81920000个Data Blocks, 此时, Data Block Index所占用的大小为81920000*62bytes, 约为4.7GB。

这是HFile V2设计的初衷, HFile V2期望显著降低RegionServer启动时加载HFile的时延, 更希望解决一次全量加载数百MB级别的BloomFilter数据带来的时延过大的问题。

下图是HFile V2的数据组织结构:

"Scanned block" section	Data Block		
	...		
	Leaf index block / Bloom block		
	...		
	Data Block		
	...		
	Leaf index block / Bloom block		
	...		
	Data Block		
"Non-scanned block" section	Meta block	...	Meta block
	Intermediate Level Data Index Blocks (optional)		
"Load-on-open" section	Root Data Index		Fields for midkey
	Meta Index		
	File Info		
	Bloom filter metadata (interpreted by StoreFile)		
Trailer	Trailer fields		Version

HFile V2文件主要分为四个部分: **Scanned block section**, **Non-scanned block section**, **Opening-time data section**和**Trailer**。

- **Scanned block section**: 顾名思义, 表示顺序扫描HFile时所有的数据块将会被读取, 包括Leaf Index Block和Bloom Block。
- **Non-scanned block section**: 表示在HFile顺序扫描的时候数据不会被读取, 主要包括Meta Block和Intermediate Level Data Index Blocks两部分。
- **Load-on-open-section**: 这部分数据在HBase的region server启动时, 需要加载到内存中。包括FileInfo、Bloom filter block、data block index和meta block index。
- **Trailer**: 这部分主要记录了HFile的基本信息、各个部分的偏移值和寻址信息。

较之HFile V1, 我们来看看HFile V2的几点显著变化:

• 分层索引

无论是Data Block Index还是Bloom Filter, 都采用了**分层索引**的设计。

Data Block的索引, 在HFile V2中最多可支持三层索引: 最底层的数据块索引称之为Leaf Index Block, 可直接索引到Data Block; 中间层称之为Intermediate Index Block, 最上层称之为Root Data Index, Root Data index存放在一个称之为**"Load-on-open Section"**区域, Region Open时会被加载到内存中。基本的索引逻辑为: 由Root Data Index索引到Intermediate Block Index, 再由Intermediate Block Index索引到Leaf Index Block, 最后由Leaf Index Block查找到对应的Data

Block。在实际场景中，**Intermediate Block Index基本上不会存在**，文末部分会通过详细的计算阐述它基本不存在的原因，因此，索引逻辑被简化为：由Root Data Index直接索引到Leaf Index Block，再由Leaf Index Block查找到的对应的Data Block。

Bloom Filter也被拆成了多个Bloom Block，在“Load-on-open Section”区域中，同样存放了所有Bloom Block的索引数据

- **交叉存放**

在“**Scanned Block Section**”区域，Data Block(存放用户数据KeyValue)、存放Data Block索引的Leaf Index Block(存放Data Block的索引)与Bloom Block(Bloom Filter数据)交叉存在。

- **按需读取**

无论是Data Block的索引数据，还是Bloom Filter数据，都被拆成了多个Block，基于这样的设计，无论是索引数据，还是Bloom Filter，都可以**按需读取**，避免在Region Open阶段或读取阶段一次读入大量的数据，**有效降低时延**。

从0.98版本开始，社区引入了HFile V3版本，主要是为了支持Tag特性，在HFile V2基础上只做了微量改动。在下文内容中，主要围绕**HFile V2**的设计展开。

为什么？sstable有序文件为什么不支持 二分查找？

sstable这样的数据存储是有序的，是按照key的顺序有序存放的记录集合，

问题是，原生的sstable里面的记录不是本来就有序的，直接用二分查找不是也可以解决问题吗？

维护这个key的索引结构的意义为啥？

为什么说sstable等已经按照key拍好序的文件不支持二分查找呢？

SStable等有序文件不直接直接在数据文件中通过key进行二分查找主要原因是由于每个kv记录的长度不一样，如果支持直接在数据文件中进行二分查找的话需要较大的改造成本，并且需要额外的存储空间，

所以 sstable有序文件 不支持 二分查找。

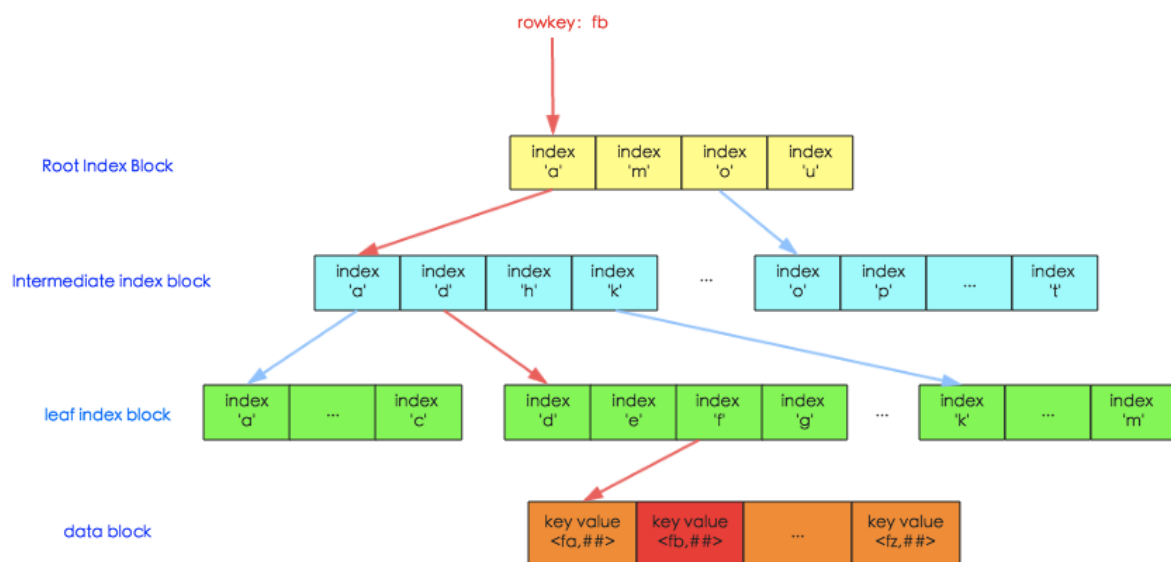
而通过把key抽取出来创建另外的三到四层高度的索引结构，性能上要比二分查找高得多，并且原数据文件也无需改造，直接按序顺序写入即可

HFile的三层索引结构

所以，hbase等还会根据key来创建一个索引结构，这样就可以通过key查找索引后，快速定位到对应的数据记录。

整个索引体系类似于MySQL的B+树结构，但是又有所不同，比B+树简单，并没有复杂的分裂操作。

具体见下图所示：



图中上面三层为索引层，在数据量不大的时候只有最上面一层，数据量大了之后开始分裂为多层，最多三层，如图所示。

最下面一层为数据层，存储用户的实际keyvalue数据。

这个索引树结构类似于InnoDB的聚集索引，只是HBase并没有辅助索引的概念。

图中红线表示一次查询的索引过程（HBase中相关类为HFileBlockIndex和HFileReaderV2），基本流程可以表示为：

1. 用户输入rowkey为fb，在root index block中通过二分查找定位到fb在'a'和'm'之间，因此需要访问索引'a'指向的中间节点。因为root index block常驻内存，所以这个过程很快。
2. 将索引'a'指向的中间节点索引块加载到内存，然后通过二分查找定位到fb在index 'd'和'h'之间，接下来访问索引'd'指向的叶子节点。
3. 同理，将索引'd'指向的中间节点索引块加载到内存，一样通过二分查找定位找到fb在index 'f'和'g'之间，最后需要访问索引'f'指向的数据块节点。
4. 将索引'f'指向的数据块加载到内存，通过遍历的方式找到对应的keyvalue。

上述流程中因为中间节点、叶子节点和数据块都需要加载到内存，所以io次数正常为3次。

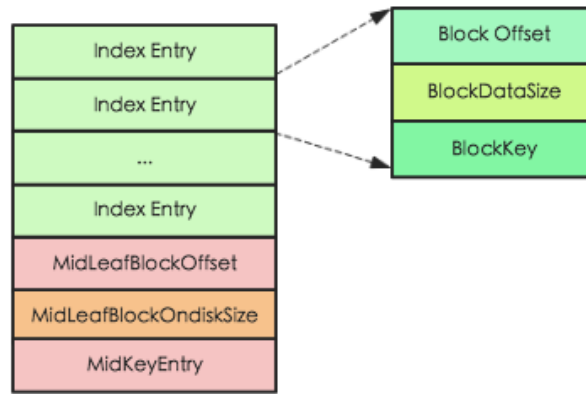
但是实际上HBase为block提供了缓存机制，可以将频繁使用的block缓存在内存中，可以进一步加快实际读取过程。

所以，在HBase中，通常一次随机读请求最多会产生3次io，如果数据量小（只有一层索引），数据已经缓存到了内存，就不会产生io。

Root Index Block结构

Root Index Block表示索引树根节点索引块，可以作为bloom的直接索引，也可以作为data索引的根索引。

而且对于single-level和mutil-level两种索引结构对应的Root Index Block略有不同，本文以mutil-level索引结构为例进行分析（single-level索引结构是mutual-level的一种简化场景），在内存和磁盘中的格式如下图所示：



其中Index Entry表示具体的索引对象，每个索引对象由3个字段组成，Block Offset表示索引指向数据块的偏移量，BlockDataSize表示索引指向数据块在磁盘上的大小，BlockKey表示索引指向数据块中的第一个key。

除此之外，还有另外3个字段用来记录MidKey的相关信息，MidKey表示HFile所有Data Block中中间的一个Data Block，用于在对HFile进行split操作时，快速定位HFile的中间位置。

需要注意的是single-level索引结构和muti-level结构相比，就只缺少MidKey这三个字段。

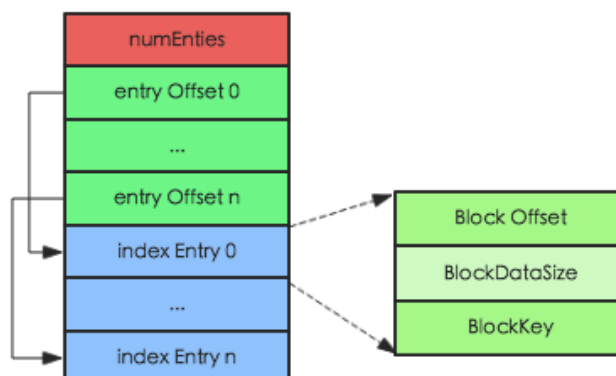
Root Index Block会在HFile解析的时候直接加载到内存中，此处需要注意在Trailer Block中有一个字段为dataIndexCount，就表示此处Index Entry的个数。

因为Index Entry并不定长，只有知道Entry的个数才能正确的将所有Index Entry加载到内存。

NonRoot Index Block结构

当HFile中Data Block越来越多，single-level结构的索引已经不足以支撑所有数据都加载到内存，需要分化为muti-level结构。

muti-level结构中NonRoot Index Block作为中间层节点或者叶子节点存在，无论是中间节点还是叶子节点，其都拥有相同的结构，如下图所示：



和Root Index Block相同，NonRoot Index Block中最核心的字段也是Index Entry，用于指向叶子节点块或者数据块。

不同的是，NonRoot Index Block结构中增加了block块的内部索引entry Offset字段，entry Offset表示index Entry在该block中的相对偏移量（相对于第一个index Entry），用于实现block内的二分查找。

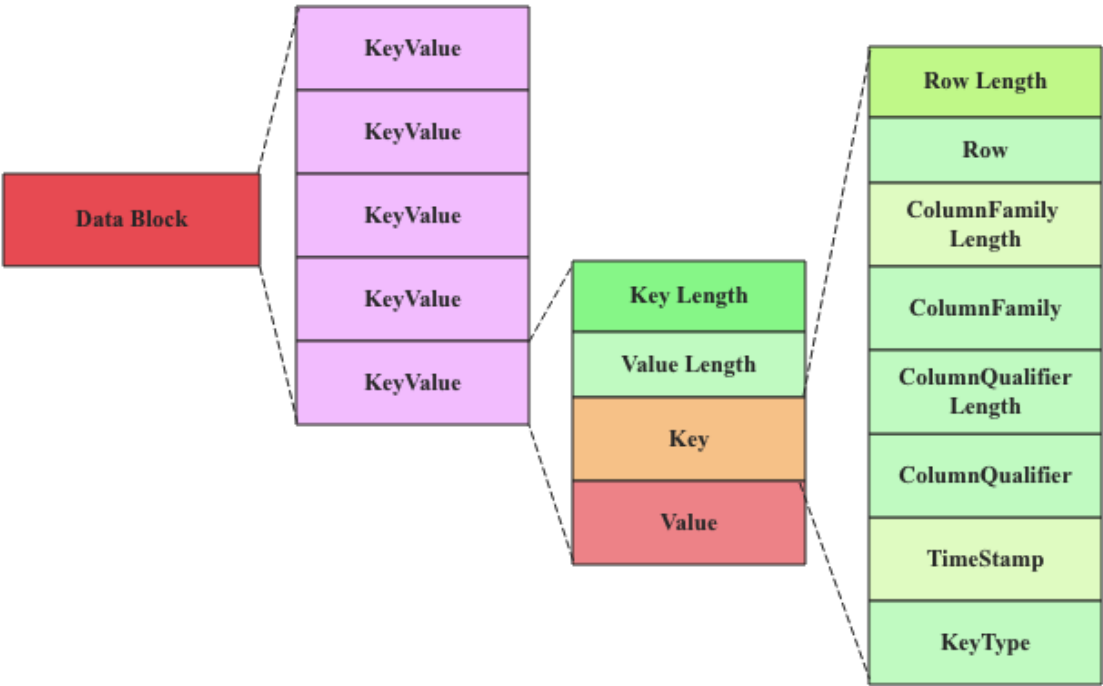
所有非根节点索引块，包括Intermediate index block和leaf index block，在其内部定位一个key的具体索引并不是通过遍历实现，而是使用二分查找算法，这样可以更加高效快速地定位到待查找key。

正由于 hfile 有三层索引的存在，所以 性能很高。

40岁老架构师尼恩提示： 以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频。

DataBlock结构

DataBlock是HBase中数据存储的最小单元。DataBlock中主要存储用户的KeyValue数据（KeyValue后面一般会跟一个timestamp，图中未标出），而KeyValue结构是HBase存储的核心，每个数据都是以KeyValue结构在HBase中进行存储。KeyValue结构在内存和磁盘中可以表示为：

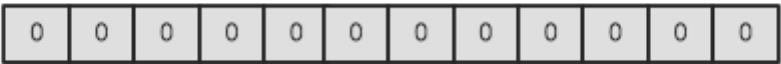


每个KeyValue都由4个部分构成，分别为key length，value length，key和value。其中key value和value length是两个固定长度的数值，而key是一个复杂的结构，首先是rowkey的长度，接着是rowkey，然后是ColumnFamily的长度，再是ColumnFamily，之后是ColumnQualifier，最后是时间戳和KeyType（keytype有四种类型，分别是Put、Delete、DeleteColumn和DeleteFamily），value就没有那么复杂，就是一串纯粹的二进制数据。

BloomFilter Meta Block & Bloom Block结构

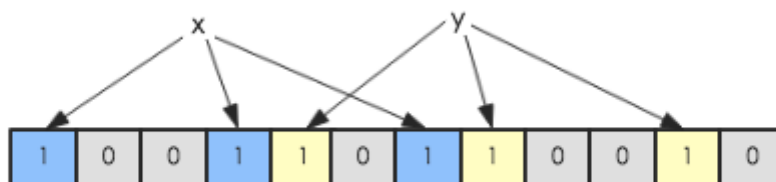
BloomFilter对于HBase的随机读性能至关重要，对于get操作以及部分scan操作可以剔除掉不会用到的HFile文件，减少实际IO次数，提高随机读性能。

在此简单地介绍一下Bloom Filter的工作原理，Bloom Filter使用位数组来实现过滤，初始状态下位数组每一位都为0，如下图所示：



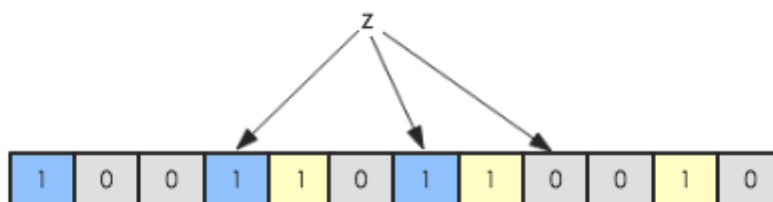
假如此时有一个集合 $S = \{x_1, x_2, \dots, x_n\}$ ，Bloom Filter使用 k 个独立的hash函数，分别将集合中的每一个元素映射到 $\{1, \dots, m\}$ 的范围。对于任何一个元素，被映射到的数字作为对应的位数组的索引，该位会被置为1。比如元素 x_1 被hash函数映射到数字8，那么位数组的第8位就会被置为1。

下图中集合 S 只有两个元素 x 和 y ，分别被3个hash函数进行映射，映射到的位置分别为 $(0, 3, 6)$ 和 $(4, 7, 10)$ ，对应的位会被置为1：



现在假如要判断另一个元素是否是在此集合中，只需要被这3个hash函数进行映射，查看对应的位置是否有0存在，如果有的话，表示此元素肯定不存在于这个集合，否则有可能存在。

下图所示就表示 z 肯定不在集合 $\{x, y\}$ 中：



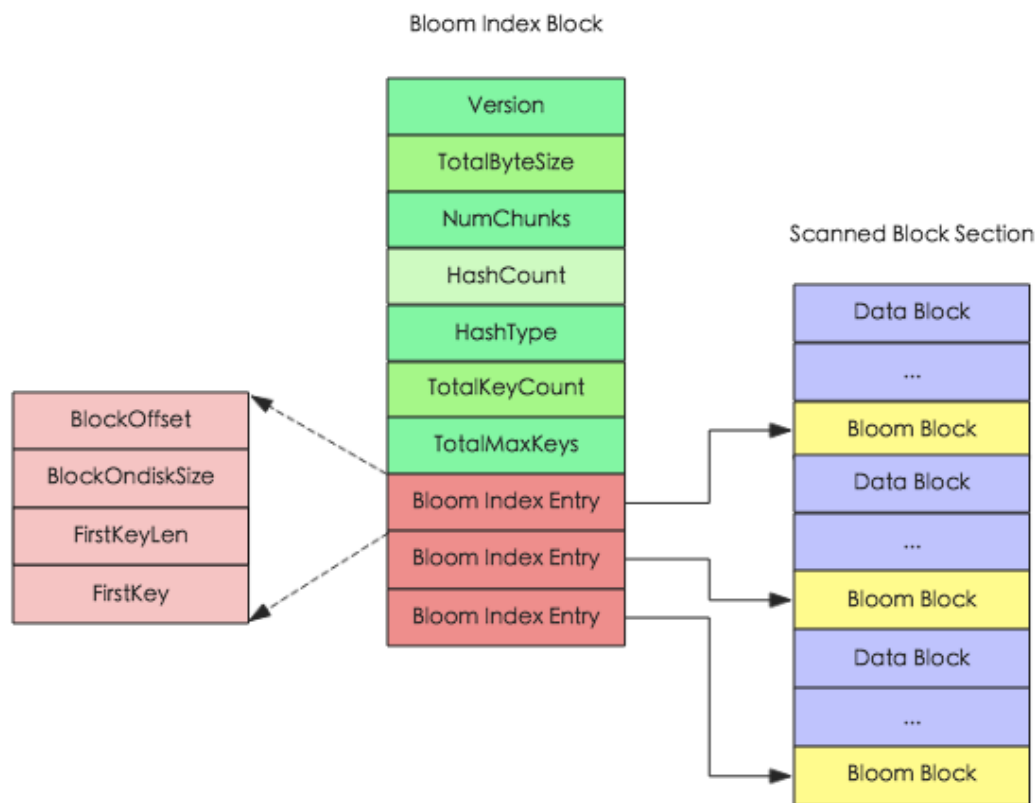
HBase中每个HFile都有对应的位数组，KeyValue在写入HFile时会先经过几个hash函数的映射，映射后将对应的数组位改为1，get请求进来之后再进行hash映射，如果在对应数组位上存在0，说明该get请求查询的数据不在该HFile中。

HFile中的位数组就是上述Bloom Block中存储的值，可以想象，一个HFile文件越大，里面存储的KeyValue值越多，位数组就会相应越大。

一旦太大就不适合直接加载到内存了，因此HFile V2在设计上将位数组进行了拆分，拆成了多个独立的位数组（根据Key进行拆分，一部分连续的Key使用一个位数组）。

这样一个HFile中就会包含多个位数组，根据Key进行查询，首先会定位到具体的某个位数组，只需要加载此位数组到内存进行过滤即可，减少了内存开支。

在结构上每个位数组对应HFile中一个Bloom Block，为了方便根据Key定位具体需要加载哪个位数组，HFile V2又设计了对应的索引Bloom Index Block，对应的内存和逻辑结构图如下：



Bloom Index Block结构中totalByteSize表示位数组的bit数，numChunks表示Bloom Block的个数，hashCount表示hash函数的个数，hashType表示hash函数的类型，totalKeyCount表示bloom filter当前已经包含的key的数目，totalMaxKeys表示bloom filter当前最多包含的key的数目，Bloom Index Entry对应每一个bloom filter block的索引条目，作为索引分别指向'scanned block section'部分的Bloom Block，Bloom Block中就存储了对应的位数组。

Bloom Index Entry的结构见上图左边所示，BlockOffset表示对应Bloom Block在HFile中的偏移量，FirstKey表示对应BloomBlock的第一个Key。

根据上文所说，一次get请求进来，首先会根据key在所有的索引条目中进行二分查找，查找到对应的Bloom Index Entry，就可以定位到该key对应的位数组，加载到内存进行过滤判断。

总之，二分查找 + Bloom 过滤，实现了HBase的高性能。

来看看：HBase的高性能指标140000+ ops

HBase 底层基于HDFS存储，高可用、高扩展、强一致性，支持上亿级别数据

很多人对HBase的印象可能都是写性能很好、读性能很差，但实际上HBase的读性能远远超过大家的预期。

10亿数据 性能基准测试结果

1. 写性能：集群吞吐量最大可以达到70000+ ops/sec，延迟在几个毫秒左右。网络带宽是主要瓶颈，如果将千兆网卡换成万兆网卡，吞吐量还可以继续增加，甚至达到目前吞吐量的两倍140000+ ops/sec。
2. 读性能：很多人对HBase的印象可能都是写性能很好、读性能很差，但实际上HBase的读性能远远超过大家的预期。集群吞吐量最大可以达到26000+，单台吞吐量可以达到8000+左右，延迟在几毫秒~20毫秒左右。IO和CPU是主要瓶颈。
3. Range 扫描性能：集群吞吐量最大可以达到14000左右，系统平均延迟在几毫秒~60毫秒之间（线程数越多，延迟越大）；其中IO和网络带宽是主要瓶颈。

缺点：

占用内存很大，且鉴于建立在为批量分析而优化的HDFS上，导致读取性能不高

测试环境的集群拓扑结构

本次测试中，测试环境总共包含4台SA5212H2物理机作为数据存储。生成数据的YCSB程序与数据库并不运行在相同的物理集群。

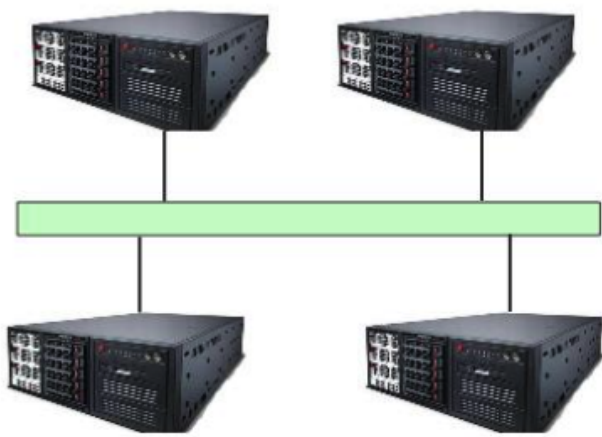


图 1 集群拓扑结构图

单台机器主机硬件配置

Name	Value
CPU 规格	Intel(R) Xeon(R) CPU E5-2440 v2 @ 1.90GHz
CPU 核数	32 核
Memory	96G
磁盘	3.6T * 12

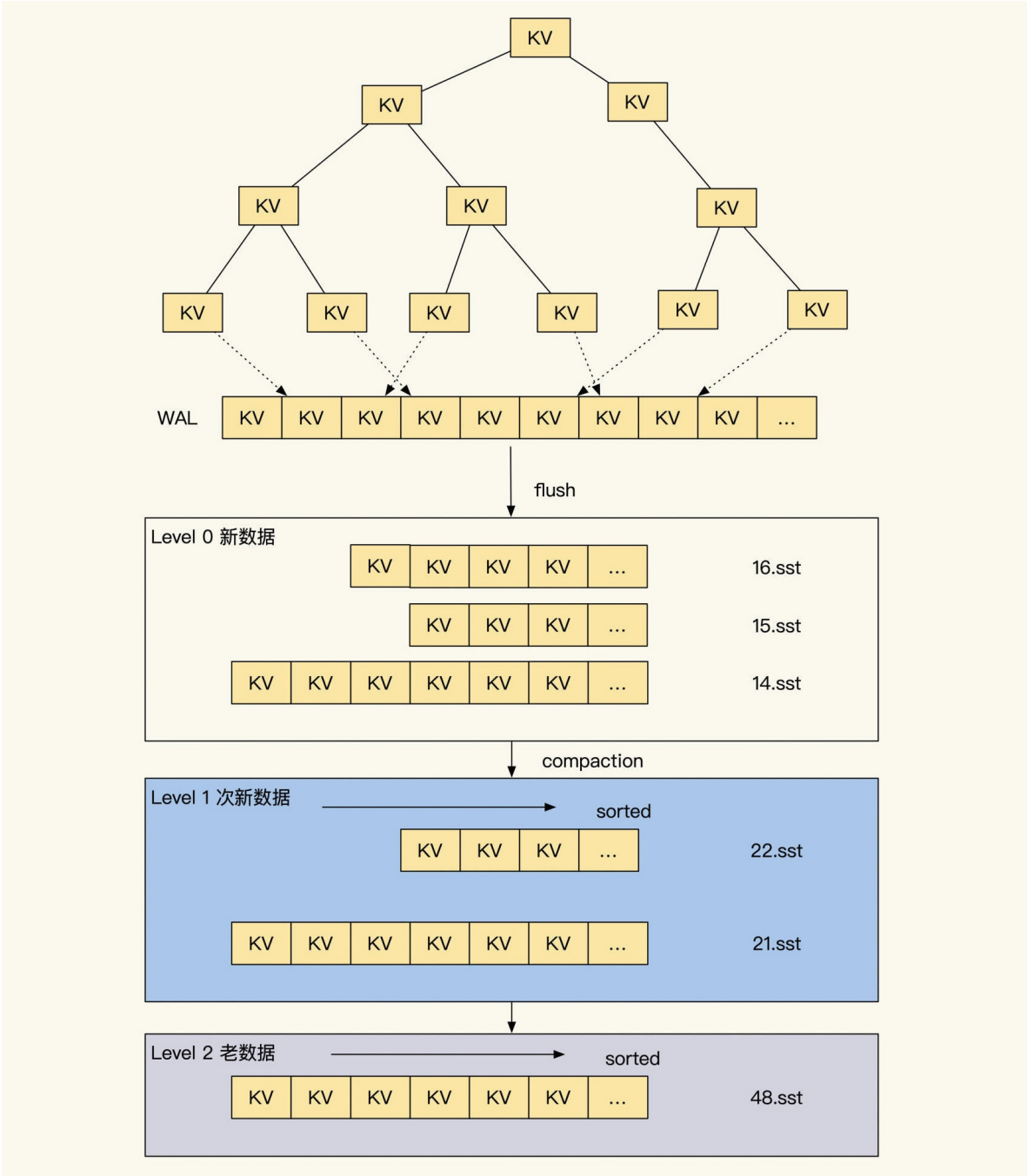
表 1 单台主机硬件配置表

40岁老架构师尼恩提示： 以上内容比较复杂，具体请参见后面发布的《百亿级海量数据存储架构》配套视频

附录： RocksDB 的 LSM Tree与存储

RocksDB之所以能够得到快速推广并受到欢迎，主要是因为它利用了 **磁盘顺序写** 的超绝特性，并以较小的性能查询代价提供了写多读少的KV数据存储查询服务，这和关系数据库的存储有很大的不同。

Rocksdb稀疏索引是如何实现的，如下图所示：

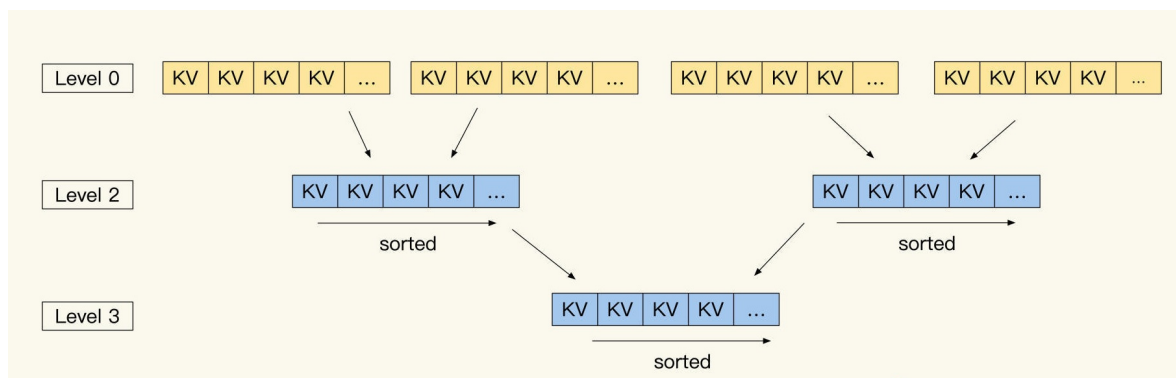


RocksDB LSM 是由一棵棵小树组成，最小的树在内存，当我们新数据写入时，会在内存中暂存，这样能够获得非常大的写并发处理能力。

而当内存中数据积累到一定程度后，会将内存中数据和索引做 **顺序写**，落地形成一个数据块。

这个数据块内保存着一棵小树和具体的数据，新生成的数据块会保存在Level 0 层（最大有几层可配置），Level 0 层会有多个类似的数据块文件。

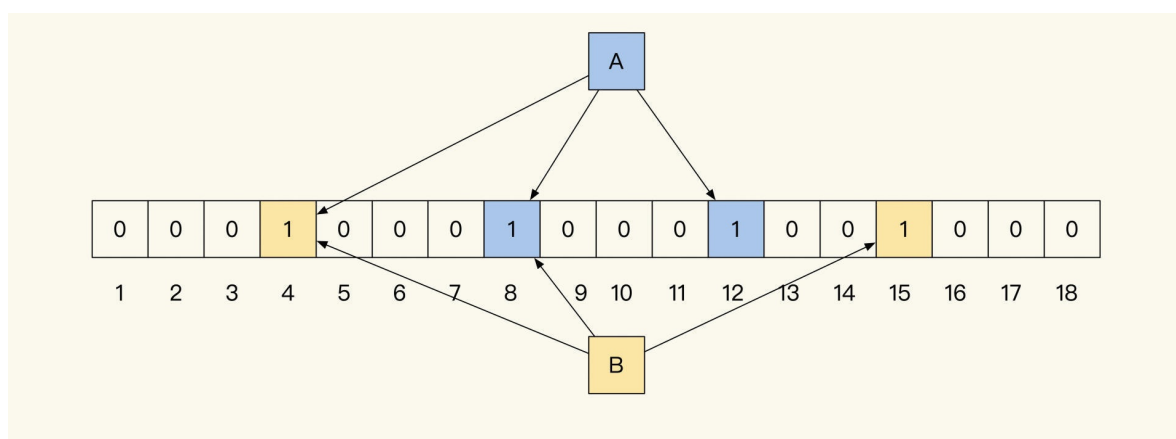
结构如下图所示：



每一层的数据块和数据量超过一定程度时，RocksDB合并不同Level的数据，将多个数据块内的数据和索引合并在一起，并推送到Level的下一层。

通过这个方式，每一层的数据块个数和数据量就能保持一定的数量，合并后的数据会更紧密、更容易被找到。

这样的设计，可以让一个Key存在于多个Level或者数据块中，但是最新的常用的数据肯定是在Level最顶部或内存（0~4层，0为顶部）中最新的数据块内。



而当我们查询一个key的时候，RocksDB会先查内存。

如果没找到，会从Level 0层到下层，每层按生成最新到最老的顺序去查询每层的数据块。

同时为了减少IO次数，每个数据块都会有一个BloomFilter辅助索引，来辅助确认这个数据块中是否 **可能有** 对应的Key；如果当前数据块没有，那么可以快速去找下一个数据块，直到找到为止。

从0到1，百亿级数据存储架构，怎么设计？

从0到1，百亿级数据存储架构，40岁老架构尼恩团队，计划用一个系列的文章帮大家实现这个架构难题，这个系列还会录成视频，并辅导大家写入简历。

这个系列包括：

- 高并发搜索ES圣经：从0到1，从入门到 ElasticSearch 工业级使用
- **100亿级任务调度篇：从0到1，从入门到 XXLJOB 工业级使用（已经发布）**
- 100亿级海量存储篇：从0到1，从入门到 HABSE 工业级使用
- 100亿级离线计算篇：从0到1，从入门到 Flink 工业级使用

已经发布的文章包括：

[字节面试：百亿级存储，怎么设计？只是分库分表？](#)

[100亿级任务调度篇：从0到1，从入门到 XXLJOB 工业级使用](#)

[高并发搜索ES圣经：从0到1，从入门到 Elasticsearch 工业级使用](#)

参考链接

[阿里2面：万亿级消息，如何做存储设计？](#)

<https://yetanotherdevblog.com/lsm/>

<https://gitee.com/bison-fork/loki/blob/v2.2.1/production/docker-compose.yaml>

SkyWalking官网 <http://skywalking.apache.org/zh/>

SkyWalking的docker github地址 <https://github.com/apache/sky...>

elasticsearch <https://www.elastic.co/guide/...>

skywalking中文文档 <https://skyapm.github.io/docu...>

agent config <https://github.com/apache/sky...>

[skywalking和其它agent一起使用的处理](#)

<https://zhuanlan.zhihu.com/p/163809795>

<https://www.cnblogs.com/you-men/p/14900249.html>

<https://cloud.tencent.com/developer/article/1684909>

<https://www.cnblogs.com/javaadu/p/11742605.html>

<https://www.jianshu.com/p/2fa99bd1997e>

https://blog.csdn.net/weixin_42073629/article/details/106775584

<https://www.cnblogs.com/kebibuluan/p/14466285.html>

https://blog.csdn.net/weixin_42073629/article/details/106775584

https://blog.csdn.net/Jerry_wo/article/details/107937902

<https://www.cnblogs.com/wzxmt/p/11031110.html>

<https://blog.csdn.net/zhangshng/article/details/104558016>

https://blog.csdn.net/yurun_house/article/details/109025588

https://blog.csdn.net/weixin_40228200/article/details/123930498

https://blog.csdn.net/lanxing_huangyao/article/details/119795303

<https://www.codenong.com/pzlong372468585/>

<https://blog.csdn.net/clz1314521/article/details/123727401>

<https://my.oschina.net/1Gk2fdm43/blog/5312538>

https://www.sohu.com/a/423360258_411876

<https://blog.csdn.net/liuxiao723846/article/details/124576811>

<https://www.163.com/dy/article/FQ1NNUE805381TA2.html>

<https://www.cnblogs.com/xstCoding/p/16353730.html>

https://blog.csdn.net/qq_21383435/article/details/122813582

<https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/replacingmergetree#replacingmergetree>

[HBase基准性能测试报告 - 网易数帆 - 博客园 \(cnblogs.com\)](#)